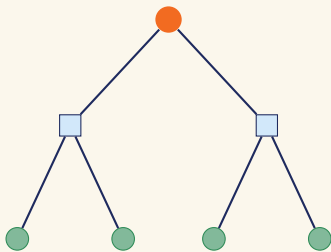
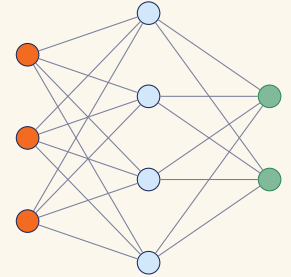
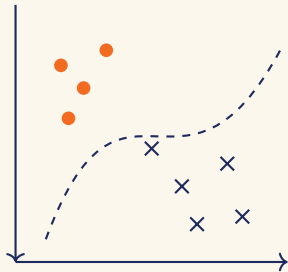


CS229

MACHINE

LEARNING

HANDNOTES



Kompaktfassung: die Übersichtsseiten zum Wiederholen



40 Kapitelseiten

Übersicht je Thema



Spickzettel

Algorithmen & Formeln



Beispiele

mit Rechnung & Lösung



Selbsttests

drei Fragen pro Seite

Andrew Ng & Tengyu Ma

Vorlesungsnotizen & Ergänzungsmaterial (notes, extra-notes, materials, section)

Lernen. Verstehen. Umsetzen. Wachsen.



Kompaktfassung: nur die
Übersichtsseiten. Alle Herlei-
tungen und Code-Beispiele
stehen in der Langfassung.



Inhalt & Übersicht

Grundlagen, Überwachtes Lernen, Deep Learning, Praxis

Merke:

Erst Grundlagen, dann
Modelle, dann Praxis, dann
Herleitungen

0 Grundlagen

01 Wahrscheinlichkeit & Statistik 4

02 Lineare Algebra 5

03 Optimierung 6

I Überwachtes Lernen

01 Lineare Regression 7

02 Logistische Regression 8

03 Generalisierte Lineare Modelle 9

04 Generative Modelle (GDA, NB) 10

05 Kernel-Methoden 11

06 Support Vector Machines 12

07 Perceptron & Online Learning 13

08 k-Nearest Neighbors 14

09 Decision Trees 15

10 Ensemble-Methoden 16

11 Verlustfunktionen 17

II Deep Learning

12 Neuronale Netze 18

13 Backpropagation 19

14 DL-Praxis: Training 20

15 CNN & RNN / LSTM 21

16 Foundation Models 22

III Generalisierung & Praxis

17 Bias & Varianz 23

18 Lerntheorie 24

19 Regularisierung 25

20 Validierung & Tuning 26

21 Gaussian Processes 27

22 Fehleranalyse 28

23 Klassifikationsmetriken 29

24 Regressionsmetriken 30

25 Feature Importance & SHAP 31

26 Fairness-Analyse 32

27 Feature Engineering 33

28 ML-Workflow 34

Inhalt & Landkarte

Weiter: unüberwachtes und bestärkendes Lernen, Spickzettel und die Landkarte des Kurses. ❤️

Teil IV--V, Anhang, roter Faden

Merke:
Modell wählen → Verlust → optimieren → prüfen!

IV Unüberwachtes Lernen

29	K-Means	35	34	PCA	40
30	Hierarchisches Clustering	36	35	ICA	41
31	EM-Algorithmus	37	36	Generative Modelle: VAE, GAN	42
32	Faktorenanalyse	38	37	Kritische Perspektiven	43
33	Hidden Markov Models	39			

V Bestärkendes Lernen

38	MDPs & Value Iteration	44
39	LQR, DDP & LQG	45
40	Policy Gradient	46

A Anhang: Spickzettel

31	Algorithmen-Tabelle	47
32	Formel-Spickzettel	48

* Der ML-Workflow (roter Faden)



* Drei Lernparadigmen

- **Überwacht** (x, y) : Vorhersage lernen (Regression, Klassifikation).
- **Unüberwacht** x : Struktur finden (Cluster, Dimensionsreduktion, Sequenzen, Generative Modelle).
- **Bestärkend**: Aktionen wählen, um Belohnung zu maximieren.

* So liest du diese Notizen

- **Beispiel** mit Rechnung & **Lösung** (alle Zahlen maschinell geprüft)
- **Pseudo-Code**, **Merke-Zettel**, Key Takeaways, **Selbsttest**
- Kompaktfassung: Herleitungen und Code stehen in der Langfassung.
- **Zusatzinfo (Internet)** = nicht aus den Notes; Quelle: unter jedem Titel

Σ Notation: was die Symbole überall bedeuten

n Anzahl Beispiele, d Anzahl Merkmale, $x^{(i)} \in \mathbb{R}^d$ Merkmalsvektor, $y^{(i)}$ Label des i -ten Beispiels, $X \in \mathbb{R}^{n \times d}$ Design-Matrix (eine Zeile pro Beispiel), θ Modellparameter, $h_{\theta}(x)$ bzw. $\hat{y}$ Vorhersage, $J(\theta)$ Kosten (zu minimieren), $\ell(\theta)$ Log-Likelihood (zu maximieren), α Lernrate, λ Regularisierungsstärke, ∇ Gradient (Vektor der Ableitungen), $\mathbb{E}$ Erwartungswert, $\mathcal{N}(\mu, \Sigma)$ Normalverteilung, $\mathbf{1}\{\cdot\}$ Indikator (1, wenn wahr), σ Sigmoid bzw. Standardabweichung (je nach Kontext), A^T transponiert, $\|\cdot\|$ Länge (Norm), $\odot$ elementweises Produkt, $\mathcal{H}$ Hypothesenraum, K Anzahl Klassen bzw. Cluster. Wird ein Symbol lokal anders benutzt, steht eine Legende direkt im Panel.

„Modell, Verlust, Optimierung -- die drei Fragen hinter jedem Algorithmus dieser Vorlesung.“

Fast jeder ML-Algorithmus ist angewandte Wahrscheinlichkeitsrechnung. ❤️

Grundlagen: Wahrscheinlichkeit

Bayes, Erwartungswerte, MLE & MAP, Entropie

Merke:
ML = Wahrscheinlichkeit + Optimierung

Quelle: Vorwissen zu CS229; angelehnt an section/cs229-prob.pdf, section/gaussians.pdf sowie hand-notes/ (Conditional Probability)

1 Grundregeln

Summenregel: $P(A) = \sum_b P(A, B=b)$
(Marginalisieren)

Produktregel: $P(A, B) = P(A|B)P(B)$

Bedingt: $P(A|B) = \frac{P(A \cap B)}{P(B)}$, $P(B) > 0$

Bayes: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

2 Unabhängigkeit

$A \perp\!\!\!\perp B \iff P(A, B) =$

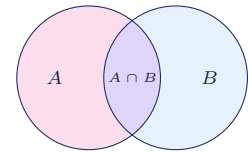
$P(A)P(B) \iff P(A|B) = P(A).$

Bedingt unabhängig: $A \perp\!\!\!\perp B \mid C \iff$

$P(A, B|C) = P(A|C)P(B|C)$ -

Grundannahme von **Naive Bayes**
(Features unabhängig gegeben die Klasse).

3 Skizze: Venn-Diagramm



$P(A|B) = \text{Anteil von } A \cap B \text{ in } B$

4 Erwartungswert, Varianz, Kovarianz

$\mathbb{E}[X] = \sum_x xP(x)$ bzw. $\int xp(x)dx$

$\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$

Linearität

$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$

$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$, $\rho = \frac{\text{Cov}}{\sigma_X \sigma_Y}$

Unabhängig $\Rightarrow \text{Cov} = 0$ (umgekehrt nicht!). Mittelwert von n iid: $\text{Var}(\bar{X}) = \sigma^2/n$.

5 Wichtige Verteilungen

Name	Träger / Dichte	$\mathbb{E}$, Var	Einsatz
Bernoulli(ϕ)	$\phi^y(1-\phi)^{1-y}$	ϕ , $\phi(1-\phi)$	LogReg
Multinomial	K Klassen		Softmax
Poisson(λ)	$\lambda^y e^{-\lambda} / y!$	λ , λ	Zählraten
Gauß	$\frac{e^{-(y-\mu)^2/2\sigma^2}}{\sqrt{2\pi}\sigma}$	μ , σ^2	Rauschen
$\mathcal{N}(\mu, \sigma^2)$			
Mehrdim. Gauß	$\mathcal{N}(\mu, \Sigma)$	μ , Σ	GDA, GP

Beispiel: Bayes beim Medizintest

Prävalenz 1%, Sensitivität 95%, Spezifität 90%. Wie wahrscheinlich ist man krank bei positivem Test?

Lösung $P(+) = 0.95 \cdot 0.01 + 0.10 \cdot 0.99 = 0.1085$.

$P(\text{krank}|+) = \frac{0.95 \cdot 0.01}{0.1085} \approx 8.8\%$ - trotz gutem Test klein, weil die Krankheit selten ist (Base-Rate-Fehler). Genau das passiert bei Klassenungleichgewicht.

6 Maximum Likelihood (MLE)

Wähle θ , das die Daten am wahrscheinlichsten macht:

$$\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^n \log p(x^{(i)}; \theta)$$

(Log: Produkt $\rightarrow$ Summe, gleiches Maximum.) Beispiel Bernoulli, k Erfolge in n : $\ell = k \log \phi + (n-k) \log(1-\phi)$,

$\ell' = \frac{k}{\phi} - \frac{n-k}{1-\phi} = 0 \Rightarrow \hat{\phi} = \frac{k}{n}$.

Gauß: $\hat{\mu} = \frac{1}{n} \sum x_i$, $\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \hat{\mu})^2$.

7 MAP & Bayes

$$p(\theta|D) \propto p(D|\theta)p(\theta)$$

MAP: $\arg \max [\log p(D|\theta) + \log p(\theta)] =$
MLE + Regularisierer (Gauß-Prior $\rightarrow$ L2, Laplace $\rightarrow$ L1).

8 Entropie, KL, Cross-Entropy

$$H(p) = - \sum p \log p,$$

$$H(p, q) = - \sum p \log q$$

$$\text{KL}(p||q) = H(p, q) - H(p) \geq 0$$

Cross-Entropy-Verlust = MLE: minimiert
KL zwischen Daten und Modell.

9 LLN & CLT

Gesetz der großen Zahlen: $\bar{X}_n \rightarrow \mathbb{E}X$.

Zentraler Grenzwertsatz:

$\sqrt{n}(\bar{X}_n - \mu) \rightarrow \mathcal{N}(0, \sigma^2)$ - warum
Rauschen oft gaußsch ist und
Hoeffding/Konzentration wirkt.

? Selbsttest: kannst du das erklären?

F1 Was besagt der Satz von Bayes?

A: Posterior = Likelihood $\times$ Prior /
Evidenz.

F2 Wie lautet der MLE-Schätzer für ϕ
bei k Erfolgen in n Versuchen?

A: $\hat{\phi} = k/n$.

F3 Was ist der Zusammenhang MAP $\leftrightarrow$
Regularisierung?

A: Log-Prior wirkt als Strafterm: Gauß
 $\rightarrow$ L2, Laplace $\rightarrow$ L1.

„Alles Lernen ist Schließen unter Unsicherheit -- Wahrscheinlichkeit ist die Sprache dafür.“

Daten sind Matrizen, Modelle sind Abbildungen: die Sprache hinter allen Formeln.



Grundlagen: Lineare Algebra

Vektoren, Matrizen, Eigenwerte & Matrixableitungen

Merke:
Kein Gradient ohne
Matrixableitung!

Quelle: Vorwissen zu CS229; angelehnt an section/cs229-linalg.pdf sowie main_notes.pdf (Normalgleichung, PCA)

1 Vektoren & Normen

Skalarprodukt

$$x^T y = \sum_i x_i y_i = \|x\| \|y\| \cos \varphi.$$

$$\|x\|_2 = \sqrt{x^T x}, \quad \|x\|_1 = \sum |x_i|.$$

$x \perp y \iff x^T y = 0$. Projektion von y auf u ($\|u\|=1$): $(u^T y) u$.

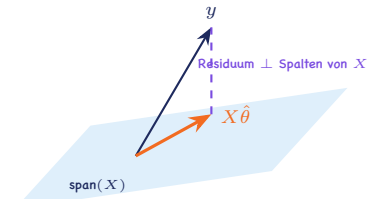
2 Matrizen: Rechenregeln

$$(AB)^T = B^T A^T, \quad (AB)^{-1} = B^{-1} A^{-1}$$

$$\text{tr}(AB) = \text{tr}(BA), \quad \text{tr}(A) = \sum \lambda_i$$

Design-Matrix $X \in \mathbb{R}^{n \times d}$: eine Zeile pro Beispiel; $X\theta$ liefert alle Vorhersagen auf einmal.

3 Skizze: Least Squares = Projektion



4 Eigenwerte, PSD, SVD

$Av = \lambda v$. Für symmetrische A : reelle λ , orthonormale

Eigenvektoren, $A = U \Lambda U^T$.

Positiv semidefinit (PSD): $x^T A x \geq 0 \forall x \iff \lambda_i \geq 0$

(Kovarianzmatrizen, Gram-Matrizen, konvexe Hesse-Matrizen).

SVD: $X = U \Sigma V^T$; Singulärwerte² = Eigenwerte von $X^T X \Rightarrow$ PCA.

5 Matrixableitungen (Gradient nach x)

Funktion	Gradient
$a^T x$	a
$x^T A x$	$(A + A^T)x$, symmetrisch: $2Ax$
$\ Xx - y\ ^2$	$2X^T(Xx - y)$
$\log \det A$	A^{-T}
$\ x\ _2^2$	$2x$

Beispiel: Eigenwerte von $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$

Lösung $\det(A - \lambda I) = (2 - \lambda)^2 - 1 = 0 \Rightarrow \lambda_1 = 3, \lambda_2 = 1$.

Eigenvektoren $v_1 = \frac{1}{\sqrt{2}}(1, 1)$, $v_2 = \frac{1}{\sqrt{2}}(1, -1)$ (orthogonal). Beide

$\lambda > 0 \Rightarrow A$ ist PSD, ja sogar positiv definit.

Als Kovarianzmatrix: Ellipse mit Achsen v_1 (Länge $\propto \sqrt{3}$) und v_2 ($\propto 1$) – genau die Hauptkomponenten.

6 Kovarianz & Gauß-Ellipsen

$\Sigma = \mathbb{E}[(x - \mu)(x - \mu)^T]$ ist symmetrisch PSD. Die Höhenlinien von $\mathcal{N}(\mu, \Sigma)$ sind Ellipsen: Richtungen = Eigenvektoren, Streuung $\propto \sqrt{\lambda_i}$. Mahalanobis-Abstand: $(x - \mu)^T \Sigma^{-1} (x - \mu)$. Σ singulär $\iff$ ein $\lambda = 0$ (Daten liegen im Unterraum $\rightarrow$ Faktorenanalyse).

7 Numerik

- `solve(A,b)` statt `inv(A)@b`
- Konditionszahl $\lambda_{\max}/\lambda_{\min}$ groß $\rightarrow$ instabil (Features skalieren)
- Regularisierung: $X^T X + \lambda I$

8 Typische Fehler

- × Dimensionen vertauschen ($X^T X$ vs. XX^T)
- × $(AB)^T = A^T B^T$ (falsch!)
- × Inverse ohne Rangprüfung

9 Wo es gebraucht wird

- Normalgleichung, GDA, Kernel-Gram-Matrix
- PCA, Faktorenanalyse, GP
- Backprop (Jacobi-Matrizen)

? Selbsttest: kannst du das erklären?

F1 Was ist der Gradient von $\|X\theta - y\|^2$?

A: $2X^T(X\theta - y)$.

F2 Woran erkennt man eine PSD-Matrix?

A: Alle Eigenwerte ≥ 0 , äquivalent $x^T A x \geq 0$.

F3 Warum ist $X^T X$ immer PSD?

A: $v^T X^T X v = \|Xv\|^2 \geq 0$.

„Wer Matrixableitungen beherrscht, sieht in jedem Lernalgorithmus dieselbe Gleichung.“

Grundlagen: Optimierung

Konvexität, Gradient Descent, Newton & Lagrange

Merke:
Konvexes Problem: jedes lokale Minimum ist global!

Quelle: Vorwissen zu CS229; angelehnt an section/cs229-cvopt.pdf, section/cs229-cvopt2.pdf und hand-notes/ML Notes.pdf (Gradient Descent)

1 Konvexität

f konvex:

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y).$$

Äquivalent: Hesse $\nabla^2 f \succeq 0$.

Lokales Minimum = globales Minimum.

Konvex: Least Squares, logistische Verluste, SVM. Nicht konvex: neuronale Netze, K-Means, EM.

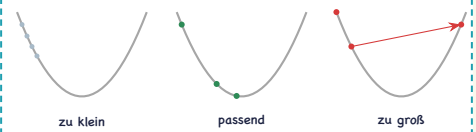
2 Gradient Descent

Bergab entlang des negativen Gradienten:

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

α = Lernrate. Der Gradient zeigt in Richtung des steilsten Anstiegs.

3 Skizze: Lernrate



4 Varianten & Algorithmus

`</>` Gradient Descent (Batch / SGD / Mini-Batch)

init θ ; **repeat** bis Konvergenz:

wähle Datenmenge B :

Batch: alle n ; SGD: 1 Beispiel; Mini-Batch: B

$$g \leftarrow \frac{1}{|B|} \sum_{i \in B} \nabla J^{(i)}(\theta)$$

$$\theta \leftarrow \theta - \alpha g$$

Batch: stabil, teuer. SGD: billig, verrauscht (entkommt flachen Stellen). Mini-Batch: Standard im Deep Learning.

5 Konvergenz & Praxis

Für $J(\theta) = \frac{1}{2} \theta^T H \theta$ konvergiert GD, wenn $\alpha < 2/\lambda_{\max}(H)$; Geschwindigkeit hängt an der Konditionszahl $\lambda_{\max}/\lambda_{\min}$.

Features standardisieren $\Rightarrow$ rundere Höhenlinien $\Rightarrow$ schnellere Konvergenz. Lernrate wählen: erst groß, dann abklingen lassen (Schedules); Verlust über die Iterationen plotten.

Beispiel: GD auf $f(x) = x^2, x_0 = 4$

Gradient $f'(x) = 2x$, Update $x \leftarrow x - 2\alpha x = (1 - 2\alpha)x$.

Lösung $\alpha = 0.1$: $x_k = 4 \cdot 0.8^k$: 3.2, 2.56, 2.05, $\dots \rightarrow 0$

konvergiert.

$\alpha = 1.1$: Faktor -1.2 : $x_1 = -4.8, x_2 = 5.76, \dots$ **divergiert**

($|1 - 2\alpha| > 1$).

$\alpha = 0.5$: $x_1 = 0$ - in einem Schritt am Ziel (nur bei diesem Quadrat).

6 Newton-Verfahren

Zweite Ordnung: $\theta \leftarrow \theta - H^{-1} \nabla J$ (H = Hesse). Für quadratische J in einem Schritt am Ziel; sonst quadratische Konvergenz nahe dem Optimum. Kosten $O(d^3)$ pro Schritt. Beispiel $f = x^2 - 4x$: $f' = 2x - 4, f'' = 2, x_1 = x_0 - \frac{2x_0 - 4}{2} = 2$ (Minimum) für jedes x_0 .

7 Lagrange & KKT (Nebenbedingungen)

$\min f(x)$ s.t. $g(x) \leq 0$: $\mathcal{L} = f + \alpha g, \alpha \geq 0$. KKT: Zulässigkeit, $\alpha \geq 0$, Complementary Slackness $\alpha g = 0, \nabla_x \mathcal{L} = 0$.

Beispiel $\min x^2$ s.t. $1 - x \leq 0$: $\mathcal{L} = x^2 + \alpha(1 - x), x = \alpha/2$, Dual $g(\alpha) = \alpha - \alpha^2/4$, max bei $\alpha^* = 2, x^* = 1$, Wert 1 (Primal = Dual).

8 Nicht konvex

- lokale Minima, Sattelpunkte
- SGD-Rauschen + gute Initialisierung helfen
- Neustarts (K-Means, EM)

9 Nicht differenzierbar

- $|x|$ (L1), Hinge, ReLU: Subgradient
- $\partial|x| = \{-1\}, [-1, 1], \{1\}$

10 Wo es gebraucht wird

- alle Trainingsverfahren
- SVM-Dual (KKT), EM (ELBO)
- Regularisierung, Backprop

? Selbsttest: kannst du das erklären?

F1 Wann ist ein lokales Minimum global?

A: Wenn die Zielfunktion konvex ist.

F2 Warum standardisiert man Features vor Gradient Descent?

A: Rundere Höhenlinien, kleinere Konditionszahl, schnellere und stabilere Konvergenz.

F3 Was sagt Complementary Slackness?

A: $\alpha_i g_i = 0$: entweder Nebenbedingung aktiv oder Multiplikator 0.

„Bergab gehen ist einfach -- die Kunst ist, die richtige Schrittweite zu wählen.“

Lineare Regression

Die Grundlage aller Vorhersagemodelle

Ziel: eine Gerade so durch die Datenwolke legen, dass sie y möglichst gut vorhersagt.



Merke:
Least Squares = Maximum Likelihood bei Gauß-Rauschen!

1 Idee & Modell

Gegeben: Daten $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ mit $x \in \mathbb{R}^d, y \in \mathbb{R}$.

Gesucht: Hypothese $h_\theta(x) \approx y$.

$$h_\theta(x) = \theta^\top x = \sum_{j=0}^d \theta_j x_j, \quad x_0 := 1$$

$x_0 = 1$ ist der **Bias-Term**

(Achsenabschnitt).

Design-Matrix $X \in \mathbb{R}^{n \times d}$: eine Zeile pro Beispiel, $y \in \mathbb{R}^n$.

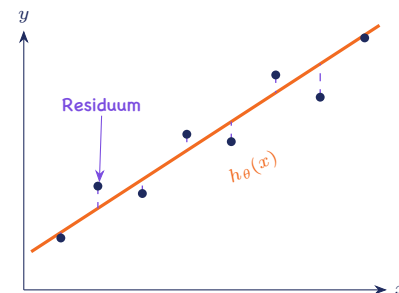
2 Kostenfunktion (MSE)

Wie schlecht ist θ ? Summe der quadrierten Fehler:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_\theta(x^{(i)}) - y^{(i)})^2 = \frac{1}{2n} \|X\theta - y\|^2$$

Konvex (Schüssel-Form): genau ein globales Minimum, keine lokalen Fallen.

3 Skizze: Residuen



4 Gradient Descent

Ableiten und bergab laufen (Lernrate α):

$$\theta_j \leftarrow \theta_j + \frac{\alpha}{n} \sum_{i=1}^n (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

Batch: alle n Punkte pro Schritt. **SGD:** 1 Punkt (oder Mini-Batch) pro Schritt – billiger, verrauschter.

`</> Batch-Gradient-Descent`

`init` $\theta \leftarrow 0$

`repeat` bis Konvergenz:

`g` $\leftarrow \frac{1}{n} X^\top (X\theta - y)$

$\theta \leftarrow \theta - \alpha g$

SGD: für jedes i in zufälliger Reihenfolge

$\theta \leftarrow \theta + \alpha (y_i - \theta^\top x_i) x_i$

Tipp: Features skalieren; α zu groß $\Rightarrow$ Divergenz, zu klein $\Rightarrow$ langsam.

5 Normalgleichung (geschlossene Lösung)

$\nabla_\theta J = 0$ setzen ergibt in einem Schritt:

$$\hat{\theta} = (X^\top X)^{-1} X^\top y$$

Gut für kleines/mittleres d (Inversion kostet $O(d^3)$). Bei sehr großem d oder singulärem $X^\top X$: Gradient Descent oder Regularisierung.

`</> Numpy-Einzeiler`

`theta = solve(X.T @ X, X.T @ y)`

besser als explizite Inverse (stabiler)

6 Probabilistische Sicht

Annahme: $y = \theta^\top x + \varepsilon, \varepsilon \sim \mathcal{N}(0, \sigma^2)$ iid. Dann ist

$$\log L(\theta) = \text{const} - \frac{1}{2\sigma^2} \sum_i (y^{(i)} - \theta^\top x^{(i)})^2$$

$\Rightarrow$ **MLE = Least Squares.**

7 Lokal gewichtete Regression

Nichtparametrisch: für jede Anfrage x neu fitten, nahe Punkte zählen mehr:

$$w^{(i)} = \exp\left(-\frac{\|x^{(i)} - x\|^2}{2\tau^2}\right)$$

Minimiere $\sum_i w^{(i)} (y^{(i)} - \theta^\top x^{(i)})^2$.

Bandbreite τ : klein $\rightarrow$ wackelig, groß $\rightarrow$ glatt.

Beispiel: 3 Punkte per Hand

Aufgabe Punkte $(1, 1), (2, 2), (3, 2)$. Finde $\hat{\theta}$ und sage $x = 4$ vorher.

Lösung $X^\top X = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}, X^\top y = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$,

$\det = 6$

$$\hat{\theta} = \frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix} \begin{pmatrix} 5 \\ 11 \end{pmatrix} = \begin{pmatrix} 2/3 \\ 1/2 \end{pmatrix}$$

$$h(4) = \frac{2}{3} + \frac{1}{2} \cdot 4 \approx \mathbf{2.67}$$

8 Vorteile

- ✓ einfach, schnell, gut interpretierbar
- ✓ geschlossene Lösung existiert
- ✓ konvexes Problem: keine lokalen Minima
- ✓ solide Baseline für jedes Regressionsproblem

9 Grenzen

- ✗ nur lineare Zusammenhänge (Abhilfe: Feature-Maps, Kernel)
- ✗ empfindlich gegen Ausreißer (quadr. Fehler)
- ✗ $X^\top X$ evtl. singulär (kollineare Features)
- ✗ Overfitting bei sehr vielen Features

10 Key Takeaways

- ✓ $h_\theta(x) = \theta^\top x$, Kosten = MSE
- ✓ Lösung: Normalgleichung oder (S)GD
- ✓ MLE unter Gauß-Rauschen
- ✓ LWLR: nichtparametrische Erweiterung

11 Polynom-Features & Anwendungen

Auch Kurven sind „linear“: $\phi(x) = (1, x, x^2, \dots, x^k)$, $h_\theta(x) = \theta^\top \phi(x)$. Linear in θ , nichtlinear in x . Der Grad k steuert die

Modellkomplexität $\rightarrow$ Bias/Varianz. Anwendungen:

Preis-/Umsatzprognose, Trends, Baseline für komplexere Modelle.

12 Annahmen (aus den ML Notes)

Linearität, Unabhängigkeit, Homoskedastizität (konstante Fehlervarianz), normalverteilte Fehler (Basis der MLE-Sicht), keine starke Multikollinearität. Residuenplot: Muster $\Rightarrow$ nichtlinear, Trichter $\Rightarrow$ Heteroskedastizität. Metriken: Kapitel Regressionsmetriken.

? Selbsttest: kannst du das erklären?

F1 Warum ist $J(\theta)$ konvex?

A: Quadratisch in θ (Hesse $X^\top X/n \succeq 0$): nur ein globales Minimum.

F2 Wann versagt die Normalgleichung?

A: Bei sehr großem d ($O(d^3)$) oder singulärem $X^\top X$: dann GD oder Regularisierung.

F3 Wirkung von τ bei LWLR?

A: Bandbreite: klein $\rightarrow$ sehr lokal/wackelig, groß $\rightarrow$ glatt (globale lineare Regression).

„Ein einfaches Modell, das man wirklich verstanden hat, schlägt ein kompliziertes, das man nur benutzt.“

Logistische Regression

Von Zahlen zu Wahrscheinlichkeiten

Klassifikation: die Ausgabe ist eine Wahrscheinlichkeit zwischen 0 und 1. ♥

Merke: Die update-Regel sieht aus wie LMS -- ist aber ein anderes Modell!

1 Idee

Binäre Klassifikation, $y \in \{0, 1\}$. Ein lineares Modell taugt nicht ($\theta^\top x \notin [0, 1]$).

Lösung: **Sigmoid** quetscht alles in $(0, 1)$:

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_\theta(x) = g(\theta^\top x) = P(y=1 | x; \theta)$$

2 Likelihood

$$P(y=1|x) = h_\theta(x),$$

$$P(y=0|x) = 1 - h_\theta(x). \text{ Kompakt:}$$

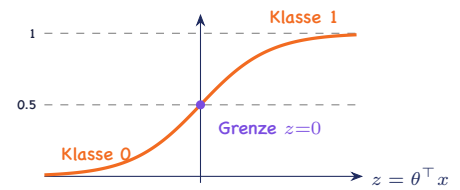
$$P(y|x) = h^y (1 - h)^{1-y}.$$

$$\ell(\theta) = \sum_i [y^{(i)} \log h^{(i)} + (1 - y^{(i)}) \log(1 - h^{(i)})]$$

Maximiere ℓ (Maximum Likelihood) = minimiere die **Cross-Entropy**.

Legende: n Beispiele, $h^{(i)} = h_\theta(x^{(i)})$
Vorhersage für Beispiel i , ℓ Log-Likelihood.

3 Skizze: Sigmoid



4 Gradient Ascent

Ableiten liefert (mit $g' = g(1 - g)$) die einfache Regel

$$\theta_j \leftarrow \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

Legende: α Lernrate, $x_j^{(i)}$ Merkmal j von Beispiel i , $y^{(i)} - h$ Fehler der Vorhersage.

Form identisch zur LMS-Regel, aber h ist nichtlinear. (Grund: GLM-Struktur.)

`</> Logistische Regression (SGD)`

init $\theta \leftarrow 0$

repeat bis Konvergenz:

for i in zufälliger Reihenfolge:

$$h \leftarrow 1/(1 + e^{-\theta^\top x_i})$$

$$\theta \leftarrow \theta + \alpha (y_i - h) x_i$$

predict: Klasse 1 falls $\theta^\top x \geq 0$

5 Newton-Verfahren (Fisher Scoring)

Nutzt Krümmung, konvergiert **quadratisch**:

$$\theta \leftarrow \theta - H^{-1} \nabla_\theta \ell, \quad H = -X^\top S X, \quad S = \text{diag}(h_i(1 - h_i))$$

Legende: H Hesse-Matrix (Krümmung), $\nabla_\theta \ell$ Gradient, S Gewichtsmatrix, d Anzahl Merkmale.

Wenige Iterationen (oft < 10), aber jeder Schritt kostet $O(d^3)$.

`</> Newton / IRLS`

repeat bis Konvergenz:

$$h \leftarrow \sigma(X\theta)$$

$$S \leftarrow \text{diag}(h \odot (1 - h))$$

$$\theta \leftarrow \theta + (X^\top S X)^{-1} X^\top (y - h)$$

6 Softmax (Mehrklassen)

K Klassen, je ein Parametervektor θ_k (j läuft über alle Klassen):

$$P(y=k|x) = \frac{e^{\theta_k^\top x}}{\sum_{j=1}^K e^{\theta_j^\top x}}$$

Verlust: $-\log P(y|x)$ (Cross-Entropy). Für $K = 2$: wieder Sigmoid.

7 Entscheidungsgrenze

$\{x : \theta^\top x = 0\}$ ist eine **Hyperebene** $\Rightarrow$

lineares Modell. Schwelle 0.5 ist nur Konvention: höhere Schwelle $\rightarrow$ mehr Precision, niedrigere $\rightarrow$ mehr Recall.

Beispiel: Vorhersage + 1 Update

$$\theta = (-3, 1, 1), x = (1, 2, 3), y = 1.$$

$$z = -3 + 2 + 3 = 2 \Rightarrow h = \sigma(2) \approx 0.88:$$

Klasse 1.

$$\text{Update } (\alpha = 0.5): (y - h) = 0.12$$

$$\Delta\theta = 0.5 \cdot 0.12 \cdot (1, 2, 3) = (0.06, 0.12, 0.18)$$

8 Vorteile

- ✓ liefert Wahrscheinlichkeiten
- ✓ konvex: kein lokales Minimum
- ✓ schnell, robust, gut interpretierbar
- ✓ Baseline für Klassifikation

9 Grenzen

- ✗ nur lineare Grenzen (sonst Features/Kernel)
- ✗ bei perfekt trennbaren Daten $\|\theta\| \rightarrow \infty$ (Regularisieren!)
- ✗ empfindlich bei starkem Klassen-Ungleichgewicht

10 Key Takeaways

- ✓ $h = \sigma(\theta^\top x)$, MLE = Cross-Entropy
- ✓ SGD-Regel: $\theta \leftarrow \theta + \alpha(y - h)x$
- ✓ Newton: wenige, teure Schritte
- ✓ Softmax = Mehrklassen-Verallgemeinerung

11 Praxis, Anwendungen & Querverweise

Tipps: Features skalieren, L2-Regularisierung gegen $\|\theta\| \rightarrow \infty$, bei Klassen-Ungleichgewicht Gewichte oder Schwelle anpassen, Wahrscheinlichkeiten auf Kalibrierung prüfen. **Anwendungen:** Spam- und Betrugserkennung, Risikoscores, Klickvorhersage, Kreditausfall. **Siehe auch:** GLMs (warum die Regel wie LMS aussieht), SVM (anderer Verlust), Neuronale Netze (letzte Schicht = Softmax).

? Selbsttest: kannst du das erklären?

F1 Warum hat die SGD-Regel die Form $(y - h)x$?

A: Gradient der Log-Likelihood;
 $g' = g(1 - g)$ kürzt sich heraus.

F2 Was passiert bei perfekt trennbaren Daten?

A: Likelihood wächst weiter, $\|\theta\| \rightarrow \infty \Rightarrow$ Regularisierung nötig.

F3 Vorteil von Newton gegenüber GD?

A: Quadratische Konvergenz, wenige Iterationen; Nachteil: $O(d^3)$ pro Schritt.

„Logistische Regression ist ein lineares Modell mit einem Wahrscheinlichkeits-Filter davor.“

Ein Rezept für viele Modelle:
Verteilung wählen, Modell
folgt automatisch. ❤️

Generalisierte Lineare Modelle

Regression und Klassifikation unter einem Dach

Merke:
Lineare, logistische und
Poisson-Regression sind alle
GLMs!

1 Idee

Statt für jedes Problem ein neues
Modell zu erfinden:

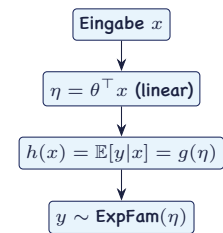
Wähle die zu y passende Verteilung

(Zahl, Ja/Nein, Zähldaten...), der Rest
ergibt sich. Das erklärt, warum die
Update-Regeln so ähnlich aussehen.

2 Exponentialfamilie

$p(y; \eta) = b(y) \exp(\eta^\top T(y) - a(\eta))$
 η : natürlicher Parameter, $T(y)$:
hinreichende Statistik, $a(\eta)$:
Log-Partitionsfunktion, $b(y)$: Basismaß.
Gauß, Bernoulli, Poisson, Multinomial
u. v. m. gehören dazu.

3 Skizze: Pipeline



4 Die drei GLM-Annahmen

1. $y | x; \theta \sim \text{ExpFam}(\eta)$
2. Hypothese: $h(x) = \mathbb{E}[T(y) | x; \theta]$
3. Linearer Zusammenhang: $\eta = \theta^\top x$

Die **Responsefunktion** $g(\eta) = \mathbb{E}[T(y); \eta]$ verbindet η mit dem
Mittelwert; ihre Inverse heißt **Linkfunktion**. Das Lernen
(Maximum Likelihood) ergibt für alle GLMs dieselbe Form (α
Lernrate, $h_\theta(x) = \mathbb{E}[y|x]$ Vorhersage):

$$\theta \leftarrow \theta + \alpha(y - h_\theta(x))x$$

5 Bekannte Vertreter

Verteilung	Response	Modell
Gauß	Identität	Lineare Regression
Bernoulli	Sigmoid	Logistische Regression
Multinomial	Softmax	Softmax-Regression
Poisson	e^η	Poisson-Regression (Zählda- ten)

6 Rezept: GLM bauen

GLM in 4 Schritten

1. Verteilung passend zu y wählen
(reell $\rightarrow$ Gauß, 0/1 $\rightarrow$ Bernoulli,
Zählwert $\rightarrow$ Poisson, K Klassen $\rightarrow$ Multinomial)
2. In Exponentialform bringen: η, T, a, b ablesen
3. $h(x) = g(\eta)$ mit $\eta = \theta^\top x$
4. Log-Likelihood maximieren (SGD):
 $\theta \leftarrow \theta + \alpha(y - h(x))x$

Beispiel: Poisson-Regression (Zähldaten)

Aufgabe Besucher pro Stunde. Modell $\theta = (0.5, 0.4)$, Feature
 $x = (1, 2)$ (z. B. Temperatur-Score). Wie viele Besucher
erwartet man?

Lösung $\eta = \theta^\top x = 0.5 + 0.8 = 1.3$, Response

$$h = e^\eta = e^{1.3} \approx 3.67.$$

Warum e^η ? Poisson: $p(y; \lambda) = \lambda^y e^{-\lambda} / y!$ mit $\eta = \log \lambda \Rightarrow$
 $\lambda = e^\eta > 0$ (Zählwerte sind nie negativ).

7 Vorteile

- ✓ einheitliche Theorie & Update-Regel
- ✓ passende Verteilung $\Rightarrow$ bessere
Modelle
- ✓ Ausgaben immer im gültigen Bereich

8 Grenzen

- ✗ Verteilung muss zur
Exponentialfamilie gehören
- ✗ η bleibt linear in x
- ✗ Wahl der Verteilung ist
Modellannahme

9 Key Takeaways

- ✓ Erst Verteilung, dann Modell
- ✓ $\eta = \theta^\top x$, $h = g(\eta)$
- ✓ Alle: $\theta \leftarrow \theta + \alpha(y - h)x$

10 Herleitung: Bernoulli $\rightarrow$ Sigmoid

$$p(y; \phi) = \phi^y (1 - \phi)^{1-y} = \exp(y \log \frac{\phi}{1-\phi} + \log(1 - \phi))$$

$$\text{Ablesen: } \eta = \log \frac{\phi}{1-\phi}, T(y) = y, a(\eta) = \log(1 + e^\eta), b(y) = 1.$$

$$\text{Umstellen: } \phi = \frac{1}{1 + e^{-\eta}} \Rightarrow$$

das ist genau die Sigmoid-Funktion.

Gauß ($\sigma^2=1$): $\eta = \mu$, $T(y) = y$, $a(\eta) = \eta^2/2 \Rightarrow h = \eta$
(Identität).

11 Praktische Eigenschaften

Für $T(y) = y$ gilt: $\mathbb{E}[y; \eta] = a'(\eta)$ und $\text{Var}(y; \eta) = a''(\eta)$.
Die log-Likelihood ist in θ **konkav** $\Rightarrow$ keine lokalen Maxima;
Gradient Ascent oder Newton finden das globale Optimum.
Anwendungen: Zähldaten (Poisson), Wartezeiten (Exponential),
Anteile (Bernoulli/Binomial).

? Selbsttest: kannst du das erklären?

F1 Welche drei Annahmen definieren ein
GLM?

A: $y|x \sim \text{ExpFam}(\eta)$; $h = \mathbb{E}[T(y)|x]$;
 $\eta = \theta^\top x$.

F2 Response bei Poisson-Regression?

A: $h = e^\eta$ (Erwartungswert $\lambda > 0$).

F3 Warum haben alle GLMs dieselbe
Update-Regel?

A: Gradient der Log-Likelihood ist
 $(y - h)x$ bei kanonischer Response.

„Wähle die Verteilung, die zu deinen Daten passt -- den Rest erledigt das GLM.“

Generative Modelle

Statt $p(y|x)$ direkt zu lernen: modelliere, wie die Daten jeder Klasse aussehen. ♥

Gaussian Discriminant Analysis & Naive Bayes

Merke:

Bayes: $p(y|x) \propto p(x|y)p(y)$

1 Diskriminativ vs. generativ

Diskriminativ (LogReg): lernt $p(y|x)$ direkt.
Generativ: lernt $p(x|y)$ und $p(y)$, klassifiziert per Bayes:

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

$$p(x) = \sum_y p(x|y)p(y)$$

2 GDA-Modell

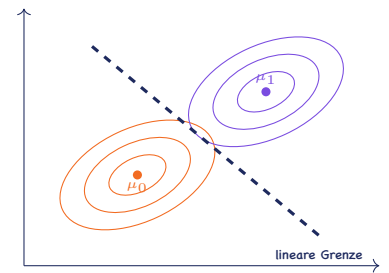
$x|y$ ist multivariat normalverteilt, gemeinsame Kovarianz Σ :

$$y \sim \text{Bernoulli}(\phi)$$

$$x|y=k \sim \mathcal{N}(\mu_k, \Sigma)$$

$$\text{Dichte: } \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)}$$

3 Skizze: zwei Gauß-Wolken



4 GDA: Schätzer (MLE) & Ablauf

Legende: $\mathbf{1}\{\cdot\}$ Indikator (1, wenn wahr), n Anzahl Beispiele, $\hat{\phi}$ Anteil der Klasse 1, $\hat{\mu}_k$ Klassenmittel, $\hat{\Sigma}$ gemeinsame Kovarianz.

$$\hat{\phi} = \frac{1}{n} \sum_i \mathbf{1}\{y^{(i)}=1\}, \quad \hat{\mu}_k = \frac{\sum_i \mathbf{1}\{y^{(i)}=k\} x^{(i)}}{\sum_i \mathbf{1}\{y^{(i)}=k\}}$$

$$\hat{\Sigma} = \frac{1}{n} \sum_i (x^{(i)} - \hat{\mu}_{y^{(i)}})(x^{(i)} - \hat{\mu}_{y^{(i)}})^T$$

Code:

```
train:  $\hat{\phi}, \hat{\mu}_0, \hat{\mu}_1, \hat{\Sigma}$  aus den Daten zählen/mitteln
predict(x):
     $s_k \leftarrow \log \mathcal{N}(x; \hat{\mu}_k, \hat{\Sigma}) + \log p(y=k)$ 
    return argmaxk sk
```

5 Naive Bayes

Naive Annahme: Features sind gegeben y unabhängig:

$$p(x|y) = \prod_{j=1}^d p(x_j|y)$$

Spam-Filter ($x_j \in \{0,1\}$: Wort j vorhanden?, d Wörter, $\hat{\phi}_{j|k} = P(x_j=1|y=k)$) mit **Laplace-Glättung** gegen Nullwahrscheinlichkeiten:

$$\hat{\phi}_{j|k} = \frac{1 + \sum_i \mathbf{1}\{x_j^{(i)}=1, y^{(i)}=k\}}{2 + \sum_i \mathbf{1}\{y^{(i)}=k\}}$$

6 Naive Bayes: Ablauf

Code:

```
train: für jede Klasse k, Feature j:
     $\hat{\phi}_{j|k} \leftarrow (1 + \#\{x_j=1, y=k\}) / (2 + \#\{y=k\})$ 
     $\hat{p}(k) \leftarrow \#\{y=k\} / n$ 
predict(x): return argmaxk
    log  $\hat{p}(k) + \sum_j \log \hat{\phi}_{j|k}$ 
# Summe der Logs statt Produkt: numerisch stabil
```

Beispiel: Spam-Filter mit einem Wort

Aufgabe 10 Trainingsmails: 4 Spam (davon 3 mit „Gratis“), 6 Ham (davon 1 mit „Gratis“). Neue Mail enthält „Gratis“.

Lösung $p(\text{Spam}) = 0.4$; mit Laplace: $P(\text{Gratis}|\text{Spam}) = \frac{3+1}{4+2} = 0.67$,

$$P(\text{Gratis}|\text{Ham}) = \frac{1+1}{6+2} = 0.25.$$

$$\text{Spam: } 0.67 \cdot 0.4 = 0.267, \text{ Ham: } 0.25 \cdot 0.6 = 0.150.$$

$$\text{Normiert: } P(\text{Spam}|x) = 0.267 / 0.417 \approx 0.64 \Rightarrow \text{Spam.}$$

7 GDA vs. LogReg

Ist $p(x|y)$ wirklich Gauß, hat der Posterior immer die logistische Form $1/(1+e^{-\theta^T x})$.
GDA: stärkere Annahmen $\Rightarrow$ bei wenig Daten und richtiger Annahme besser.
LogReg: schwächere Annahmen $\Rightarrow$ robuster, wenn die Gauß-Annahme verletzt ist.

8 Vorteile

- ✓ sehr schnell, kein Iterieren
- ✓ wenig Daten reichen
- ✓ skaliert (Naive Bayes)

9 Grenzen

- ✗ starke Verteilungsannahmen
- ✗ „naiv“: korrelierte Features
- ✗ Wahrscheinlichkeiten oft schlecht kalibriert

10 Key Takeaways & Anwendungen

- ✓ Generativ: $p(x|y)p(y)$ + Bayes
- ✓ GDA: Gauß, gleiches $\Sigma \Rightarrow$ lineare Grenze
- ✓ Naive Bayes: Unabhängigkeit + Laplace-Glättung
- ✓ Rechnen mit Log-Wahrscheinlichkeiten (stabil)

* Naive-Bayes-Varianten & allgemeine Glättung (aus den ML Notes)

Gaussian NB: stetige Features, $x_j|y \sim \mathcal{N}$. **Multinomial NB:** Wortzähler (Text). **Bernoulli NB:** Wort vorhanden ja/nein. Allgemeine Glättung:

$$P(x_i|C) = \frac{\text{count}(x_i, C) + \alpha}{\text{count}(C) + \alpha V} \quad (V \text{ Vokabulargröße, } \alpha=1 \text{ Laplace; für Bernoulli } V=2).$$

Beispiel (Vokabular win, cash, prize, offer): $P(\text{Spam}) = 0.6$, $P(\text{win}|S) = 0.4$, $P(\text{cash}|S) = 0.3$, $P(\text{prize}|S) = 0.2$; Ham: 0.4, 0.1, 0.1, 0.05. Mail "win cash prize": Spam $0.6 \cdot 0.4 \cdot 0.3 \cdot 0.2 = 0.0144$ vs. Ham $0.4 \cdot 0.1 \cdot 0.1 \cdot 0.05 = 0.0002 \Rightarrow$ **Spam**.

? Selbsttest: kannst du das erklären?

F1 Was modelliert GDA (vs. LogReg)?

A: $p(x|y)$ und $p(y)$ (Gauß, gleiches Σ); Klassifikation per Bayes.

F2 Wozu Laplace-Glättung?

A: Verhindert $P=0$ bei ungesesehenen Kombinationen (Zähler +1, Nenner +2).

F3 Wann schlägt LogReg GDA?

A: Wenn die Gauß-Annahme verletzt ist (schwächere Annahmen, robuster).

„Generative Modelle können neue Daten erzeugen – diskriminative nur sortieren.“

Kernel-Methoden

Der Kernel-Trick

Nichtlinear lernen, ohne den hochdimensionalen Feature-Raum je zu berechnen. ❤️

Merke:
Nur Skalarprodukte zählen:
 $K(x, z) = \phi(x)^\top \phi(z)$

1 Feature-Maps

Eine Abbildung $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^p$ macht ein lineares Modell nichtlinear:
 $h(x) = \theta^\top \phi(x)$. Problem: p kann riesig (sogar unendlich) sein $\Rightarrow \phi(x)$ explizit auszurechnen ist zu teuer.

2 Der Kernel-Trick

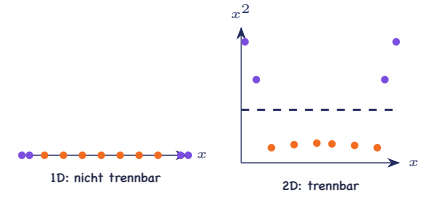
Legende: ϕ Feature-Map, K Kernel, β_i Gewicht von Trainingspunkt i , n Anzahl Trainingspunkte.

Die Lösung ist stets $\theta = \sum_i \beta_i \phi(x^{(i)})$.
Damit

$$h(x) = \sum_{i=1}^n \beta_i \underbrace{\phi(x^{(i)})^\top \phi(x)}_{K(x^{(i)}, x)}$$

$K(x, z)$ oft in $O(d)$ berechenbar, obwohl ϕ riesig ist.

3 Skizze: Anheben in 2D



4 Wichtige Kernels

Kernel	$K(x, z)$	Feature-Raum
Linear	$x^\top z$	$\mathbb{R}^d$
Polynom	$(x^\top z + c)^k$	$O(d^k)$ -dim.
Gauß (RBF)	$\exp(-\frac{\ x-z\ ^2}{2\sigma^2})$	∞ -dim.

RBF: σ klein $\rightarrow$ sehr lokal (Overfitting), σ groß $\rightarrow$ fast linear.

5 Mercers Satz: wann ist K gültig?

K ist genau dann ein Kernel, wenn für beliebige Punkte die **Gram-Matrix** $[K]_{ij} = K(x^{(i)}, x^{(j)})$ **symmetrisch und positiv semidefinit** ist.

Bausteine: Summe, positive Skalierung und Produkt gültiger Kernels sind wieder Kernels.

6 Representer-Theorem

Legende: L Verlustfunktion, λ Regularisierungsstärke, α_i Gewicht von Punkt i , L' Ableitung von L .

Für $J_\lambda(\theta) = \frac{1}{n} \sum_i L(\theta^\top x^{(i)}, y^{(i)}) + \frac{\lambda}{2} \|\theta\|^2$ mit konvexem L gibt es ein Optimum

$$\theta = \sum_{i=1}^n \alpha_i x^{(i)}$$

Beweisidee:

$$\nabla J = \frac{1}{n} \sum_i L'(\cdot) x^{(i)} + \lambda \theta = 0 \Rightarrow \theta = -\frac{1}{n\lambda} \sum_i L'(\cdot) x^{(i)}.$$

Rechtfertigt den Kernel-Trick.

7 Kernelisierte Regression

Kernel-LMS

```

init  $\beta \leftarrow 0$ 
repeat bis Konvergenz, for  $i = 1..n$ :
   $r \leftarrow y_i - \sum_j \beta_j K(x_j, x_i)$ 
   $\beta_i \leftarrow \beta_i + \alpha r$ 
predict( $x$ ):  $\sum_i \beta_i K(x_i, x)$ 
# Gram-Matrix  $K$  vorher berechnen:  $O(n^2)$ 
```

Beispiel: Polynom-Kernel $K(x, z) = (x^\top z)^2$

$x = (1, 2)$, $z = (3, 1)$. **Lösung**

Kernel: $x^\top z = 3 + 2 = 5 \Rightarrow K = 25$.

Explizit: $\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$: $\phi(x) = (1, 2.83, 4)$, $\phi(z) = (9, 4.24, 1)$

$\phi(x)^\top \phi(z) = 9 + 12 + 4 = 25$ **gleiches Ergebnis, viel billiger**

8 Vorteile

- ✓ nichtlinear ohne Feature-Design
- ✓ konvexe Optimierung bleibt
- ✓ Kernel für Text, Graphen, Strings

9 Grenzen

- × Gram-Matrix: $O(n^2)$ Speicher/Zeit
- × Wahl von Kernel & Parametern
- × schlecht skalierbar bei großem n

10 Key Takeaways

- ✓ $K(x, z) = \phi(x)^\top \phi(z)$
- ✓ $\theta = \sum \beta_i \phi(x_i)$ (Representer)
- ✓ Gültig $\Leftrightarrow$ Gram-Matrix PSD

11 Wo Kernels vorkommen

- SVM (nächstes Kapitel)
- Kernel-Ridge/Gauß-Prozesse
- Kernel-PCA, Kernel- k -Means

? Selbsttest: kannst du das erklären?

F1 Wann ist K ein gültiger Kernel?

A: Gram-Matrix symmetrisch und positiv semidefinit (Mercer).

F2 Was sagt das Representer-Theorem?

A: Es gibt ein Optimum $\theta = \sum \alpha_i x^{(i)} \Rightarrow$ Kernel-Trick anwendbar.

F3 Wirkung von σ beim RBF-Kernel?

A: Klein: sehr lokal (Overfitting); groß: fast linear.

„Nichtlinearität zum Preis eines Skalarprodukts.“

Support Vector Machines

Maximaler Margin, Dualität & Kernel

Finde die Trennlinie mit dem größten Abstand zu beiden Klassen. ♥

Merke:
Nur die Support Vectors ($\alpha_i > 0$) bestimmen die Grenze!

Quelle: main_notes.pdf (Kap. SVM), notes/cs229-notes3.pdf, materials/smo.pdf, section/cs229-ovxopt2.pdf

1 Idee: Margin

Labels $y \in \{-1, +1\}$, Klassifikator $\text{sign}(w^\top x + b)$.

Funktionaler Margin:

$$\hat{\gamma}^{(i)} = y^{(i)}(w^\top x^{(i)} + b)$$

Geometrischer Margin: $\gamma^{(i)} = \hat{\gamma}^{(i)} / \|w\|$ (echter Abstand).

Ziel: den **kleinsten** geometrischen Margin maximieren.

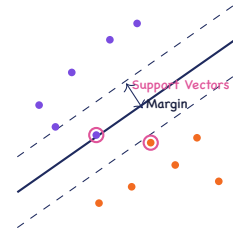
2 Primal (trennbar)

Normierung $\min_i \hat{\gamma}^{(i)} = 1$ liefert ein konvexes quadratisches Programm:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1$$

Margin-Breite = $2/\|w\|$.

3 Skizze



4 Dual-Form & Kernel

Lagrange: $w = \sum_i \alpha_i y^{(i)} x^{(i)}$. Dual-Problem:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle$$

s.t. $\alpha_i \geq 0$, $\sum_i \alpha_i y^{(i)} = 0$. **Nur Skalarprodukte** $\Rightarrow$ ersetze durch $K(x^{(i)}, x^{(j)})$:

$$f(x) = \sum_i \alpha_i y^{(i)} K(x^{(i)}, x) + b$$

5 Soft-Margin (nicht trennbar)

Schlupfvariablen $\xi_i \geq 0$ erlauben Verletzungen:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1 - \xi_i$$

Dual: nur $0 \leq \alpha_i \leq C$. Gleichbedeutend mit **Hinge-Verlust**

$$\xi_i = \max(0, 1 - y^{(i)} f(x^{(i)})).$$

C groß $\rightarrow$ wenige Fehler (Overfitting), C klein $\rightarrow$ großer Margin.

6 KKT-Bedingungen

Allgemein: Primal-/Dual-Zulässigkeit, Complementary Slackness $\alpha_i g_i = 0$, Stationarität. Für die SVM:

- $\alpha_i = 0 \Rightarrow y_i f_i \geq 1$
- $0 < \alpha_i < C \Rightarrow y_i f_i = 1$
- $\alpha_i = C \Rightarrow y_i f_i \leq 1$

7 SMO-Algorithmus (vereinfacht)

Aktualisiert immer zwei α gleichzeitig (wegen $\sum \alpha_i y_i = 0$). Grenzen: $y_i \neq y_j$:
 $L = \max(0, \alpha_j - \alpha_i)$, $H = \min(C, C + \alpha_j - \alpha_i)$; sonst $L = \max(0, \alpha_i + \alpha_j - C)$,
 $H = \min(C, \alpha_i + \alpha_j)$.

Simplified SMO

$\alpha \leftarrow 0$, $b \leftarrow 0$

repeat bis sich kein α mehr ändert:

for $i = 1..n$: **if** α_i verletzt KKT (Toleranz):

wähle $j \neq i$ zufällig; $E_k \leftarrow f(x_k) - y_k$

$\eta \leftarrow 2K_{ij} - K_{ii} - K_{jj}$; **if** $\eta = 0$: **continue**

$\alpha_j \leftarrow \text{clip}_{[L,H]}(\alpha_j - y_j(E_i - E_j)/\eta)$

$\alpha_i \leftarrow \alpha_i + y_i y_j (\alpha_j^{\text{alt}} - \alpha_j)$

$b \leftarrow$ so, dass KKT für i, j gilt (Mittel von b_1, b_2)

Beispiel: 1D-SVM von Hand

Aufgabe $x = -1, 0$ mit $y = -1$; $x = 2, 3$ mit $y = +1$. Finde w, b, α und klassifiziere $x = 1.4$.

Lösung Support Vectors: $x = 0$ ($y = -1$) und $x = 2$ ($y = +1$): $b = -1$, $2w + b = 1 \Rightarrow w = 1$.

Margin-Breite $2/\|w\| = 2$, Grenze bei $x = 1$. Kontrolle: $x = 3$: $f = 2 \geq 1 \checkmark$, $x = -1$: $yf = 2 \geq 1 \checkmark$.

Dual: $w = 2\alpha_{(2)} = 1 \Rightarrow \alpha_{(2)} = 0.5$, $\sum \alpha y = 0 \Rightarrow \alpha_{(0)} = 0.5$; alle anderen $\alpha = 0$.

$f(1.4) = 1.4 - 1 = 0.4 > 0 \Rightarrow$ Klasse **+1**.

11 Varianten, Kernels & Praxis (aus den ML Notes)

Linear, nichtlinear (Kernel), SVR (Regression, ϵ -Schlauch). Kernels: linear, Polynom $(\gamma x^\top z + r)^d$, RBF $e^{-\gamma \|x-z\|^2}$, Sigmoid. γ klein: glatt; groß: lokal, Overfitting. Praxis: C, γ per CV, Features skalieren, Multiklasse one-vs-rest, erst linear, dann RBF.

8 Vorteile

- ✓ konvex, globales Optimum
- ✓ Kernel $\Rightarrow$ nichtlinear
- ✓ sparse: nur Support Vectors zählen
- ✓ gut bei hoher Dimension

9 Grenzen

- ✗ Training teuer bei großem n
- ✗ liefert keine Wahrscheinlichkeiten
- ✗ Wahl von C und Kernel-Parametern

10 Key Takeaways

- ✓ Max. Margin: $\min \frac{1}{2} \|w\|^2$
- ✓ Dual + Kernel: $f = \sum \alpha_i y_i K + b$
- ✓ Soft-Margin: C steuert Trade-off
- ✓ SMO: 2 α pro Schritt

? Selbsttest: kannst du das erklären?

F1 Was sind Support Vectors?

A: Punkte mit $\alpha_i > 0$; nur sie bestimmen die Grenze.

F2 Rolle von C ?

A: Klein: großer Margin, mehr Fehler; groß: wenige Fehler, Overfitting-Risiko.

F3 Warum sind SVMs kernelfähig?

A: Das Dual enthält nur $\langle x_i, x_j \rangle \rightarrow K(x_i, x_j)$.

„Nicht die Masse der Daten entscheidet, sondern die wenigen Punkte am Rand.“

Online-Lernen: erst vorher-sagen, dann das wahre Label sehen. Gezählt werden die Fehler. ❤️

Perceptron & Online Learning

Lernen aus einem Strom von Beispielen

Merke:
Fehlerschranke $(D/\gamma)^2$ --
unabhängig von n und d !

Quelle: notes/cs229-notes6.pdf (Perceptron and large margin classifiers)

1 Online vs. Batch

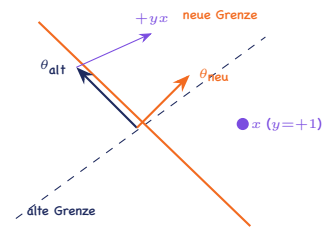
Batch: Trainingsdaten vorab, danach Test.
Online: Beispiele kommen nacheinander;
für jedes $x^{(i)}$ wird zuerst vorhergesagt,
dann $y^{(i)}$ enthüllt. Bewertung:

Gesamtzahl der Fehler .

2 Perceptron-Regel

$y \in \{-1, +1\}$, $h_\theta(x) = g(\theta^\top x)$ mit
 $g(z) = +1$ für $z \geq 0$, sonst -1 .
Update nur bei Fehler (θ Gewichte, x
Merkmale, y wahres Label):
 $\theta \leftarrow \theta + yx$
Lernrate egal (skaliert nur θ).

3 Skizze: ein Update



4 Algorithmus

```
</> Perceptron (online)
theta <- 0
for t = 1, 2, ... (Strom von Beispielen):
    y_hat <- sign(theta^T x_t) # Vorhersage
    if y_hat != y_t: # Label enthüllt
        theta <- theta + y_t x_t
    mistakes <- mistakes + 1
```

Nicht-probabilistisch; passt sich bei jedem Fehler an. Trennbare
Daten $\Rightarrow$ nach endlich vielen Fehlern fehlerfrei.

5 Satz von Block & Novikoff (1962)

Gilt $\|x^{(i)}\| \leq D$ und trennt ein Einheitsvektor u die Daten mit
Margin γ ($y^{(i)} u^\top x^{(i)} \geq \gamma$), macht der Perceptron höchstens
 $(D/\gamma)^2$ Fehler.

Beweisidee ($\theta^{(k)}$ = Parameter beim k -ten Fehler):

- $\theta^{(k+1)\top} u \geq k\gamma$ (wächst linear)
- $\|\theta^{(k+1)}\|^2 \leq kD^2$ (wächst höchstens linear)
- Cauchy-Schwarz: $k\gamma \leq \sqrt{k}D \Rightarrow k \leq (D/\gamma)^2$

Beispiel: 2 Fehler, dann fertig

Folge: $(x_1=(1, 2), y=-1)$, $(x_2=(2, 1), y=+1)$, wiederholt. Start
 $\theta = (0, 0)$.

Lösung (1) $\theta^\top x_1 = 0 \Rightarrow \hat{y} = +1 \neq -1$: Fehler, $\theta = (-1, -2)$.

(2) $\theta^\top x_2 = -4 \Rightarrow \hat{y} = -1 \neq +1$: Fehler,
 $\theta = (-1, -2) + (2, 1) = (1, -1)$.

(3) $x_1: -1 \Rightarrow -1$ ✓ (4) $x_2: +1 \Rightarrow +1$ ✓

Kontrolle: $D = \sqrt{5}$, $\gamma = 1/\sqrt{2} \Rightarrow (D/\gamma)^2 = 10 \geq 2$

Schranke erfüllt .

6 Verbindung zu anderen Kapiteln

- SVM:** gleiche Margin-Sprache, die Schranke wächst mit $1/\gamma^2$.
- Kernel-Perceptron:** $\theta = \sum \beta_i \phi(x_i)$, Vorhersage über K
(Aufgabenblatt 2).
- Logistische Regression:** ähnliche Regel, aber mit weichem h .

7 Vorteile

- ✓ extrem einfach & schnell
- ✓ speichert nur θ
- ✓ Fehlerschranke ohne n, d

8 Grenzen

- ✗ nur bei Trennbarkeit sinnvoll
- ✗ kein Wahrscheinlichkeitsmodell
- ✗ Ergebnis hängt von Reihenfolge ab

9 Key Takeaways

- ✓ Update nur bei Fehler: $\theta \leftarrow \theta + yx$
- ✓ Mistake Bound $(D/\gamma)^2$
- ✓ Bindeglied zu SVM & Kernels

? Selbsttest: kannst du das erklären?

F1 Wie lautet die Fehlerschranke?

A: $\leq (D/\gamma)^2$, unabhängig von n und d .

F2 Wann ändert sich θ ?

A: Nur bei Fehlklassifikation: $\theta \leftarrow \theta + yx$.

F3 Nicht trennbare Daten?

A: Keine Konvergenz: es treten immer
wieder Fehler auf.

„Lerne nur aus deinen Fehlern -- und beweise, dass es nur endlich viele sind.“

k-Nearest Neighbors

Lazy Learning: kein Training – vorhergesagt wird über die nächsten Nachbarn.



Merke:
Abstände zählen: Features vorher skalieren!

Nichtparametrisch, einfach, überraschend stark

Quelle: hand-notes/ML Notes.pdf (KNN); Ergänzungen (Fluch der Dimension, Bezug zu Bias/Varianz) aus dem Kursstoff

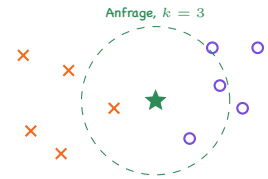
1 Idee

Für einen neuen Punkt x : suche die k ähnlichsten Trainingspunkte und übernahm ihre **Mehrheitsklasse** (Klassifikation) oder ihren **Mittelwert** (Regression). Nichtparametrisch: die Daten sind das Modell.

2 Algorithmus

```
</> k-NN-Vorhersage
for jedes  $x_i$ :  $d_i \leftarrow \text{dist}(x, x_i)$ 
 $\mathcal{N} \leftarrow$  Indizes der  $k$  kleinsten  $d_i$ 
classify:  $\arg \max_c \sum_{i \in \mathcal{N}} \mathbf{1}[y_i = c]$ 
regress:  $\frac{1}{k} \sum_{i \in \mathcal{N}} y_i$ 
```

3 Skizze: $k = 3$



4 Abstandsmaße

Euklid: $d(x, z) = \sqrt{\sum_j (x_j - z_j)^2}$
 Manhattan: $d(x, z) = \sum_j |x_j - z_j|$
 Minkowski: $d(x, z) = (\sum_j |x_j - z_j|^p)^{1/p}$
 Kosinus: $d(x, z) = 1 - \frac{x^\top z}{\|x\| \|z\|}$

Legende: x, z zwei Punkte, j Merkmalsindex, p Exponent (Minkowski).

$p = 2$ Euklid, $p = 1$ Manhattan. Kosinus für Text (Richtung statt Länge).

5 Wahl von k (Bias-Varianz)

k klein $\rightarrow$ flexible Grenze, hohe Varianz (Overfitting, $k=1$: Trainingsfehler 0). k groß $\rightarrow$ glatt, hoher Bias (Underfitting).
 Faustregel: ungerades k bei 2 Klassen; k per **Kreuzvalidierung** wählen.
sklearn: `n_neighbors`, `weights='uniform'/'distance'`, `metric`, `p`.

Beispiel: die Vorhersage hängt von k ab

Anfrage (0, 0); Trainingspunkte:

Punkt	(1, 0)	(0, 2)	(1, 2)	(3, 0)	(0, 4)
Abstand	1	2	2.24	3	4
Klasse	A	B	B	A	A

Lösung $k = 1$: A. $k = 3$: A, B, B $\Rightarrow$ B. $k = 5$: A, B, B, A, A $\Rightarrow$ A (3:2).

Regression mit $y = (3, 5, 4, 8, 9)$, $k = 3$: $\frac{3+5+4}{3} = 4$.

6 Fluch der Dimension

In hohen Dimensionen sind alle Punkte etwa gleich weit entfernt: "nächste" Nachbarn sind nicht mehr nah. Abhilfe: Features auswählen, PCA, mehr Daten. Aufwand naiv $O(nd)$ pro Anfrage; Beschleunigung mit k-d-Baum/Ball-Tree (nur niedrige d).

7 Vorteile

- ✓ kein Training, sehr einfach
- ✓ nichtlineare Grenzen
- ✓ Klassifikation und Regression

8 Grenzen

- ✗ langsame Vorhersage bei großem n
- ✗ empfindlich bei Skalierung/Ausreißern
- ✗ leidet unter hoher Dimension

9 Key Takeaways

- ✓ Mehrheit/Mittel der k Nachbarn
- ✓ k steuert Bias-Varianz
- ✓ skalieren + Abstand wählen

? Selbsttest: kannst du das erklären?

F1 Warum ist $k=1$ overfittet?

A: Jeder Trainingspunkt ist sein eigener Nachbar $\Rightarrow$ Trainingsfehler 0, sehr wackelige Grenze (hohe Varianz).

F2 Warum skalieren?

A: Ein Feature mit großer Skala dominiert den Abstand.

F3 Warum leidet k-NN in hohen Dimensionen?

A: Abstände konzentrieren sich; "nächste" Nachbarn sind kaum näher als andere Punkte.

„Sage mir, wer deine Nachbarn sind, und ich sage dir, wer du bist.“

Decision Trees

Nichtlinear, interpretierbar, aber instabil

Ja/Nein-Fragen zerlegen den Raum in Rechtecke, jedes mit eigener Vorhersage. ♥

Merke:
Zum Splitten Cross-Entropy nehmen -- nicht die Fehlerrate!

Quelle: notes/cs229-notes-dt.pdf (Decision Trees, R. Townshend)

1 Idee

Zerlege $\mathcal{X}$ in disjunkte Regionen $R_1, \dots, R_m$ und sage pro Region konstant voraus: Mehrheitsklasse (Klassifikation) bzw. Mittelwert (Regression). Erstes

genuin nichtlineares Verfahren im Kurs.

2 Greedy-Split

Optimale Regionen zu finden ist inaktuell $\Rightarrow$ gierig, top-down:

$$R_1 = \{x \in R_p : x_j < t\},$$

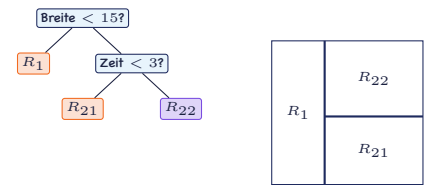
$$R_2 = \{x \in R_p : x_j \geq t\}.$$

Legende: R_p Eltern-Region, R_1, R_2 Kinder, j Feature, t Schwelle, L Unreinheit einer Region, $|R|$ Anzahl Beispiele.

Wähle (R_p, j, t) mit maximalem Rückgang:

$$L(R_p) - \frac{|R_1|L(R_1) + |R_2|L(R_2)}{|R_1| + |R_2|}$$

3 Skizze: Baum & Regionen



4 Split-Verluste

$$L_{\text{misclass}} = 1 - \max_c \hat{p}_c$$

$$L_{\text{cross}} = - \sum_c \hat{p}_c \log_2 \hat{p}_c \quad (\text{Information Gain})$$

$$L_{\text{Gini}} = 1 - \sum_c \hat{p}_c^2$$

$$L_{\text{squared}} = \frac{1}{|R|} \sum_{i \in R} (y_i - \hat{y})^2 \quad (\text{Regression})$$

Legende: $\hat{p}_c$ Anteil der Klasse c in der Region, $\hat{y}$ Mittelwert der Region, R Region.

Cross-Entropy/Gini sind **strikt konkav**: jeder informative Split senkt den Verlust; die Fehlerrate nicht.

5 Algorithmus

</> Baum wachsen lassen

grow(R):

if Stopp(R): **return** Blatt(Mehrheit/Mittel)
 $(j, t) \leftarrow \arg \max$ Gain über alle Features/Schwellen
 $R_1 \leftarrow \{x \in R : x_j < t\}, R_2 \leftarrow \{x \in R : x_j \geq t\}$
return Knoten($j, t, \text{grow}(R_1), \text{grow}(R_2)$)

prune: erst voll wachsen, dann Knoten entfernen, die den Validierungsfehler am wenigsten erhöhen

Beispiel: warum Cross-Entropy?

Eltern: 400+ / 100- ($p = 0.8$). Split A: 150+ / 100- & 250+ / 0-. Split B: 300+ / 100- & 100+ / 0-.

Fehlerrate: Eltern 0.2; A: $\frac{250 \cdot 0.4 + 0}{500} = 0.2$; B: $\frac{400 \cdot 0.25 + 0}{500} = 0.2 \Rightarrow$

kein Unterschied!

Cross-Entropy: Eltern 0.722; A: $0.5 \cdot 0.971 = 0.485$ (Gain 0.237); B: $0.8 \cdot 0.811 = 0.649$ (Gain 0.073) $\Rightarrow$ A ist besser.

6 Regularisierung & Praxis

Voll ausgewachsen (1 Punkt/Blatt) $\Rightarrow$ niedriger Bias, hohe Varianz. Stopp-Regeln: Mindest-Blattgröße, max. Tiefe, max. Blattzahl - oder Pruning.

Kategoriale Features: direkt per Teilmengen-Frage ($x_j \in S$); Vorsicht bei vielen Kategorien ($2^{|S|}$ Fragen).

Laufzeit: Vorhersage $O(\text{Tiefe})$, Training $O(nf \cdot \text{Tiefe})$.

7 Vorteile

- ✓ leicht erklärbar
- ✓ kategoriale Daten nativ
- ✓ schnell, keine Skalierung nötig

8 Grenzen

- ✗ hohe Varianz (instabil)
- ✗ additive Struktur ($x_1 + x_2$) nur mit vielen Splits
- ✗ einzeln oft geringe Genauigkeit

9 Key Takeaways

- ✓ Greedy, rekursive Splits
- ✓ Cross-Entropy/Gini statt Fehlerrate
- ✓ Pruning gegen Overfitting
- ✓ Lösung der Instabilität: Ensembles

* Information Gain & Hyperparameter (aus den ML Notes)

Entropy(S) = $-\sum_i p_i \log_2 p_i$, IG(S, A) = Entropy(S) - $\sum_v \frac{|S_v|}{|S|} \text{Entropy}(S_v)$ (Teilmengen S_v je Wert v von Feature A). Algorithmen:

ID3 (IG), **C4.5** (Gain Ratio, korrigiert Bias zu vielwertigen Features), **CART** (Gini, auch Regression via Varianzreduktion).

Stoppkriterien: max. Tiefe, Mindestzahl Beispiele pro Knoten/Blatt, kein Gain mehr. sklearn: criterion, max_depth, min_samples_split, min_samples_leaf, max_features. Pruning: Pre (früh stoppen) oder Post (Cost-Complexity, danach zurückschneiden).

? Selbsttest: kannst du das erklären?

F1 Warum Cross-Entropy statt Fehlerrate?

A: Strikt konkav: informative Splits senken den Verlust immer; die Fehlerrate kann gleich bleiben.

F2 Wie regularisiert man Bäume?

A: Max. Tiefe, Mindest-Blattgröße, Pruning per Validierungsfehler.

F3 Schwäche bei $x_1 + x_2$?

A: Additive Struktur braucht viele achsenparallele Splits.

„Ein Baum erklärt sich selbst -- ein Wald wird genauer.“

Ensemble-Methoden

Viele Modelle schlagen eins:
Bagging senkt die Varianz,
Boosting den Bias. ❤️

Bagging, Random Forests & Boosting

Merke:
 $\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$

Quelle: notes/cs229-notes-ensemble.pdf (R. Townshend), extra-notes/boosting.pdf (J. Duchi); Web-Zusatz zu XGBoost markiert

1 Warum Ensembles?

Mittel von n Modellfehlern (Varianz σ^2 , Korrelation ρ):

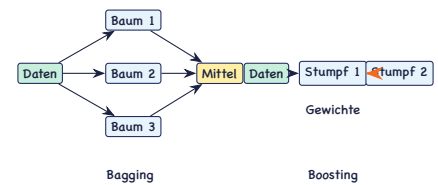
$$\text{Var} = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$$

Mehr Modelle ($n \uparrow$) und
weniger Korrelation ($\rho \downarrow$) senken die Varianz.

2 Bagging

Bootstrap: (M Modelle, n Daten) M Stichproben mit Zurücklegen (Größe n), je ein Modell G_m , dann $G(x) = \frac{1}{M} \sum_m G_m(x)$. Senkt Varianz, Bias steigt leicht.
Out-of-Bag: je $\approx \frac{1}{3}$ der Daten ungenutzt $\Rightarrow$ gratis Validierung.

3 Skizze: parallel vs. sequentiell



4 Random Forest

Bagging von Decision Trees plus zufällige Feature-Teilmenge pro Split $\Rightarrow$ Bäume dekorrelieren ($\rho \downarrow$).

```
</> Random Forest
for m = 1..M:
    Z_m ← Bootstrap-Stichprobe von S
    Baum G_m auf Z_m; bei jedem Split nur
    zufällige Teilmenge der Features testen
predict: Mehrheit (bzw. Mittel) über alle G_m
```

5 AdaBoost

Schwache Lerner (z. B. Stümpfe) nacheinander, falsch klassifizierte Punkte bekommen mehr Gewicht.

Legende: w_i Gewicht von Beispiel i , G_m Lerner m , err_m gewichtete Fehlerrate, α_m Stimmgewicht, N Beispiele.

```
</> AdaBoost
w_i ← 1/N
for m = 1..M:
    G_m ← Lerner auf gewichteten Daten
    err_m ← ∑ w_i 1[y_i ≠ G_m(x_i)] / ∑ w_i
    α_m ← log(1/err_m)
    w_i ← w_i e^{α_m 1[y_i ≠ G_m(x_i)]}
return sign ∑ α_m G_m(x)
```

6 Forward Stagewise & Gradient Boosting

$f_m = f_{m-1} + \beta_m G(x; \gamma_m)$ mit
 $(\beta_m, \gamma_m) = \arg \min \sum_i L(y_i, f_{m-1}(x_i) + \beta G(x_i; \gamma))$.

Legende: f_m Ensemble nach m Schritten, G schwacher Lerner mit Parametern γ_m , β_m Gewicht, L Verlust, g_i negativer Gradient (Residuum).

Quadr. Verlust $\Rightarrow$ Residuen fiten; $e^{-y\hat{y}} \Rightarrow$ AdaBoost.

```
</> Gradient Boosting
f ← 0
for m = 1..M:
    g_i ← -∂L(y_i, f(x_i))/∂f(x_i)
    G ← Regressor auf (x_i, g_i) (Least Squares)
    f ← f + β G
```

Beispiel: eine AdaBoost-Runde

5 Punkte, Gewichte je 0.2. Stumpf 1 klassifiziert einen falsch.

Lösung $\text{err} = 0.2$, $\alpha = \log \frac{0.8}{0.2} = \log 4 \approx 1.39$.

Update: falscher Punkt $0.2 \cdot e^{1.39} = 0.8$, die anderen bleiben 0.2. Summe = 1.6.

Normiert: falscher Punkt 0.5, die vier richtigen je 0.125 $\Rightarrow$

Runde 2 muss den schwierigen Punkt lösen.

7 Bagging vs. Boosting

	Bagging	Boosting
Ziel	Varianz senken	Bias senken
Lerner	stark, tief (hohe Var.)	schwach (hoher Bias)
Aufbau	parallel, unabhängig	sequentiell, abhängig
Mehr Modelle	kein Overfitting	Overfitting möglich
Extra	Out-of-Bag, RF	additive Modelle

8 Key Takeaways

- ✓ $\text{Var} = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$
- ✓ Bagging: Bootstrap + Mittel
- ✓ RF: + Feature-Subsampling
- ✓ Boosting: sequentiell, gewichtet

9 Voting, Stacking & Praxis (ML Notes; Internet: XGBoost)

Voting: Mehrheit (hart) bzw. Mittel der Wahrscheinlichkeiten (weich). **Stacking:** ein Meta-Modell kombiniert die Vorhersagen der Basis-Modelle; diese müssen out-of-fold sein (sonst Leakage). Wahl: hohe Varianz $\rightarrow$ Bagging, hoher Bias $\rightarrow$ Boosting, beste Leistung $\rightarrow$ Stacking. **Internet-Zusatz** (nicht aus den Notes): XGBoost, LightGBM und CatBoost sind moderne Gradient-Boosting-Bibliotheken und dominieren tabellarische Daten; Random Forests: Breiman (2001). Feature Importance und Code: Vertiefungs- und SHAP-Seiten.

? Selbsttest: kannst du das erklären?

F1 Wie senkt Bagging die Varianz?

A: Bootstrap dekorreliert die Modelle:
 $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$.

F2 Ziel von Bagging vs. Boosting?

A: Bagging: Varianz; Boosting: Bias (schwache Lerner, sequentiell).

F3 Was ist der Out-of-Bag-Fehler?

A: Fehler auf den $\approx \frac{1}{3}$ nicht gezogenen Punkten: gratis Validierung.

„Viele schwache Meinungen, klug kombiniert, ergeben ein starkes Urteil.“

Verlustfunktionen

Konvexe Ersatzverluste (Surrogate Losses)

Der 0/1-Verlust ist nicht optimierbar -- also nutzen wir konvexe Ersatzverluste. ♥

Merke:
Margin $z = y\theta^\top x$: $z > 0$ heißt richtig klassifiziert!

Quelle: extra-notes/loss-functions.pdf und extra-notes/representer-function.pdf (J. Duchi)

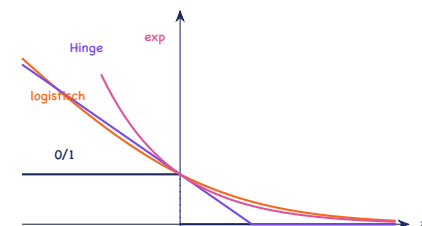
1 Der Margin

Für $y \in \{-1, +1\}$ und $\hat{y} = \theta^\top x$ ist $z = y\theta^\top x$ der **Margin**: $z > 0$ richtig, $z < 0$ falsch; $|z|$ misst die Sicherheit.
Empirisches Risiko:
 $J(\theta) = \frac{1}{n} \sum_i \phi(y^{(i)}\theta^\top x^{(i)})$.

2 Problem: 0/1-Verlust

$\phi_{01}(z) = \mathbf{1}\{z \leq 0\}$ zählt Fehler, ist aber unstetig, nicht konvex und NP-hart zu minimieren.
Wunsch: ϕ konvex, stetig, $\phi(z) \rightarrow 0$ für $z \rightarrow \infty$, $\phi(z) \rightarrow \infty$ für $z \rightarrow -\infty$.

3 Skizze: Verluste über z



4 Drei Verluste, drei Verfahren

Verlust	$\phi(z)$	Verfahren
Logistisch	$\log(1 + e^{-z})$	Logistische Regression
Hinge	$\max(1 - z, 0)$	Support Vector Machine
Exponentiell	e^{-z}	AdaBoost

Verbindendes Prinzip: alle minimieren $\frac{1}{n} \sum \phi(y\theta^\top x)$ – nur ϕ unterscheidet sich; alle sind konvexe obere Schranken des 0/1-Verlusts.

5 Generisches Training

(/) ERM mit Ersatzverlust

```
init  $\theta \leftarrow 0$ 
repeat: wähle Beispiel  $(x_i, y_i)$ 
   $z \leftarrow y_i \theta^\top x_i$ 
   $g \leftarrow \phi'(z) \cdot y_i x_i$ 
   $\theta \leftarrow \theta - \alpha g$ 
#  $\alpha$ : Lernrate,  $\phi'$ : Ableitung; logistisch  $-1/(1 + e^z)$ 
# Hinge:  $-1$  falls  $z < 1$ , sonst 0 (Subgradient)
# exp:  $-e^{-z}$ 
```

Beispiel: Verluste bei drei Margins

z	0/1	logist.	Hinge	exp
3	0	0.05	0	0.05
0.5	0	0.47	0.5	0.61
-1	1	1.31	2	2.72

Selbst richtig klassifizierte Punkte mit kleinem Margin ($z = 0.5$) werden noch bestraft $\Rightarrow$ Ansporn, den Margin zu vergrößern.

6 Verallgemeinerte Verluste

Allgemein: $L(\theta^\top x, y)$ (z : Score, k : Anzahl Klassen). Regression: $\frac{1}{2}(z - y)^2$. Binär: $\log(1 + e^{-yz})$. Multiklasse mit $\Theta = [\theta_1 \dots \theta_k]$:

$$L(\Theta^\top x, y) = \log \sum_{i=1}^k e^{x^\top (\theta_i - \theta_y)}$$

(Cross-Entropy/Softmax; θ_y : Parameter der wahren Klasse, Θ : alle Klassenparameter als Spalten). Mit konvexem L und ℓ_2 -Regularisierung gilt das Representer-Theorem.

7 Wann welcher Verlust?

- Wahrscheinlichkeiten nötig $\rightarrow$ logistisch
- Margin/Sparsität $\rightarrow$ Hinge
- Boosting $\rightarrow$ exponentiell (ausreißeranfällig)

8 Key Takeaways

- ✓ Margin $z = y\theta^\top x$
- ✓ 0/1 nicht optimierbar $\Rightarrow$ Surrogat
- ✓ logistisch = LogReg, Hinge = SVM, exp = AdaBoost

9 Querverweise

- Logistische Regression, SVM, Ensembles
- Praktische Aufgabe: p01_lr.py (Aufgabenblatt 2) nutzt genau diese ± 1 -Notation

? Selbsttest: kannst du das erklären?

F1 Warum nicht den 0/1-Verlust minimieren?

A: Unstetig, nichtkonvex, NP-hart.

F2 Welcher Verlust gehört zu SVM, AdaBoost, LogReg?

A: Hinge, exponentiell, logistisch.

F3 Was ist der Margin?

A: $z = y\theta^\top x$: $z > 0$ richtig, $|z|$ Sicherheit.

„Optimiere, was du optimieren kannst -- und hoffe, dass es mit dem zusammenfällt, was du willst.“

Neuronale Netze

Deep Learning: Architektur & Bausteine

Schicht für Schicht: lineare Abbildung plus Nichtlinearität -- beliebig tief. ❤️

Merke:
Ohne Nichtlinearität kollabiert jedes Netz zu einem linearen Modell!

Quelle: main_notes.pdf (Deep Learning), notes/cs229-notes-deep_learning.pdf

1 Idee

Lineare Modelle sind beschränkt. Durch **Komposition nichtlinearer Schichten** approximiert ein Netz beliebig komplexe Funktionen. Ein Netz lernt seine Merkmale selbst, statt sie von Hand zu entwerfen.

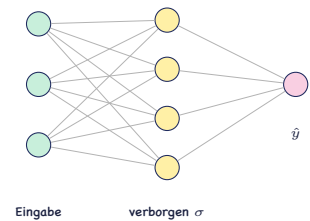
2 Forward-Pass

$$\begin{aligned} a^{[0]} &= x \\ z^{[\ell]} &= W^{[\ell]} a^{[\ell-1]} + b^{[\ell]} \\ a^{[\ell]} &= \sigma(z^{[\ell]}), \quad \ell < r \\ \hat{y} &= W^{[r]} a^{[r-1]} + b^{[r]} \end{aligned}$$

Legende: r Schichten, ℓ Schichtindex, $W^{[\ell]}, b^{[\ell]}$ Gewichte und Bias, z Vorstufe, a Aktivierung, σ nichtlineare Aktivierung, $\hat{y}$ Ausgabe.

Ohne σ : $W^{[r]} \dots W^{[1]} x = \tilde{W} x$ (linear!).

3 Skizze: 3-4-1-Netz



4 Aktivierungsfunktionen

Name	Formel	Einsatz
Sigmoid	$1/(1 + e^{-z})$	Ausgabe binär
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	veraltet (verborgen)
ReLU	$\max(z, 0)$	Standard (verborgen)
Leaky ReLU	$\max(z, \gamma z)$, $\gamma \approx 0.01$	gegen „dying ReLU“
GELU	$z \Phi(z)$, Φ : Gauß-CDF	Transformer
Softmax	$e^{z_k} / \sum_j e^{z_j}$	Ausgabe Mehrklassen

5 Moderne Bausteine

Legende: W_1, W_2, b_1, b_2 Gewichte/Bias, $\hat{\mu}, \hat{\sigma}$ Mittel und Streuung über die Merkmale, γ, β lernbare Skala/Verschiebung, k Filtergröße, s Schrittweite, w_j Filtergewichte, m Breite.

ResNet: $\text{Res}(z) = z + \sigma(W_2 \sigma(W_1 z + b_1) + b_2)$ – Gradient fließt durch die Abkürzung (gegen Vanishing Gradients).

LayerNorm: $\text{LN}(z) = \gamma \odot \frac{z - \hat{\mu}}{\hat{\sigma}} + \beta$ (skaleninvariant in W).

Faltung (CNN): $\text{Conv}(z)_i = \sum_{j=1}^k w_j z_{i-s-j+1}$.

Parameter-Sharing: $O(m^2) \rightarrow O(km)$.

Beispiel: Forward-Pass 2-2-1 mit ReLU

$x = (1, 2)$, $W^{[1]} = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}$, $b^{[1]} = 0$, $W^{[2]} = (1, 2)$, $b^{[2]} = -1$.

Lösung $z^{[1]} = (1 - 2, 0.5 + 2) = (-1, 2.5)$

$a^{[1]} = \text{ReLU}(z^{[1]}) = (0, 2.5)$

$\hat{y} = 1 \cdot 0 + 2 \cdot 2.5 - 1 = 4$; Verlust ($y=3$): $\frac{1}{2}(4 - 3)^2 = 0.5$.

6 Beispiel: Parameter zählen

Netz $100 \rightarrow 50 \rightarrow 10$ (voll verbunden):

Schicht 1: $100 \cdot 50 + 50 = 5050$

Schicht 2: $50 \cdot 10 + 10 = 510$

Summe: **5560** Parameter.

Allgemein: Schicht mit $m_{\text{in}} \rightarrow m_{\text{out}}$ hat $m_{\text{in}} m_{\text{out}} + m_{\text{out}}$ Parameter.

7 Vorteile

- ✓ universelle Approximation
- ✓ lernt Merkmale selbst
- ✓ skaliert mit Daten & Rechenleistung

8 Grenzen

- ✗ nichtkonvex, viele Hyperparameter
- ✗ datenhungrig, schwer interpretierbar
- ✗ anfällig für Adversarial Examples

9 Key Takeaways

- ✓ $z = W a + b$, $a = \sigma(z)$
- ✓ ReLU = Standard, Softmax am Ausgang
- ✓ ResNet/LayerNorm/CNN als Bausteine

10 Ausgabe & Verlust

- binär: Sigmoid + Cross-Entropy
- Mehrklassen: Softmax + Cross-Entropy
- Regression: linear + MSE

11 Start des Trainings

- Gewichte klein zufällig initialisieren (nicht 0: Symmetriebroch!)
 - Eingaben normalisieren
 - Mini-Batch-SGD (nächste Seite)

12 Querverweise

- Backpropagation: wie der Gradient entsteht
- Regularisierung: Weight Decay = L2
- Foundation Models: Transformer = MLP + Attention

? Selbsttest: kannst du das erklären?

F1 Wozu Nichtlinearität?

A: Ohne σ ist ein tiefes Netz nur eine lineare Abbildung $\tilde{W}x$.

F2 Wozu ResNets?

A: Skip-Connections lassen den Gradient durch viele Schichten fließen.

F3 Parameter einer Schicht $m_{\text{in}} \rightarrow m_{\text{out}}$?

A: $m_{\text{in}} m_{\text{out}} + m_{\text{out}}$.

„Tiefe Netze sind einfache Bausteine -- nur sehr oft hintereinander.“

Rückwärts durch das Netz:
die Kettenregel, systematisch
angewandt. ♡

Backpropagation & Optimierung

Gradienten effizient berechnen und nutzen

Merke:
Der Gradient kostet nur so viel
wie der Forward-Pass: $O(N)$!

Quelle: main_notes.pdf (Backpropagation), notes/cs229-notes-backprop.pdf

1 Theorem

Lässt sich der Verlust in einem Forward-Pass mit Aufwand N berechnen, so lässt sich der vollständige Gradient $\nabla_{\theta} J$ ebenfalls in $O(N)$ berechnen. Backprop ist "nur" die Kettenregel mit Zwischenspeicherung.

2 Formeln

Legende: $u^{[\ell]}$ Zwischenwert nach Modul ℓ , M_{ℓ} Modul (Schicht), $\theta^{[\ell]}$ dessen Parameter, J Verlust, B Rückwärtsfunktion (transponierte Ableitung des Moduls), L Anzahl Module.

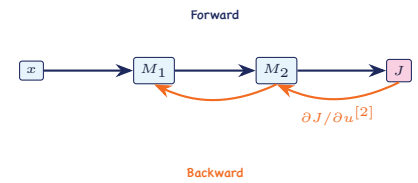
Forward: $u^{[\ell]} = M_{\ell}(u^{[\ell-1]}; \theta^{[\ell]})$, $J = u^{[L]}$.
Backward:

$$\frac{\partial J}{\partial u^{[L]}} = 1$$

$$\frac{\partial J}{\partial u^{[\ell-1]}} = B[M_{\ell}, u^{[\ell-1]}] \left(\frac{\partial J}{\partial u^{[\ell]}} \right)$$

$$\frac{\partial J}{\partial \theta^{[\ell]}} = B_{\theta}[M_{\ell}, \theta^{[\ell]}] \left(\frac{\partial J}{\partial u^{[\ell]}} \right)$$

3 Skizze: Vor- & Rückwärts



4 Backprop im MLP

```
</> Backpropagation
# Forward (Zwischenwerte speichern)
a0 ← x; for ℓ = 1..L:
    zℓ ← Wℓ aℓ-1 + bℓ; aℓ ← σ(zℓ)
# Backward
δL ← ∂J/∂zL
for ℓ = L..1:
    ∂J/∂Wℓ ← δℓ aℓ-1T; ∂J/∂bℓ ← δℓ
    δℓ-1 ← (WℓT δℓ) ⊙ σ'(zℓ-1)
```

5 Mini-Batch-SGD

```
</> Mini-Batch SGD
init θ (zufällig)
repeat bis Konvergenz:
    Batch B ⊂ {1..n}, |B| = B
    g ← 1/B ∑i∈B ∇θ J(i)(θ)
    θ ← θ - α g
```

(B Batch, B Batchgröße, $J^{(i)}$ Verlust von Beispiel i , g Gradientenschätzung, α Lernrate.) $B=1$: reines SGD (verrauscht); $B=n$: Batch-GD (teuer). Kleine Batches sind billig und wirken leicht regularisierend.

Web-Zusatz: in der Praxis oft Momentum bzw. Adam statt reinem SGD.

Beispiel: Backprop von Hand

$\hat{y} = w_2 \text{ReLU}(w_1 x)$, $x = 2$, $y = 0$, $w_1 = 1$, $w_2 = 0.5$,
 $J = \frac{1}{2}(\hat{y} - y)^2$.

Lösung Forward: $z = 2$, $a = 2$, $\hat{y} = 1$, $J = 0.5$.

Backward: $\partial J / \partial \hat{y} = 1$; $\partial J / \partial w_2 = 1 \cdot a = 2$;
 $\partial J / \partial a = 1 \cdot w_2 = 0.5$; $\partial J / \partial z = 0.5$ ($\text{ReLU}' = 1$);
 $\partial J / \partial w_1 = 0.5 \cdot x = 1$.

SGD ($\alpha=0.1$): $w_2 = 0.3$, $w_1 = 0.9 \Rightarrow \hat{y} = 0.54$, $J = 0.146 < 0.5$

Verlust sinkt.

6 Praxis & Fallstricke

- **Gradient-Check:** $\frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$ mit Backprop vergleichen.
- **Vanishing/Exploding Gradients:** Sigmoid/Tanh sättigen; Abhilfe: ReLU, ResNet, LayerNorm, gute Initialisierung.
- Lernrate zu groß $\rightarrow$ Divergenz, zu klein $\rightarrow$ Stillstand.

7 Vorteile

- ✓ exakter Gradient
- ✓ Aufwand $\approx$ ein Forward-Pass
- ✓ Autodiff-Frameworks übernehmen es

8 Grenzen

- ✗ Zwischenwerte brauchen Speicher
- ✗ nichtkonvex: nur lokale Minima
- ✗ Hyperparameter-Tuning nötig

9 Key Takeaways

- ✓ Backprop = Kettenregel rückwärts
- ✓ Kosten $O(N)$
- ✓ SGD in Mini-Batches

? Selbsttest: kannst du das erklären?

F1 Aufwand von Backprop?

A: $O(N)$, wie ein Forward-Pass.

F2 Was ist ein Gradient-Check?

A: $\frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$ mit Backprop vergleichen.

F3 Wirkung der Batchgröße?

A: $B=1$ verrauscht, $B=n$ teuer; Mini-Batch als Kompromiss.

„Rückwärts rechnen, vorwärts lernen.“

Neuronale Netze trainieren:
Optimierer, Initialisierung,
Normalisierung und Regularisierung. ❤️

Deep Learning: Training & Praxis

Optimierer, Init, BatchNorm, Dropout, Early Stopping

Merke:
Adam $\approx$ Momentum +
adaptive Lernraten

Quelle: main_notes.pdf (Deep Learning: LayerNorm, SGD, Regularisierung); Adam, Dropout, BatchNorm, Initialisierung als Web-Ergänzung (markiert),
Code: vertiefungsseite

1 Momentum & Adam (Web)

Legende: g Gradient, θ Parameter, α Lernrate, v Geschwindigkeit, m gleitender Mittelwert der Gradienten, $\hat{v}$ (bei Adam) der Quadrate, β, β_1, β_2 Abklingraten, t Schritt, ϵ kleine Konstante.

Momentum: $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \alpha v$ (glättet Zickzack).

Adam: $m \leftarrow \beta_1 m + (1 - \beta_1) g$,
 $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$, $\hat{m} = \frac{m}{1 - \beta_1^t}$,

$\hat{v} = \frac{v}{1 - \beta_2^t}$:

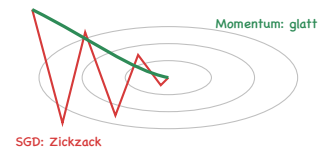
$$\theta \leftarrow \theta - \alpha \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}}$$

2 Lernraten-Schedules

Konstante Lernrate ist selten optimal: Step decay, Cosine-Abklingen, Warmup (klein starten, dann hoch). Zu große Rate $\rightarrow$ Verlust springt/divergiert; zu kleine $\rightarrow$ Stillstand. Vor dem Tuning:

Verlustkurve plotten.

3 Skizze: SGD vs. Momentum



4 Initialisierung (Web)

Nicht alle Gewichte = 0: dann sind alle Neuronen einer Schicht identisch (Symmetrie!). Zufällig mit passender Varianz (n_{in} Anzahl Eingänge): Xavier $\text{Var}(W) = \frac{1}{n_{in}}$ (Tanh/Sigmoid), He

$\text{Var}(W) = \frac{2}{n_{in}}$ (ReLU).

Ziel: Aktivierungen und Gradienten weder verschwinden noch explodieren lassen.

5 Normalisierung

BatchNorm (Web): $\text{BN}(z) = \gamma \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$ mit Batch-Statistiken

μ_B, σ_B^2 (γ, β lernbare Skala und Verschiebung) (im Test: gleitende Mittelwerte).

LayerNorm (Notes): Statistik pro Beispiel über die Features, unabhängig von der Batchgröße $\Rightarrow$ Standard in Transformern (Kapitel Neuronale Netze).

Wirkung: stabileres, schnelleres Training, größere Lernraten.

Beispiel: erster Adam-Schritt

$g = 2$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\alpha = 0.001$, $t = 1$.

Lösung $m = 0.1 \cdot 2 = 0.2$, $v = 0.001 \cdot 4 = 0.004$.

$\hat{m} = \frac{0.2}{0.1} = 2$, $\hat{v} = \frac{0.004}{0.001} = 4$.

Schritt $= \alpha \frac{\hat{m}}{\sqrt{\hat{v}}} = 0.001 = \alpha$: Adam skaliert so, dass der erste Schritt etwa die Größe α hat, unabhängig von der Gradientengröße.

Dropout ($p_{\text{keep}} = 0.5$): aus $(1, 2, 3, 4)$ wird z. B. $(2, 0, 6, 0)$ (verbleibende $\times 2$, inverted dropout).

6 Regularisierung im Deep Learning

- **Weight Decay** = L2 (Notes: $\frac{\lambda}{2} \|\theta\|^2$).
- **Dropout (Web):** im Training zufällig Neuronen ausschalten $\rightarrow$ implizites Ensemble.
- **Early Stopping:** Training beenden, wenn der Validierungsfehler steigt.
- **Datenaugmentierung:** Bilder drehen/spiegeln/zuschneiden.

7 Training debuggen

1. Erst auf kleinem Batch überanpassen (Verlust $\rightarrow 0$?) – sonst Bug.
2. Anfangsverlust prüfen ($\approx \log K$ bei K Klassen).
3. Lernrate über Größenordnungen probieren.
4. Train- vs. Validierungskurve: Gap $\Rightarrow$ Overfitting.
5. Gradient Clipping bei explodierenden Gradienten.

8 PyTorch-Trainingschleife (Skizze)

```
</> PyTorch
for Epoche in Epochen:
    model.train()
    for (x, y) in Mini-Batches:
        opt.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward() # Backprop
        opt.step()      # Update
```

Vollständiger, getesteter Code: Vertiefungsseite.

9 Key Takeaways

- ✓ Adam als robuster Standard, SGD+Momentum oft bessere Endleistung; Schedules und Warmup
- ✓ He-Initialisierung für ReLU; BatchNorm/LayerNorm stabilisieren; Dropout, Weight Decay, Early Stopping gegen Overfitting

? Selbsttest: kannst du das erklären?

F1 Warum nicht alle Gewichte mit 0 initialisieren?

A: Alle Neuronen bleiben identisch (gleiche Gradienten): Symmetrie wird nie gebrochen.

F2 Was bewirkt Dropout?

A: Zufälliges Ausschalten erzwingt redundante Merkmale, wirkt wie ein Ensemble.

F3 Was unterscheidet BatchNorm und LayerNorm?

A: BatchNorm normiert über den Batch, LayerNorm über die Features eines Beispiels.

„Training ist Handwerk: beobachte die Kurven, verändere eine Sache nach der anderen.“

Struktur nutzen: Faltungen für Bilder, Rekurrenz und Gates für Sequenzen. ♥

CNN & RNN / LSTM

Faltungsnetze und rekurrente Netze

Merke:
Parameter-Sharing macht CNNs klein, LSTM-Gates machen RNNs lernfähig

Quelle: main_notes.pdf (Modules in modern neural networks: Faltungsschichten); RNN/LSTM, Größenformeln und Pooling als Web-Ergänzung (markiert), Code: Vertiefungsseite

1 Faltung

Ein kleiner Filter w gleitet über die Eingabe z (1D-Notes; k Filterbreite, s Schrittweite (Stride), ℓ Startversatz, i Ausgabeposition):

$$\text{Conv}(z)_i = \sum_{j=1}^k w_j z_{is-\ell+(j-1)}$$

Parameter-Sharing: $O(km)$ statt $O(m^2)$
Parameter. Ausgabegröße: $\lfloor \frac{n+2p-k}{s} \rfloor + 1$ (n Eingabelänge, Padding p , Stride s , k Filterbreite).

2 Aufbau eines CNN (Web)

Conv $\rightarrow$ ReLU $\rightarrow$ Pooling wiederholen, am Ende Flatten + voll verbundene Schicht + Softmax. **Max-Pooling** (2×2) halbiert die Auflösung und macht robust gegen kleine Verschiebungen. Parameter je Conv-Schicht: $k^2 c_{in} c_{out} + c_{out}$.

3 Skizze: 1D-Filter



4 Warum CNNs funktionieren

- **Lokalität:** Kanten, Ecken hängen nur von Nachbarpixeln ab.
- **Translationsäquivalenz:** derselbe Filter erkennt ein Muster überall.
- **Hierarchie:** tiefe Schichten kombinieren Kanten zu Formen zu Objekten (Rezeptives Feld wächst).
- Weniger Parameter $\Rightarrow$ weniger Overfitting, effizient.

5 RNN: Gedächtnis über Zeit

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b), \quad y_t = W_y h_t$$

Gleiche Gewichte in jedem Zeitschritt; der versteckte Zustand h_t trägt die Historie. Training per Backpropagation Through Time: der Gradient wird über viele Faktoren W_h multipliziert $\Rightarrow$ **Vanishing/Exploding Gradients**, lange Abhängigkeiten gehen verloren.

Beispiel: 1D-Faltung von Hand

$z = (1, 2, 3, 4, 5)$, Filter $w = (1, 0, -1)$, Stride 1, kein Padding.

Lösung Größe $\frac{5-3}{1} + 1 = 3$. Ausgabe:

$1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) = -2$; $2 \cdot 4 = -2$; $3 \cdot 5 = -2 \Rightarrow (-2, -2, -2)$ (konstanter Anstieg $\rightarrow$ konstante Antwort: der Filter ist ein Kantendetektor).

CNN-Parameter: 28×28 , Conv 3×3 ($1 \rightarrow 8$): $3 \cdot 3 \cdot 1 \cdot 8 + 8 = 80$; nach 2 Blöcken und FC ($784 \rightarrow 10$): $80 + 1168 + 7850 = 9098$ (Code: Vertiefungsseite).

6 LSTM: Gates gegen Vergessen (Web)

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \text{ (Vergessen)}$$

$$i_t = \sigma(W_i[\cdot] + b_i), \quad \tilde{c}_t = \tanh(W_c[\cdot] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t), \quad o_t = \sigma(W_o[\cdot] + b_o)$$

Der Zellzustand c_t ist eine "Datenautobahn" mit additivem Update $\Rightarrow$ Gradient bleibt erhalten. GRU: vereinfachte Variante.

7 Von RNN zu Attention

RNNs verarbeiten Sequenzen nacheinander (nicht parallelisierbar, Gedächtnis begrenzt). **Self-Attention** (Kapitel Foundation Models) lässt jedes Token direkt auf alle anderen zugreifen und ist parallelisierbar $\Rightarrow$ Transformer haben RNNs bei Sprache weitgehend abgelöst.

8 Vorteile

- ✓ CNN: wenige Parameter, Bildstruktur
- ✓ LSTM: lange Abhängigkeiten
- ✓ beide End-to-End trainierbar

9 Grenzen

- ✗ CNN: kaum globaler Kontext
- ✗ RNN: sequentiell, langsam
- ✗ alle: viel Daten und Rechenzeit

10 Key Takeaways

- ✓ Faltung: lokal + geteilte Gewichte
- ✓ RNN: h_t als Gedächtnis
- ✓ LSTM-Gates lösen Vanishing Gradients

? Selbsttest: kannst du das erklären?

F1 Wie groß ist die Faltungsausgabe?

A: $\lfloor (n + 2p - k) / s \rfloor + 1$ (Länge n , Filter k , Padding p , Stride s).

F2 Warum haben CNNs weniger

Parameter als voll verbundene Netze?

A: Derselbe kleine Filter wird an allen Positionen wiederverwendet (Sharing).

F3 Wie helfen LSTM-Gates?

A: Additives Update des Zellzustands mit Vergessens-/Eingabe-Gate erhält den Gradienten über lange Zeit.

„Die richtige Architektur ist Vorwissen über die Struktur der Daten.“

Foundation Models

Selbstüberwachtes Lernen & Transformer

Labels aus den Daten selbst erzeugen: maskierte oder nächste Teile vorhersagen.



Merke:

Next-Token:

$$\mathcal{L} = - \sum_t \log P(x_t | x_{<t}; \theta)$$

Quelle: main_notes.pdf (Self-supervised learning and foundation models)

1 Motivation

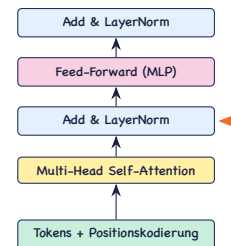
Annotierte Daten sind teuer.

Selbstüberwachtes Lernen: nutze Struktur in den Daten selbst als Supervision (Pretraining auf riesigen unlabeled Daten), dann Anpassung an Zielaufgaben.

2 Pretraining: Vision

- **Kontrastiv** (SimCLR, MoCo): zwei Augmentierungen desselben Bildes = positiv, andere Bilder = negativ.
- **Masked Image Modeling** (MAE): verdeckte Bildteile aus dem Kontext rekonstruieren.

3 Skizze: Transformer-Block



4 Self-Attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Jedes Token bildet Query, Key, Value (X Token-Matrix, W_Q, W_K, W_V gelernte Matrizen, d_k Key-Dimension; $Q = XW_Q$, $K = XW_K$, $V = XW_V$) und mittelt die Values nach Ähnlichkeit. **Multi-Head:** mehrere Attention-Köpfe parallel.

`</> Self-Attention`

```
S ← QK⊤ / √dk
P ← softmax(S) # zeilenweise
return PV
```

Beispiel: Attention mit 2 Tokens

$d_k = 2$, $Q = K = I$, $V = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$.

Lösung $QK^\top / \sqrt{2} = \text{diag}(0.71, 0.71)$. Zeile 1:

$\text{softmax}(0.71, 0) = (0.67, 0.33)$.

Ausgabe Zeile 1: $0.67 \cdot (1, 2) + 0.33 \cdot (3, 4) = (1.66, 2.66)$ - Mischung, die stärker Token 1 gewichtet.

5 Bausteine & Training

Feed-Forward: positionsweises MLP nach der Attention.

Positionskodierung: Attention ist permutationsinvariant, also Positionsinformation addieren. Residual + LayerNorm stabilisieren.

Pretraining: Sprachmodell sagt das nächste Token voraus, $\mathcal{L} = - \sum_t \log P(x_t | x_1 \dots x_{t-1})$.

Zero-Shot / In-Context: Aufgaben lösen allein durch Prompt (mit Beispielen), ohne Fine-Tuning.

6 Foundation-Model-Paradigma

1. **Pretrain** selbstüberwacht auf riesigen Daten
 2. **Adaptieren:** Fine-Tuning, Prompting, In-Context-Learning
 3. **Einsetzen** für viele Aufgaben mit wenig Labels
- Skalierung (Daten, Parameter, Rechenleistung) verbessert Fähigkeiten, oft emergent. (Siehe auch Double Descent: große Modelle generalisieren trotzdem.)

7 Vorteile

- ✓ wenige Labels nötig
- ✓ ein Modell für viele Aufgaben
- ✓ skaliert mit Daten

8 Grenzen

- ✗ enorme Rechen-/Datenkosten
- ✗ schwer interpretierbar
- ✗ übernimmt Verzerrungen der Daten

9 Key Takeaways

- ✓ Supervision aus den Daten selbst
- ✓ Attention: $\text{softmax}(QK^\top / \sqrt{d_k})V$
- ✓ Pretrain → Adaptieren

? Selbsttest: kannst du das erklären?

F1 Was ist selbstüberwachtes Lernen?

A: Labels aus den Daten selbst (nächstes Token, maskierte Teile, Augmentierungen).

F2 Formel der Self-Attention?

A: $\text{softmax}(QK^\top / \sqrt{d_k})V$.

F3 Wozu Positionskodierung?

A: Attention ist permutationsinvariant $\Rightarrow$ Position explizit addieren.

„Lies das ganze Internet, sage das nächste Wort vorher -- und lerne dabei die Welt.“

Bias & Varianz

Zu einfach $\Rightarrow$ Bias (underfitting). Zu flexibel $\Rightarrow$ Varianz (overfitting). $\heartsuit$

Generalisierung, Underfitting, Overfitting & Double Descent

Merke:
Test-MSE
 $= \sigma^2 + \text{Bias}^2 + \text{Varianz}$

Quelle: main_notes.pdf (Generalization, Kap. 8), notes/cs229-notes4.pdf, notes/error-analysis.pdf

1 Intuition

Bias: Fehler, der bleibt, selbst mit unendlich vielen Daten – die Hypothesenklasse ist zu schwach (z. B. Gerade statt Parabel).

Varianz: wie stark h_S zwischen verschiedenen Trainingssets gleicher Größe schwankt – das Modell lernt Rauschen mit.

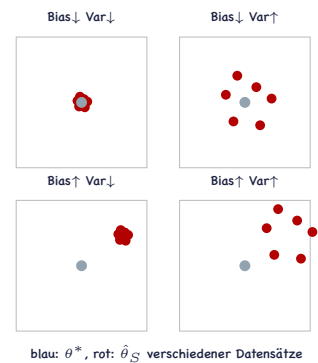
2 Zerlegung (Regression)

$$\mathbb{E}[(y - h_S(x))^2] = \underbrace{\sigma^2}_{\text{Rauschen}} + \underbrace{(h^* - \bar{h})^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{h} - h_S)^2]}_{\text{Varianz}}$$

Legende: y Beobachtung, h_S auf Trainingsmenge S gelerntes Modell, h^* wahre Funktion, σ^2 Rauschvarianz, $\mathbb{E}_S$ Erwartung über Trainingsmengen.

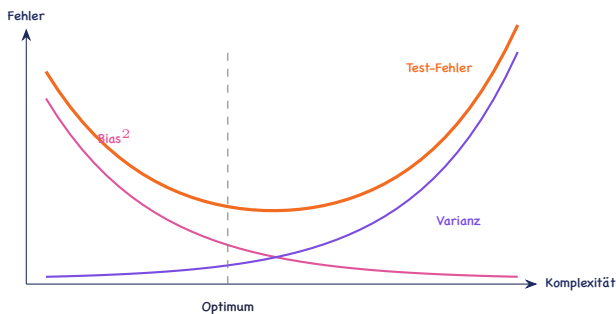
$\bar{h} = \mathbb{E}_S[h_S]$ ist das "mittlere Modell" (Zentrum der Punktwolke rechts).

3 Parameterraum: Zielscheiben

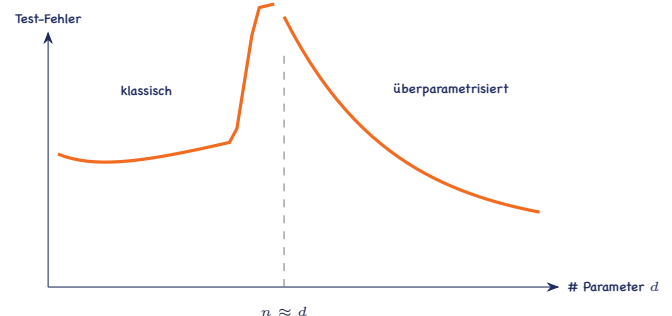


blau: θ^* , rot: θ_S verschiedener Datensätze

4 Modellkomplexität



5 Double Descent



Im überparametrisierten Regime wählt Gradient Descent implizit die Minimum-Norm-Lösung – oft gut.

Beispiel: Polynomgrad wählen (illustrative Zahlen)

Daten aus einer Parabel + Rauschen.

Grad	1	2	5
Train-MSE	0.30	0.05	0.00
Test-MSE	0.32	0.06	0.45

Grad 1: beide hoch $\Rightarrow$ **Bias**. Grad 5: Train ≈ 0 , Test hoch $\Rightarrow$ **Varianz**. Grad 2: bester Kompromiss.

6 Diagnose-Regeln

- **Hoher Bias:** Trainingsfehler schon hoch, kleine Lücke zum Testfehler; mehr Daten helfen nicht.
- **Hohe Varianz:** große Lücke Train/Test; Testfehler sinkt noch mit mehr Daten.
- Irreduzibles Rauschen σ^2 kann man nie eliminieren.

7 Gegen Bias

- komplexeres Modell / mehr Features
- weniger Regularisierung
- länger trainieren

8 Gegen Varianz

- mehr Trainingsdaten
- Regularisierung, weniger Features
- Bagging / Ensembles

9 Key Takeaways

- ✓ $\sigma^2 + \text{Bias}^2 + \text{Var}$
- ✓ Bias: Klasse zu schwach
- ✓ Varianz: Datensatz-Empfindlichkeit
- ✓ Double Descent bei $n \approx d$

? Selbsttest: kannst du das erklären?

F1 Zerlegung des Test-MSE?

A: σ^2 (Rauschen) + Bias^2 + Varianz.

F2 Woran erkennt man hohen Bias?

A: Trainingsfehler schon hoch, kleine Lücke zum Testfehler.

F3 Was ist Double Descent?

A: Testfehler steigt bei $n \approx d$ und fällt im überparametrisierten Regime erneut.

„Ein Modell muss so komplex sein wie die Daten – und keinen Deut komplexer.“

Lerntheorie

Wann darf ich dem Trainingsfehler trauen? Die Antwort liefert die Lerntheorie.



Hoeffding, Uniform Convergence & VC-Dimension

Merke:
Generalisierungslücke
 $\approx \sqrt{\log |\mathcal{H}|/n}$: nur
logarithmisch in $|\mathcal{H}|$!

Quelle: notes/cs229-notes4.pdf, extra-notes/hoeffding.pdf (J. Duchi), main_notes.pdf (Kap. 8)

1 Die Frage

Trainingsfehler

$J(h) = \frac{1}{n} \sum_i \mathbf{1}\{h(x^{(i)}) \neq y^{(i)}\}$ vs.
wahrer Fehler $L(h)$ (h Hypothese, $\mathcal{H}$
Hypothesenraum, 1 Indikator, n
Trainingsgröße). Wir wollen:

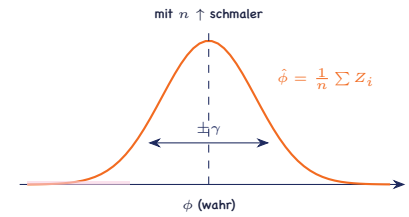
$|J(h) - L(h)|$ klein für alle $h \in \mathcal{H}$

(uniform convergence), denn $\hat{h}$ wird erst
nach Ansicht der Daten gewählt.

2 Werkzeugkasten

- **Markov:** $P(Z \geq t) \leq \mathbb{E}[Z]/t$ ($Z \geq 0$)
- **Chebyshev:**
 $P(|Z - \mathbb{E}Z| \geq t) \leq \text{Var}(Z)/t^2$
- **Chernoff:** exponentiell, über $\mathbb{E}[e^{\lambda Z}]$
- **Union Bound:** $P(\cup A_i) \leq \sum P(A_i)$

3 Skizze: Konzentration



4 Hoeffdingsche Ungleichung

Seien $Z_i \in [a, b]$ unabhängig (t : erlaubte Abweichung des
Mittelwerts). Dann für $t \geq 0$:

$$P\left(\left|\frac{1}{n} \sum_i (Z_i - \mathbb{E}Z_i)\right| \geq t\right) \leq 2 \exp\left(-\frac{2nt^2}{(b-a)^2}\right)$$

Für Fehler-Indikatoren ($a=0, b=1$):

$P(|J(h) - L(h)| \geq \gamma) \leq 2e^{-2\gamma^2 n}$. Beweis: Chernoff + Hoeffdings
Lemma $\mathbb{E}[e^{\lambda(Z - \mathbb{E}Z)}] \leq e^{\lambda^2(b-a)^2/8}$.

5 Endliche Hypothesenklasse

Union Bound über $k = |\mathcal{H}|$ Hypothesen (γ Toleranz, δ
Fehlerwahrscheinlichkeit, $\hat{h}$ ERM-Lösung):

$P(\exists h : |J - L| > \gamma) \leq 2k e^{-2\gamma^2 n}$. Mit Wahrscheinlichkeit
 $\geq 1 - \delta$:

$$|J(h) - L(h)| \leq \sqrt{\frac{\log(2k/\delta)}{2n}} \quad \forall h \in \mathcal{H}$$

Stichprobenkomplexität: $n \geq \frac{1}{2\gamma^2} \log \frac{2k}{\delta}$. Für ERM:
 $L(\hat{h}) \leq \min_h L(h) + 2\gamma$.

6 VC-Dimension

$VC(\mathcal{H})$ = Größe der größten Punktmenge, die $\mathcal{H}$ in jeder
Beschriftung korrekt trennen kann ("shattern"). Lineare
Klassifikatoren in $\mathbb{R}^d$: $VC = d + 1$.

Für unendliches $\mathcal{H}$ gilt (mit Wkt. $\geq 1 - \delta$; $O(\cdot)$: bis auf
Konstanten)

$$L(h) \leq J(h) + O\left(\sqrt{\frac{VC}{n} \log \frac{n}{VC}} + \frac{1}{n} \log \frac{1}{\delta}\right)$$

$\Rightarrow$ Bedarf an Daten wächst linear mit VC.

Beispiel: zwei Rechnungen

(a) $k = 10^6$ Hypothesen, $\delta = 0.05$, $\gamma = 0.05$:

$$n \geq \frac{\ln(2 \cdot 10^6 / 0.05)}{2 \cdot 0.05^2} = \frac{17.5}{0.005} \approx 3500 \text{ Beispiele.}$$

(b) Geraden in 2D ($d=2$): 3 Punkte in allgemeiner Lage lassen
sich in allen 2^3 Beschriftungen trennen $\Rightarrow VC = 3 = d + 1$. Vier
Punkte in XOR-Anordnung nicht.

7 Grenzen der Theorie

- Schranken oft sehr pessimistisch
- Deep Learning: $VC \gg n$, trotzdem
gute Generalisierung (Double Descent)

8 Key Takeaways

- ✓ Hoeffding: $2e^{-2n\gamma^2}$
- ✓ Union Bound $\Rightarrow \log |\mathcal{H}|$
- ✓ VC-Dim. für unendliche $\mathcal{H}$

9 Querverweise

- Bias-Varianz: γ ist die Varianz-Seite
- Regularisierung senkt effektive
Komplexität
- Perceptron: Fehlerschranke ähnlich im
Geist

? Selbsttest: kannst du das erklären?

F1 Was sagt Hoeffding?

A: $P(|\bar{Z} - \mathbb{E}\bar{Z}| \geq t) \leq 2e^{-2nt^2/(b-a)^2}$.

F2 Woher kommt $\log |\mathcal{H}|$?

A: Union Bound über $|\mathcal{H}|$ Hypothesen
(uniform convergence).

F3 VC-Dimension linearer Klassifikatoren
im $\mathbb{R}^d$?

A: $d + 1$.

„Mit genug Daten und einer nicht zu großen Hypothesenklasse wird aus Training Vertrauen.“

Große Gewichte bestrafen: das Modell wird einfacher, die Varianz sinkt. ❤️

Regularisierung & Modellauswahl

Overfitting bändigen, Hyperparameter richtig wählen

Merke:
L2 = Gauß-Prior, L1 = Laplace-Prior (MAP-Sicht)

Quelle: main_notes.pdf (Regularization and model selection), notes/cs229-notes5.pdf

1 Idee

Zum Verlust einen **Strafterm** addieren:

L2 (Ridge): $J_\lambda = J + \frac{\lambda}{2} \|\theta\|_2^2$

L1 (Lasso): $J_\lambda = J + \lambda \|\theta\|_1$

λ steuert den Kompromiss Bias $\leftrightarrow$ Varianz (J : ursprünglicher Verlust, $\|\theta\|_2^2 = \sum \theta_j^2$, $\|\theta\|_1 = \sum |\theta_j|$).

2 Bayessche Sicht (MAP)

$$\theta_{\text{MAP}} = \arg \max_{\theta} [\log p(y|X, \theta) + \log p(\theta)]$$

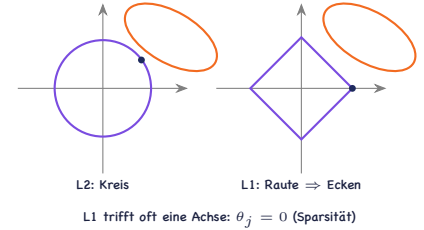
Gauß-Prior $\theta_j \sim \mathcal{N}(0, \tau^2)$ (τ^2

Prior-Varianz) $\Rightarrow$ L2. Laplace-Prior

$p(\theta_j) \propto e^{-|\theta_j|/b} \Rightarrow$ L1. Lineare

Regression: $\theta = (X^T X + 2\lambda\sigma^2 I)^{-1} X^T y$
(immer invertierbar; σ^2 Rauschvarianz, b Laplace-Skala, I Einheitsmatrix).

3 Skizze: L1 vs. L2



4 Kreuzvalidierung

K-Fold Cross-Validation

teile Daten in K gleich große Teile

for jeden Kandidaten λ :

for $k = 1..K$:

trainiere auf allen Teilen außer k

$e_k \leftarrow$ Fehler auf Teil k

$$CV(\lambda) \leftarrow \frac{1}{K} \sum_k e_k$$

wähle $\lambda^* = \arg \min CV(\lambda)$

trainiere neu auf allen Trainingsdaten

5 Modellauswahl-Strategien

- **Hold-out** (z. B. 70/30 oder Train/Val/Test): schnell, aber verschwendet Daten.
- **K-Fold** ($K=5, 10$): stabilere Schätzung.
- **Leave-one-out** ($K=n$): fast unverzerrt, teuer.
- **Feature-Auswahl**: Forward Search (Features einzeln hinzufügen), Backward Search (entfernen), Filter (Korrelation).

Testset nur einmal am Ende benutzen!

Beispiel: Ridge in 1D

Daten $(1, 1), (2, 2), (3, 2)$, Modell $y \approx \theta x$ (ohne Bias). $\sum xy = 11$, $\sum x^2 = 14$.

Lösung $\theta_\lambda = \frac{\sum xy}{\sum x^2 + \lambda} = \frac{11}{14 + \lambda}$

$\lambda = 0$: $\theta = 0.786$ | $\lambda = 2$: $\theta = 0.688$ | $\lambda = 14$: $\theta = 0.393$

Je größer λ , desto stärker schrumpft θ Richtung 0.

6 Implizite Regularisierung

Auch ohne Strafterm regularisiert Gradient Descent: im überparametrisierten Regime wählt SGD unter allen Interpolatoren bevorzugt die mit minimaler Norm (ähnlich L2). Frühes Stoppen wirkt ähnlich.

Tipp: λ per Kreuzvalidierung auf logarithmischem Gitter suchen ($10^{-4}, \dots, 10^2$).

7 Vorteile

- ✓ weniger Overfitting
- ✓ L1: automatische Feature-Auswahl
- ✓ stabilere Lösung ($X^T X$ invertierbar)

8 Grenzen

- ✗ λ muss gewählt werden
- ✗ zu großes $\lambda \Rightarrow$ Bias
- ✗ Features vorher skalieren

9 Key Takeaways

- ✓ L2 schrumpft, L1 macht spärlich
- ✓ Regularisierung = Prior (MAP)
- ✓ λ via Kreuzvalidierung

? Selbsttest: kannst du das erklären?

F1 L1 vs. L2?

A: L2 schrumpft gleichmäßig; L1 erzeugt Sparsität (Nullen).

F2 Bayessche Deutung?

A: L2 = Gauß-Prior, L1 = Laplace-Prior (MAP).

F3 Wozu K-Fold-CV?

A: Hyperparameter wählen, danach auf allen Daten neu trainieren.

„Regularisierung ist Vorsicht in Formeln: lieber ein bisschen zu einfach als zu eifrig.“

Gute Modelle brauchen gute Validierung und die richtigen Hyperparameter. ❤️

Validierung & Hyperparameter-Tuning

Cross-Validation, Grid/Random Search, Nested CV

Merke:
Testset nur einmal am Ende benutzen!

Quelle: hand-notes/ML Notes.pdf (Cross-Validation & Hyperparameter Tuning), notes/os229-notes5.pdf (Model selection)

1 Warum Kreuzvalidierung?

Ein einzelner Train/Test-Split liefert eine verrauschte Schätzung des Generalisierungsfehlers. CV nutzt mehrere Splits und mittelt $\Rightarrow$ stabilere Schätzung, bessere Datennutzung.

2 K-Fold

Daten in K Teile; jeder Teil ist einmal Validierung, der Rest Training:

$$CV = \frac{1}{K} \sum_{k=1}^K \text{Score}_k$$

Üblich $K = 5$ oder 10 . Danach: mit dem besten Hyperparameter auf allen Daten neu trainieren.

3 Skizze: 5 Folds

1	Test	Train	Train	Train	Train
2	Train	Test	Train	Train	Train
3	Train	Train	Test	Train	Train
4	Train	Train	Train	Test	Train
5	Train	Train	Train	Train	Test

4 CV-Varianten

- **Stratified K-Fold:** Klassenanteile pro Fold erhalten (Ungleichgewicht).
- **Leave-One-Out:** $K = n$, fast unverzerrt, sehr teuer.
- **Repeated K-Fold:** mit neuen Zufallssplits wiederholen.
- **Zeitreihen-CV (Walk-Forward):** Training immer vor der Validierung, nie in die Zukunft blicken.
- **Group-CV:** zusammengehörige Beispiele nie über Folds verteilen.

6 Grid vs. Random Search

`</>` Random Search mit CV

repeat T mal:

ziehe Kombination h zufällig aus dem Suchraum

$s(h) \leftarrow$ CV-Score von Modell(h)

wähle $h^* = \arg\max_h s(h)$; trainiere neu auf allen Daten

Grid: alle Kombinationen (exponentiell viele). Random: oft effizienter, weil meist nur wenige Hyperparameter wirklich zählen.

5 Parameter vs. Hyperparameter

Parameter werden gelernt (Gewichte θ , W). Hyperparameter legt man vorher fest und tut sie (Lernrate, λ , C , γ , k , Baumtiefe, K bei K-Means, Netzgröße). Gute Hyperparameter $\Rightarrow$ besseres Modell; sie nie mit dem Testset wählen.

7 Nested Cross-Validation

Wird der selbe CV-Score zum Tunen und zum Berichten genutzt, ist er optimistisch verzerrt. Lösung:

`</>` Nested CV

for äußerer Fold $k = 1..K$:

Trainingsteil $\rightarrow$ innere CV: wähle h^*

trainiere mit h^* auf dem ganzen Trainingsteil

bewerte auf dem äußeren Testfold $\rightarrow s_k$

Bericht: $\frac{1}{K} \sum_k s_k$

Beispiel: Auswertung & Rechenaufwand

5-Fold-Scores (0.82, 0.85, 0.80, 0.84, 0.83).

Lösung Mittel = **0.828**, Standardabweichung $s \approx 0.019 \Rightarrow$

"0.83 $\pm$ 0.02".

Random Forest, Gitter $3 \times 2 = 6$ Kombinationen, 5-Fold: $6 \cdot 5 = 30$ Fits (+1 Refit).

Nested (5 außen, 3 innen): $5 \cdot (3 \cdot 6 + 1) = 95$ Fits – deshalb bei großen Modellen Random Search.

8 Tuning-Workflow

1. Suchraum und Metrik festlegen
2. Suchstrategie (Grid/Random/Bayes)
3. CV-Score je Kombination
4. beste Kombination wählen
5. auf allen Trainingsdaten neu trainieren
6. einmal auf dem Testset bewerten

9 Fallstricke

- × Testset zum Tunen benutzen
- × Vorverarbeitung vor dem Split (Leakage) $\rightarrow$ Pipeline
- × ungeeignete CV für Zeitreihen/Gruppen
- × zu große Suche $\Rightarrow$ Overfitting auf den Validierungsfolds

10 Key Takeaways

- ✓ Mehr Splits $\Rightarrow$ stabilere Schätzung; Stratified bei Ungleichgewicht, Walk-Forward bei Zeitreihen
- ✓ Random Search $\geq$ Grid bei vielen Hyperparametern; Nested CV für ehrliche Bewertung

? Selbsttest: kannst du das erklären?

F1 Wozu Nested CV?

A: Tuning und Bewertung trennen: sonst ist der CV-Score optimistisch.

F2 Warum Walk-Forward bei Zeitreihen?

A: Zufällige Folds würden Zukunftsdaten ins Training mischen (Leakage).

F3 Warum am Ende neu auf allen Daten trainieren?

A: CV dient nur der Auswahl; das finale Modell soll alle Trainingsdaten nutzen.

„Validiere klug, tune weise, teste nur einmal.“

Erst diagnostizieren, dann reparieren -- nicht planlos herumprobieren. ❤️

Praktische Fehleranalyse

Diagnostik, Error & Ablative Analysis

Merke:
Lernkurve lesen: Lücke $\Rightarrow$ Varianz, hoher Train-Fehler $\Rightarrow$ Bias

Quelle: materials/ML-advice.pdf (A. Ng), notes/error-analysis.pdf (Y. Le Colonneo), section/error-analysis.pdf

1 Problem

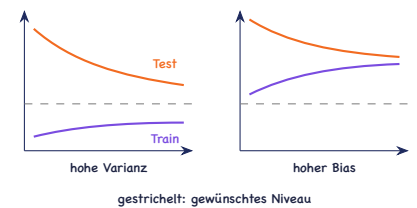
Spamfilter mit 20% Testfehler. Mögliche Maßnahmen: mehr Daten, weniger/mehr Features, mehr Iterationen, Newton, anderes λ , SVM... Blindes Ausprobieren kostet Wochen. **Diagnose zuerst!**

2 Lernkurven

Plotte Train- und Testfehler über der Trainingsgröße m :

- **Varianz:** große Lücke, Test fällt noch $\Rightarrow$ mehr Daten helfen
- **Bias:** Train schon hoch, kleine Lücke $\Rightarrow$ Modell ausdrucksstärker machen

3 Skizze: zwei Diagnosen



4 Welche Maßnahme behebt was?

Maßnahme	behebt
mehr Trainingsbeispiele	hohe Varianz
weniger Features	hohe Varianz
mehr / bessere Features	hohen Bias
mehr Iterationen, Newton	Optimierungsalgorithmus
anderes λ / C , andere Klasse	Optimierungsziel

5 Algorithmus oder Ziel?

Ein SVM schlägt BLR (Bayessche Logistische Regression) bei der wahren Metrik $a(\theta)$, obwohl BLR das Ziel $J(\theta)$ maximiert (θ Parameter, J Zielfunktion, a Metrik, die uns wirklich interessiert). Vergleiche $J(\theta_{SVM})$ mit $J(\theta_{BLR})$:

```

</> Diagnose
if  $J(\theta_{BLR}) < J(\theta_{SVM})$ :
    Problem = Optimierer (nicht konvergiert)
     $\rightarrow$  mehr Iterationen, Newton, andere  $\alpha$ 
else: #  $J$  erreicht, aber  $a$  schlecht
    Problem = Zielfunktion falsch
     $\rightarrow$  anderes  $\lambda/C$ , Gewichte, andere Klasse
    
```

6 Error Analysis (Pipeline)

Bei mehrstufigen Systemen jede Komponente einzeln durch die Ground Truth ersetzen und die Gesamtgenauigkeit messen:

+ perfektes Preprocessing	85.1 %
+ perfekte Gesichtsdetektion	91 %
+ perfekte Augen-Segmentierung	95 %

(Start: 85.1%) Große Sprünge zeigen, wo Arbeit sich lohnt.

7 Ablative Analyse

Umgekehrt: vom guten System aus Komponenten entfernen.

Gesamtsystem	99.9 %
– Rechtschreibung	99.0 %
– Text-Parser	95 %
– Bild-Features (Baseline)	94 %

Zeigt, welche Features wirklich beitragen (gut für Publikationen).

Beispiel: Fehler manuell zählen

Katzenklassifikator, Accuracy 90%. Von 100 Fehlern sind ...

Lösung 5 Hunde: Selbst mit perfektem Hunde-Filter steigt die Accuracy nur 90% $\rightarrow$ **90.5%** – nicht lohnend.

50 Hunde: $\rightarrow$ **95%** – hier lohnt sich die Arbeit!

8 Vorgehen in 5 Schritten

1. Baseline bauen, Metrik festlegen
2. Lernkurve $\rightarrow$ Bias oder Varianz?
3. Optimierer vs. Ziel prüfen
4. Error Analysis der Pipeline
5. ca. 100 Fehler kategorisieren, priorisieren

9 Key Takeaways

- ✓ Diagnose vor Aktion
- ✓ Lernkurve: Lücke = Varianz, Train hoch = Bias
- ✓ J vs. a : Optimierer oder Ziel schuld?
- ✓ Fehleranalyse zeigt die Obergrenze des Gewinns

? Selbsttest: kannst du das erklären?

F1 Lernkurve: Varianz oder Bias?

A: Große Lücke $\Rightarrow$ Varianz; hoher Trainingsfehler $\Rightarrow$ Bias.

F2 $J(\theta_{BLR}) < J(\theta_{SVM})$ bedeutet?

A: Optimierer hat sein Ziel nicht erreicht (Konvergenzproblem).

F3 Error vs. Ablative Analysis?

A: Error: Komponenten durch Ground Truth ersetzen; Ablation: Komponenten entfernen.

„Wer nicht misst, verbessert das Falsche.“

Die Trainings-Kosten sind nur ein Proxy -- Metriken übersetzen das echte Ziel in eine Zahl. ❤️

Evaluationsmetriken

Confusion Matrix, Precision/Recall, ROC & PR

Merke:
Accuracy < AUROC < AUPRC bei starkem Klassenungleichgewicht

Quelle: section/evaluation_metrics_spring2020.pdf (Y. Chen, nach A. Avati)

1 Warum Metriken?

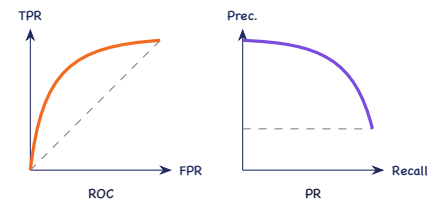
Nicht alle Fehler sind gleich teuer (übersehener Tumor vs. Fehlalarm). Metriken quantifizieren die Lücke zwischen Baseline, aktuellem und gewünschtem Ergebnis und steuern die Teamarbeit.

2 Konfusionsmatrix

	Label +	Label -
Vorhersage +	TP	FP
Vorhersage -	FN	TN

FP = Fehler 1. Art, FN = Fehler 2. Art

3 Skizze: ROC & PR



4 Punkt-Metriken

Metrik	Formel	Frage
Accuracy	$\frac{TP + TN}{\text{alle}}$	Anteil richtig
Precision	$\frac{TP}{TP + FP}$	Positive echt?
Recall (Sens.)	$\frac{TP}{TP + FN}$	Positive gefunden?
Specificity	$\frac{TN}{TN + FP}$	Negative gefunden?
F1	$\frac{2PR}{P + R}$	harm. Mittel (P, R)

Beispiel: Schwelle 0.5 vs. 0.6

20 Beispiele (10 pos., 10 neg.).

Lösung Schwelle 0.5: TP=9, FP=2, FN=1, TN=8: Acc = $\frac{17}{20} = 0.85$,

Prec = $\frac{9}{11} = 0.82$, Recall = 0.90, Spec = 0.80, F1 = **0.857**.

Schwelle 0.6: TP=7, FP=2, FN=3, TN=8: Acc = 0.75, Prec = 0.78, Recall = 0.70, F1 = 0.737.

Höhere Schwelle $\Rightarrow$ weniger Recall.

5 Schwellwert, ROC & PR

Score-Modelle (LogReg, SVM) brauchen einen **Schwellwert**:
niedriger $\Rightarrow$ Recall $\uparrow$, Precision $\downarrow$. Es gibt $n + 1$ effektive Schwellwerte.
ROC: Sensitivity über 1-Specificity; AUROC
= $P(\text{zufälliges Positiv höher als zufälliges Negativ})$, unabhängig von der Prävalenz.
PR-Kurve: Precision über Recall; AUPRC; robuster bei Ungleichgewicht.

6 Berechnung

ROC-Kurve per Schwellensean

sortiere Beispiele nach Score (absteigend)

$TP \leftarrow 0, FP \leftarrow 0$

for Beispiel in dieser Reihenfolge:

if $y = +1$: $TP \leftarrow TP + 1$ else: $FP \leftarrow FP + 1$

Punkt (FP/N_+ , TP/N_+) speichern # $N_{\pm}$: Anzahl Pos./Neg.

AUROC $\leftarrow$ Trapezregel über die Punkte

7 Klassenungleichgewicht

Prävalenz < 5%: Accuracy ist trivial hoch (immer Mehrheitsklasse), AUROC leicht hochzuhalten (viele Negative sehr niedrig scoren), AUPRC robuster.

Log-Loss/Brier-Score $\frac{1}{n} \sum (\hat{p}_i - y_i)^2$ ($\hat{p}_i$ vorhergesagte Wkt., $y_i \in \{0, 1\}$) bewerten Kalibrierung und strafen selbstsichere Fehler hart (proper scoring rules).

8 Welche Metrik?

- FN teuer $\rightarrow$ Recall
- FP teuer $\rightarrow$ Precision
- Ungleichgewicht $\rightarrow$ AUPRC, F1
- Wahrscheinlichkeiten $\rightarrow$ Log-Loss

9 Fallstricke

- × Accuracy täuscht bei seltenen Klassen
- × Test-Metrik nie zum Tuning nutzen
- × Ranking-Metriken ignorieren Kalibrierung

10 Key Takeaways

- ✓ TP/FP/FN/TN als Basis
- ✓ Precision $\leftrightarrow$ Recall Tradeoff
- ✓ AUROC vs. AUPRC bei Ungleichgewicht

* Weitere Metriken & Mehrklassen (aus den ML Notes)

FPR = $\frac{FP}{FP + TN}$ = 1 - Spec, FNR = $\frac{FN}{FN + TP}$ = 1 - Recall, NPV = $\frac{TN}{TN + FN}$. Mehrklassen: One-vs-Rest, dann Metriken je Klasse mitteln - Macro (einfacher Mittelwert), Micro (globale Zählung aller TP/FP/FN), Weighted (nach Klassenhäufigkeit). Beispiel (80 TP, 20 FP, 10 FN, 90 TN): Acc = 0.85, Prec = 0.80, Recall = 0.89, F_1 = 0.84. Wahl: ausgeglichen $\rightarrow$ Accuracy/ F_1 ; Ungleichgewicht $\rightarrow$ PR-AUC/ F_1 ; FP teuer $\rightarrow$ Precision/Spezifität; FN teuer $\rightarrow$ Recall; Ranking $\rightarrow$ ROC-AUC/PR-AUC.

? Selbsttest: kannst du das erklären?

F1 Precision vs. Recall?

A: Precision = $\frac{TP}{TP + FP}$, Recall = $\frac{TP}{TP + FN}$.

F2 Wann AUPRC statt AUROC?

A: Bei starkem Klassenungleichgewicht (AUROC bleibt leicht hoch).

F3 Wirkung einer höheren Schwelle?

A: Weniger positive Vorhersagen: Recall sinkt, Precision steigt meist.

„Was du misst, wird besser -- also miss, was wirklich zählt.“

Regressionsmetriken

MAE, MSE, RMSE, R^2 und Adjusted R^2

Wie gut ist ein Regressionsmodell wirklich? Fehlermaße und Varianzerklärung. ❤️

Merke:
MSE ↔ Gauß-Rauschen,
MAE ↔ Laplace-Rauschen

Quelle: hand-notes/ML Notes.pdf (Regression Metrics); Rechenbeispiel dort korrigiert ($R^2 = 0.874$, nicht 0.884: $SST = 12.30$); Bezug zu MLE aus dem Kursstoff

1 Notation

y_i wahr, $\hat{y}_i$ vorhergesagt, $\bar{y}$ Mittel der wahren Werte, n Beispiele, p Prädiktoren, Residuum $e_i = y_i - \hat{y}_i$.
Ziel: kleine Fehler und Generalisierung (auf dem Testset messen!).

2 Fehlerbasierte Metriken

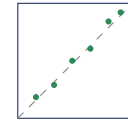
$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

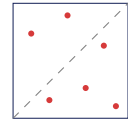
$$RMSE = \sqrt{MSE}$$

Kleiner ist besser. RMSE hat die Einheit von y ; MSE/RMSE bestrafen große Fehler stärker.

3 Skizze: gut vs. schlecht



gut: nahe an $y = \hat{y}$



schlecht: weit weg

4 R^2 und Adjusted R^2

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$R^2 = 1$: perfekt; $R^2 = 0$: nicht besser als der Mittelwert;
 $R^2 < 0$: schlechter als der Mittelwert.

$$R^2_{adj} = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

Adjusted bestraft zusätzliche Prädiktoren $\Rightarrow$ fairer Vergleich bei verschiedener Featureanzahl.

5 Welche Metrik wann?

Metrik	Einsatz
MAE	robust gegenüber Ausreißern, leicht interpretierbar
MSE / RMSE	große Fehler sind besonders schlimm
R^2	Anteil erklärter Varianz, skalenfrei
Adj. R^2	Modelle mit verschiedener Featurezahl vergleichen

Beispiel: 5 Hauspreise (in 100 k€)

$y = (5.0, 6.0, 7.5, 8.0, 9.5)$, $\hat{y} = (4.6, 5.5, 8.0, 7.2, 10.0)$, Fehler $(0.4, 0.5, -0.5, 0.8, -0.5)$.

Lösung MAE = $\frac{0.4+0.5+0.5+0.8+0.5}{5} = 0.54$; MSE = $\frac{1.55}{5} = 0.31$;

RMSE = 0.557.

$\bar{y} = 7.2$, SST = $4.84 + 1.44 + 0.09 + 0.64 + 5.29 = 12.30$, SSE = 1.55:

$R^2 = 1 - \frac{1.55}{12.30} = 0.874$ (maschinell nachgerechnet).

6 Warum genau diese Fehler? (MLE)

Gauß-Rauschen $\Rightarrow$ MLE = MSE minimieren (Least Squares).
Laplace-Rauschen $\Rightarrow$ MAE (robust). Huber-Verlust: quadratisch für kleine, linear für große Fehler – Kompromiss. Metrik und Trainingsverlust sollten zum Anwendungsziel passen (Kapitel Fehleranalyse: Zielfunktion vs. Metrik).

7 Fallstricke

- × nur eine Metrik betrachten
- × R^2 auf Trainingsdaten (steigt immer)
- × verschiedene Skalen vergleichen (MSE)

8 Weitere Metriken (Web)

- MAPE (prozentual, Problem bei $y \approx 0$)
- Median AE (sehr robust)
- Quantil-/Pinball-Verlust

9 Key Takeaways

- ✓ MAE, MSE, RMSE, R^2
- ✓ $R^2 = 1 - SSE/SST$
- ✓ Metrik zum Ziel passend wählen

? Selbsttest: kannst du das erklären?

F1 Was bedeutet $R^2 = 0$?

A: Das Modell ist nicht besser als die Vorhersage des Mittelwerts $\bar{y}$.

F2 Wann MAE statt MSE?

A: Bei Ausreißern; MSE quadriert große Fehler.

F3 Warum Adjusted R^2 ?

A: R^2 steigt mit jedem Feature; Adjusted bestraft überflüssige Prädiktoren.

„Keine einzelne Zahl erzählt die ganze Geschichte – lies mehrere Metriken zusammen.“

Feature Importance & SHAP

Nicht nur wie gut, sondern warum: welche Features treiben die Vorhersage? ❤️

Merke:
Shapley: faire Aufteilung der Vorhersage auf die Features

Globale und lokale Erklärungen von Modellen

Quelle: notes/cs229-notes-ensemble.pdf (Variable Importance), materials/critiques-ml-aut19.pdf (Interpretierbarkeit); Permutation Importance und SHAP (Lundberg & Lee 2017) als Web-Zusatz markiert

1 Warum erklären?

Vertrauen, Fehlersuche, Regulierung.

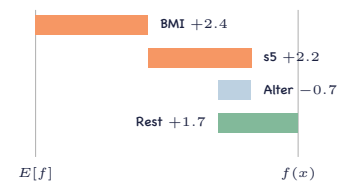
Global: welche Features zählen insgesamt?

Lokal: warum bekam dieses Beispiel diese Vorhersage? Erklärt wird das Modell, nicht die Welt.

2 Impurity Importance (Bäume)

Mittlere Impurity-Abnahme über alle Splits eines Features (Notes: "variable importance"). Gratis beim Training, aber auf Trainingsdaten berechnet und verzerrt bei vielen Ausprägungen und korrelierten Features.

3 Skizze: lokale Erklärung (schematisch)



4 Permutation Importance

Wichtig ist, was fehlt, wenn man es zerstört: Feature mischen und Leistungsverlust messen.

```
</> Permutation Importance
s0 ← Score des Modells auf Testdaten # R: Wiederholungen
for Feature j: repeat R mal:
    X^π ← X mit Spalte j zufällig permutiert
    s_j^(r) ← Score auf X^π
PI_j ← s0 - mean_r s_j^(r)
```

Modell-agnostisch; Problem: gemischte Werte sind evtl. unrealistisch.

6 SHAP

Shapley-Werte für ML: $v(S) = \mathbb{E}[f(x) | x_S]$ (unbekannte Features "ausintegrieren"). Additive Erklärung:

$$f(x) = \varphi_0 + \sum_{j=1}^d \varphi_j, \quad \varphi_0 = \mathbb{E}[f(X)]$$

TreeSHAP (exakt, schnell), KernelSHAP (modellagnostisch, gewichtete Regression), DeepSHAP. Global: $\frac{1}{n} \sum_i |\varphi_j^{(i)}|$.

7 Fallstricke

- × SHAP ≠ Kausalität: erklärt das Modell, nicht die Welt
- × korrelierte Features: Wichtigkeit verteilt sich, unrealistische Punkte
- × Aufwand $O(2^d) \Rightarrow$ Näherungen; Hintergrunddaten bestimmen φ_0

5 Shapley-Werte (Spieltheorie)

Features sind "Spieler", $v(S)$ der Wert einer Koalition, die Vorhersage der Gewinn:

$$\varphi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(n-|S|-1)!}{n!} [v(S \cup \{j\}) - v(S)]$$

= mittlerer Grenzbeitrag von j über alle Reihenfolgen. **Axiome:** Effizienz $\sum_j \varphi_j = v(N) - v(\emptyset)$, Symmetrie, Dummy (unwichtig $\Rightarrow 0$), Additivität. Eindeutig.

Beispiel: Shapley-Werte von Hand

$f(x_1, x_2) = 3x_1 + 2x_2 + 4x_1x_2$, Instanz $x = (1, 1)$, Baseline $(0, 0)$: $v(\emptyset) = 0$, $v(\{1\}) = 3$, $v(\{2\}) = 2$, $v(\{1, 2\}) = 9$.

Lösung $\varphi_1 = \frac{1}{2}(3-0) + \frac{1}{2}(9-2) = 5$; $\varphi_2 = \frac{1}{2}(2-0) + \frac{1}{2}(9-3) = 4$.

Summe = 9 = $f(x) - f(0)$ (Effizienz). Die Interaktion $4x_1x_2$ wird fair je zur Hälfte aufgeteilt (2 + 2).

10 Key Takeaways

- ✓ Impurity schnell, aber verzerrt $\rightarrow$ Permutation (Testdaten) robuster $\rightarrow$ SHAP: global und lokal, additiv
- ✓ Shapley = mittlerer Grenzbeitrag; $\sum \varphi_j = f(x) - \text{Baseline}$; Herleitung und Code: nächste Seite

8 Vergleich der Verfahren

Verfahren	Ebene	Misst	Daten	Schwäche
Koeffizienten w_j (linear)	global	Effekt pro Einheit	Training	nur lineare Modelle, skalenabhängig
Impurity Importance	global	Split-Gewinn	Training	Bias, verzerrt bei Korrelation
Permutation Importance	global	Leistungsverlust	Test	unrealistische Mischungen
SHAP (mean $ \varphi $)	global und lokal	Beitrag zur Vorhersage	Hintergrund + Instanz	teuer, nicht kausal

9 Einordnung: Leistungsmetriken, Feature Importance/SHAP, Fairness

Leistungsmetriken: wie gut? Feature Importance/SHAP: warum (braucht keine Labels). Fairness: für wen (Metrik je Gruppe, braucht ein sensibles Merkmal). Tabelle: Kapitel Fairness-Analyse.

? Selbsttest: kannst du das erklären?

F1 Was unterscheidet Permutation Importance von Impurity Importance?

A: Permutation misst den Leistungsverlust auf Testdaten, Impurity den Split-Gewinn auf Trainingsdaten (verzerrt bei vielen Ausprägungen).

F2 Was garantiert die Effizienz-Eigenschaft?

A: $\sum_j \varphi_j = f(x) - \mathbb{E}[f(X)]$: die Beiträge erklären die Vorhersage vollständig.

F3 Wodurch unterscheiden sich SHAP und Fairness-Metriken?

A: SHAP erklärt warum (Features), Fairness misst für wen (Gruppen).

„Erst wissen, wie gut das Modell ist -- dann verstehen, warum.“

Fairness-Analyse

Ein Modell kann im Mittel gut sein und trotzdem bestimmte Gruppen systematisch benachteiligen. ❤️

Gruppenmetriken: Demographic Parity, Equalized Odds & Co.

Merke:
Gleiche Accuracy ≠ gleiche Behandlung

Quelle: materials/critiques-ml-aut19.pdf (Ethische Fragen); Definitionen nach Hardt et al. 2016, Chouldechova 2017, Kleinberg et al. 2016 sowie fairlearn als Web-Zusatz markiert; Rechenbeispiel selbst erstellt und maschinell geprüft

1 Woher kommt Unfairness?

- **Historischer Bias** in Daten (frühere Entscheidungen)
- **Stichprobenbias**: Gruppen unterrepräsentiert
- **Label-Bias**: Ziel misst Ersatzgröße (Festnahme statt Straftat)
- **Proxies**: PLZ korreliert mit Herkunft

2 Setup

A sensibles Merkmal (Geschlecht, Herkunft, Alter), Y wahres Label, $\hat{Y}$ Vorhersage, S Score. Fairness heißt: eine Statistik von $\hat{Y}$ soll sich zwischen den Gruppen $A = a$ und $A = b$ nicht (zu stark) unterscheiden.

Merkmal einfach weglassen genügt nicht (Proxies).

3 Skizze: Konfusionsmatrix je Gruppe

	Gruppe A		Gruppe B	
	$Y=1$	$Y=0$	$Y=1$	$Y=0$
$\hat{Y}=1$	6	2	1	1
$\hat{Y}=0$	2	10	3	15

Accuracy beide 0.80!

4 Gruppenmetriken (Definitionen)

Demographic Parity: $P(\hat{Y}=1|A=a) = P(\hat{Y}=1|A=b)$

Equal Opportunity: $P(\hat{Y}=1|Y=1, a) = P(\hat{Y}=1|Y=1, b)$ (TPR)

Equalized Odds: TPR und FPR gleich

Predictive Parity: $P(Y=1|\hat{Y}=1, a) = \dots b$ (PPV)

Kalibrierung: $P(Y=1|S=s, a) = s \forall a$

Disparate Impact: $\frac{P(\hat{Y}=1|b)}{P(\hat{Y}=1|a)}$, **80%-Regel**: Verhältnis < 0.8 gilt als Warnsignal.

5 Was welche Metrik verlangt

- **Demographic Parity**: gleiche Auswahlrate, ignoriert das Label – passt, wenn Unterschiede in Y selbst Folge von Diskriminierung sind.
- **Equal Opportunity**: Qualifizierte haben gleiche Chance (TPR), z. B. Kreditvergabe.
- **Equalized Odds**: zusätzlich gleiche Fehlalarme (FPR).
- **Predictive Parity/Kalibrierung**: ein positives Ergebnis bedeutet in jeder Gruppe dasselbe.
- **Individuelle Fairness**: ähnliche Personen → ähnliche Ergebnisse.

Beispiel: Kreditmodell mit 40 Anträgen (je 20 pro Gruppe; Zahlen aus der Konfusionsmatrix oben)

Lösung Auswahlrate: $A = \frac{8}{20} = 0.40$, $B = \frac{2}{20} = 0.10 \Rightarrow$ Disparate Impact $= \frac{0.10}{0.40} = 0.25 < 0.8$ verletzt. TPR: $A = \frac{6}{8} = 0.75$, $B = \frac{1}{4} = 0.25$

(Differenz **0.50**). FPR: $A = \frac{2}{12} = 0.17$, $B = \frac{1}{16} = 0.06$. PPV: $A = 0.75$, $B = 0.50$. Accuracy: beide $\frac{16}{20} = 0.80$.

Das Modell erkennt kreditwürdige Antragsteller in Gruppe B nur zu 25% – die Gesamtgenauigkeit verrät das nicht (Kapitel Klassifikationsmetriken: Accuracy täuscht).

6 Unmöglichkeit (Kurzfassung)

Unterscheiden sich die Basisraten $p = P(Y=1)$ (hier 0.40 vs. 0.20), können Kalibrierung/PPV und gleiche FPR und FNR nicht gleichzeitig gelten – außer der Klassifikator ist perfekt. Fairness ist deshalb

eine Wertentscheidung: welches Kriterium passt zum Kontext (rechtlich, ethisch)? Herleitung: nächste Seite.

7 Gegenmaßnahmen

Pre-Processing: Daten reparieren (Reweighting, Resampling).

In-Processing: Nebenbedingung oder Strafterm im Training, adversariales Debiasing. **Post-Processing**: gruppenspezifische Schwellwerte nach dem Training (Hardt et al.). Jede Maßnahme kostet oft etwas Gesamtleistung.

8 Einordnung: Fairness, Feature Importance und Leistungsmetriken

	Leistungsmetriken	Feature Importance / SHAP	Fairness-Metriken
Frage	Wie gut ist das Modell?	Warum sagt es das?	Für wen und wie gleich gut?
Vergleicht	Vorhersage mit Label	Vorhersage mit Eingaben	Metrik zwischen Gruppen
Braucht A?	nein	nein (Proxy-Suche mit SHAP)	ja
Zusammenspiel	Gesamtwert versteckt Gruppen	zeigt, ob A/Proxies das Modell treiben	Metrik je Gruppe statt Mittel

9 Praxis-Workflow

1. Kontext und Schaden klären, Kriterium wählen
2. Metriken pro Gruppe berechnen (nicht nur global)
3. mit SHAP prüfen, ob A/Proxies stark wirken
4. Gegenmaßnahme wählen, Leistungsverlust messen
5. dokumentieren (Model Card), nach dem Deployment überwachen

10 Key Takeaways

- ✓ Gruppenmetriken vergleichen Statistiken zwischen Gruppen
- ✓ Accuracy allein verbirgt Ungleichbehandlung
- ✓ Kriterien widersprechen sich bei ungleichen Basisraten
- ✓ Vertiefung mit Herleitung und Code: nächste Seite

? Selbsttest: kannst du das erklären?

F1 Was ist der Unterschied zwischen Demographic Parity und Equal Opportunity?

A: DP gleicht die Auswahlrate an (ohne Label); Equal Opportunity die TPR unter den tatsächlich Positiven.

F2 Warum reicht es nicht, das sensible Merkmal zu entfernen?

A: Proxies (z. B. PLZ) tragen dieselbe Information.

F3 Was besagt die 80%-Regel?

A: Verhältnis der Auswahlraten < 0.8 deutet auf nachteilige Wirkung.

„Fairness ist keine Zahl, sondern eine begründete Entscheidung.“

Feature Engineering & Preprocessing

Rohdaten sind erst der Anfang: saubere Daten und gute Features schlagen oft das bessere Modell. ❤️

Merke:
Garbage in, garbage out!

Fehlende Werte, Skalierung, Kategorien, Ausreißer, Feature-Auswahl

Quelle: hand-notes/ML Notes.pdf (Feature Engineering & Preprocessing); Beispiele selbst ergänzt

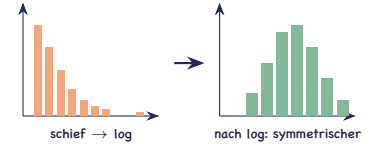
1 Warum?

Modelle lernen nur, was in den Features steckt. **Preprocessing** macht Daten sauber und nutzbar; **Feature Engineering** erzeugt zusätzliche Signale (Kombinationen, Transformationen, Zeitmerkmale). Iterativer Prozess.

2 Fehlende Werte

MCAR (zufällig), MAR (hängt von anderen Features ab), MNAR (hängt vom fehlenden Wert selbst ab).
Imputation: Mittel/Median (numerisch), Modus oder Kategorie "fehlt" (kategorial), KNN- oder Modell-Imputation; Zeilen/Spalten löschen, wenn zu viel fehlt.

3 Skizze: Ausreißer & Log



4 Skalierung

$$\text{Standardisierung: } z = \frac{x - \mu}{\sigma} \quad \text{Min-Max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(μ, σ : Mittel und Streuung der Trainingsdaten; $x_{\min}, x_{\max}$ kleinster/größter Wert.) Standardisierung: Mittel 0, Std 1, empfindlich bei Ausreißern. Min-Max: Bereich $[0, 1]$, sehr empfindlich bei Ausreißern; robust: Median/IQR.

Nötig für distanz- und gradientenbasierte Verfahren (k-NN, SVM, K-Means, PCA, neuronale Netze, GD); nicht für Bäume.

5 Kategoriale Variablen

Verfahren	Idee / Vorsicht
Label-Encoding	Zahl pro Kategorie; erzeugt Scheinordnung
One-Hot	eine 0/1-Spalte je Kategorie (Dummy-Falle: eine weglassen bei linearen Modellen)
Ordinal	sinnvolle Rangfolge (niedrig < mittel < hoch)
Target-Encoding	Mittelwert des Ziels je Kategorie (Leakage-Gefahr!)

Beispiel: Skalieren von $x = (10, 20, 30, 100)$

Lösung $\mu = 40, \sigma = \sqrt{\frac{900+400+100+3600}{4}} = 35.4$

Standardisierung: $z = (-0.85, -0.57, -0.28, 1.70)$.

Min-Max: $x' = \frac{x-10}{90} = (0, 0.11, 0.22, 1)$.

Der Ausreißer 100 quetscht die anderen Werte zusammen $\Rightarrow$ vorher cappen oder log-transformieren.

One-Hot: Stadt $\in \{\text{Delhi, Mumbai}\} \rightarrow (1, 0)$ bzw. $(0, 1)$.

6 Ausreißer & Transformationen

Erkennen: Boxplot/IQR-Regel (Q_1, Q_3 unteres/oberes Quartil, $IQR = Q_3 - Q_1$) ($x < Q_1 - 1.5 IQR$ oder $> Q_3 + 1.5 IQR$), z -Score > 3 . Behandeln: cappen (Winsorizing an Perzentilen), transformieren (log, Wurzel, Box-Cox), entfernen nur bei echten Fehlern (nicht bei seltenen, aber echten Werten). Weitere Transformationen: Binning, Interaktionen $A \times B$, Polynom-Features.

7 Feature-Auswahl

- **Filter:** Korrelation, χ^2 , Mutual Information – unabhängig vom Modell.
- **Wrapper:** Forward/Backward-Selektion, RFE – teuer, modellbezogen.
- **Embedded:** L1/Lasso, Elastic Net, Baum-Importance – Auswahl im Training.

8 Data Leakage vermeiden

Skalierer, Imputer und Encoder **nur auf dem Trainingsset anpassen** (fit) und dann auf Val/Test anwenden (transform). Am sichersten: alles in eine Pipeline (siehe Vertiefungsseite zu Workflow). Keine Zukunftsinformation in Zeitreihen-Features.

9 Zeit-Features

Aus Datum: Jahr, Monat, Wochentag, Quartal, Wochenende/Feiertag; Lag-Features, Rolling-Statistiken (Mittel, Std), Zeit seit letztem Ereignis – immer nur aus der Vergangenheit berechnen.

10 Key Takeaways

- ✓ Fehlende Werte, Ausreißer, Skalen zuerst klären
- ✓ Kategorien passend kodieren
- ✓ fit nur auf Train, transform überall

? Selbsttest: kannst du das erklären?

F1 Für welche Modelle ist Skalierung nötig?

A: Distanz-/gradientenbasierte (k-NN, SVM, K-Means, PCA, NN); nicht für Bäume.

F2 Warum One-Hot statt Label-Encoding?

A: Label-Encoding täuscht eine Ordnung vor, die es nicht gibt.

F3 Was ist Data Leakage beim Skalieren?

A: Mittel/Std aus dem gesamten Datensatz: Testinformation fließt ins Training.

„Gute Features sind halbe Modellierung.“

Ein guter Workflow macht aus einer Idee ein zuverlässiges, wiederholbares System.



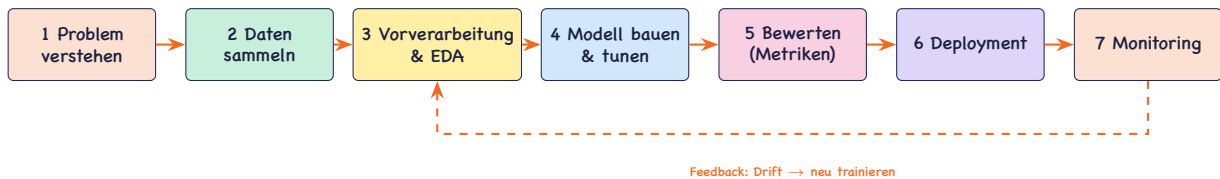
ML-Workflow & Best Practices

Von der Problemdefinition bis zum Monitoring

Merke:
Iterieren, validieren,
dokumentieren -- nicht hetzen

Quelle: hand-notes/ML Notes.pdf (ML Fundamentals; End-to-End ML Workflow & Best Practices); MLOps-Teil größtenteils Web-Wissen (markiert)

1 Der End-to-End-Workflow (mit Rückkopplung)



2 Lernparadigmen & Aufgaben

- **Überwacht:** Klassifikation (Spam ja/nein), Regression (Hauspreis)
- **Unüberwacht:** Clustering, Dimensionsreduktion, Anomalieerkennung
- **Bestärkend:** Agent lernt aus Belohnung

Begriffe: Feature, Target, Modell, Training, Inferenz, Muster.
Datentypen: numerisch, kategorial (nominal/ordinal), Text, Bild, Zeitreihe.

4 Data Leakage

Falsch: Preprocessing auf allen Daten, dann splitten. **Richtig:** erst splitten, Transformer nur auf Train fiten.
Weitere Leaks: Ziel-Info in Features (Target Leakage), Zukunftsdaten bei Zeitreihen, Duplikate über Train/Test, Gruppen (Patienten) über Splits verteilt.
Symptom: zu gute Testergebnisse, die im Einsatz einbrechen.

6 Deployment & Monitoring (MLOps, Web)

- Modell versionieren, speichern; Inferenz-Pipeline testen; Eingaben validieren
- Latenz und Verfügbarkeit überwachen
- **Data/Concept Drift:** Verteilung ändert sich → Leistung fällt; Alarmer, regelmäßig neu trainieren
- Werkzeuge (Beispiele): Git/DVC, MLflow, CI/CD

Beispiel: Churn-Vorhersage (Kundenabwanderung) -- konkreter Ablauf

Lösung Problem: wird ein Kunde kündigen? (binäre Klassifikation, F1/AUPRC, da selten). **Daten:** CRM, Nutzung, Support-Tickets.

Vorverarbeitung: Median-Imputation, One-Hot (Region), Standardisierung; Split vor dem Fit, alles in einer Pipeline. **Modell:** Baseline LogReg, dann Random Forest/Gradient Boosting, Tuning per Random Search + Stratified CV. **Bewertung:** Test einmal (AUPRC, Confusion Matrix, Fehleranalyse). **Betrieb:** API, Drift-Monitoring, monatliches Retraining.

8 Key Takeaways

- ✓ Erst Problem und Metrik klären, dann Daten, dann Modell – Baseline zuerst
- ✓ Split vor Preprocessing, Testset einmal benutzen, alles in Pipelines
- ✓ Deployment ist nicht das Ende: überwachen und nachtrainieren

? Selbsttest: kannst du das erklären?

F1 Wofür braucht man ein Validation- und ein Testset?

A: Validation zum Wählen/Tunen, Test zur einmaligen, unverzerrten Endbewertung.

F2 Nenne zwei Arten von Data Leakage.

A: Preprocessing vor dem Split; Target-/Zukunftsinformation in Features.

F3 Was ist Concept Drift?

A: Die Beziehung zwischen Features und Ziel ändert sich im Betrieb ⇒ Modell veraltet.

„Ein starker Workflow schlägt ein geniales Modell in einer chaotischen Pipeline.“

K-Means Clustering

Ohne Labels: finde Gruppen ähnlicher Punkte. ♥

unüberwachtes Lernen: Zentroide alternierend anpassen

Merke:
Jeder Schritt senkt J -- aber nur bis zu einem lokalen Minimum!

Quelle: main_notes.pdf (Clustering), notes/cs229-notes#a.pdf

1 Idee

Gegeben Daten $\{x^{(i)}\}$ ohne Labels: teile sie in K Cluster mit Mittelpunkt (Zentroiden) $\mu_1, \dots, \mu_K$, sodass Punkte nahe an ihrem Zentroid liegen.

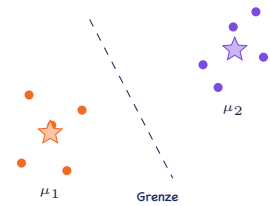
2 Zielfunktion

Summe der quadratischen Abstände zum zugewiesenen Zentroid:

$$J(c, \mu) = \sum_{i=1}^n \|x^{(i)} - \mu_{c(i)}\|^2$$

$c^{(i)} \in \{1..K\}$: Clusterzuweisung von Punkt i , μ_k : Zentroid von Cluster k , n : Anzahl Punkte. J ist nicht konvex.

3 Skizze: 2 Cluster



4 Algorithmus

K-Means

initialisiere $\mu_1, \dots, \mu_K$ (z.B. zufällige Datenpunkte)

repeat bis sich nichts mehr ändert:

Zuweisung

for jedes i : $c^{(i)} \leftarrow \arg \min_k \|x^{(i)} - \mu_k\|^2$

Update

for jedes k : $\mu_k \leftarrow$ Mittel aller $x^{(i)}$ mit $c^{(i)} = k$

Beide Schritte senken $J \Rightarrow$ Konvergenz zu einem (lokalen) Minimum.

5 Initialisierung & Wahl von K

K-Means++: nächsten Startzentroid mit Wahrscheinlichkeit $\propto \|x - \mu_{\text{nächster}}\|^2$ ziehen (verteilt Startpunkte) $\Rightarrow$ meist bessere Lösung.

Mehrere Neustarts, bestes J behalten.

K ist Hyperparameter: **Ellenbogen-Methode** (Web/Praxis):

$J(K)$ plotten, wo die Kurve knickt.

Beispiel: lokales Minimum in 1D

Daten $\{1, 2, 9, 10, 20\}$, $K = 2$. **Aufgabe** Was passiert bei Start $\mu = (1, 10)$ bzw. $\mu = (2, 20)$?

Lösung Start $(1, 10)$: $\{1, 2\} \rightarrow \mu_1$; $\{9, 10, 20\} \rightarrow \mu_2$. Update: $\mu_1 = 1.5$, $\mu_2 = 13$.

Zuweisung bleibt $\Rightarrow$ konvergiert: $J = 0.5 + 16 + 9 + 49 = 74.5$.

Start $(2, 20)$: $\{1, 2, 9, 10\} \rightarrow \mu_1$, $\{20\} \rightarrow \mu_2$; $\mu = (5.5, 20)$,

$J = 20.25 + 12.25 + 12.25 + 20.25 = 65$ **besser!**

Gleiche Daten, andere Startwerte, unterschiedliches Ergebnis.

6 Vorteile

- ✓ einfach, schnell ($O(nKd)$ pro Runde)
- ✓ skaliert auf große Daten
- ✓ gut interpretierbar

7 Grenzen

- × lokale Minima, Startabhängig
- × K muss gewählt werden
- × nur kugelförmige, ähnlich große Cluster

8 Key Takeaways

- ✓ alternieren: zuweisen / Mittel bilden
- ✓ J fällt monoton
- ✓ K-Means++ & Neustarts

9 Querverweise

K-Means ist ein "hartes" EM mit kugelförmigen Gauß-Komponenten; weicher Nachfolger: Gauß-Gemisch (nächste Seite). Anwendungen: Segmentierung, Farbquantisierung.

* K-Means in der Praxis (aus den ML Notes)

Zielfunktion heißt auch WCSS/Inertia (within-cluster sum of squares). **Ellenbogen**: WCSS über K plotten. **Silhouette** je Punkt:

$s = \frac{b-a}{\max(a,b)} \in [-1, 1]$ (a mittlerer Abstand im eigenen Cluster, b zum nächsten anderen); Mittel nahe 1 = gut getrennt. sklearn:

`n_clusters, init='k-means++', n_init (Neustarts), max_iter, tol`. Vorher skalieren; nicht geeignet für nichtkonvexe/verschieden große Cluster und kategoriale Daten. Einsatz: Kundensegmentierung, Bildkompression (Farbquantisierung), Anomalieerkennung.

? Selbsttest: kannst du das erklären?

F1 Warum konvergiert K-Means?

A: Beide Schritte senken J ; endlich viele Zuordnungen $\Rightarrow$ lokales Minimum.

F2 Was ist K-Means++?

A: Startzentroide mit Wkt. $\propto$ Abstand² zum nächsten Zentroid.

F3 Wo ist K-Means schwach?

A: Lokale Minima, K nötig, nur kugelförmige Cluster.

„Zuweisen, mitteln, wiederholen -- bis nichts mehr wackelt.“

Kein K vorgeben: baue einen Baum von Clustern und schneide ihn an einer Höhe.



Hierarchisches Clustering

Agglomerativ, Linkage & Dendrogramm

Merke:
Die Höhe im Dendrogramm ist der Abstand beim Verschmelzen

Quelle: hand-notes/ML Notes.pdf (Hierarchical Clustering); Rechenbeispiel selbst ergänzt

1 Idee

Unüberwachtes Clustern ohne feste Clusterzahl: es entsteht eine

Hierarchie (Baum, Dendrogramm). Die gewünschte Clusterzahl wählt man erst danach durch einen horizontalen Schnitt.

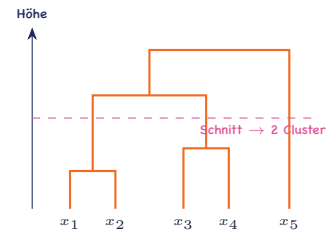
2 Agglomerativ vs. divisiv

Agglomerativ (bottom-up, Standard):

jeder Punkt ein Cluster, das nächstliegende Paar verschmelzen, bis nur noch eines bleibt.

Divisiv (top-down): alle in einem Cluster, rekursiv teilen.

3 Skizze: Dendrogramm



4 Linkage: Abstand zwischen Clustern

- **Single:** $\min_{a \in A, b \in B} d(a, b)$ – neigt zu Ketten
- **Complete:** $\max_{a, b} d(a, b)$ – kompakte Cluster
- **Average:** Mittel aller Paarabstände
- **Ward:** verschmelzt so, dass die Zunahme der Varianz innerhalb der Cluster minimal ist (ähnlich K-Means)

Abstandsmaß: Euklid, Manhattan, Kosinus...

5 Algorithmus (agglomerativ)

Agglomeratives Clustering

jeder Punkt $\leftarrow$ eigenes Cluster

berechne Distanzmatrix D

repeat bis 1 Cluster:

$(A, B) \leftarrow \arg \min D(A, B)$ # nächstes Paar

verschmelze $A \cup B$; merke Höhe = $D(A, B)$

aktualisiere D (gemäß Linkage)

schneide Dendrogramm an Höhe h oder für K Cluster

Aufwand $O(n^2)$ Speicher, bis $O(n^3)$ Zeit.

Beispiel: 10-Punkte $\{1, 2, 6, 8, 15\}$

Lösung Single Linkage: verschmelze (1, 2) bei $d = 1$; (6, 8) bei 2; $\{1, 2\} - \{6, 8\}$ bei $\min = 6 - 2 = 4$; zuletzt mit 15 bei $15 - 8 = 7$.

Complete Linkage: 1; 2; $\max = 8 - 1 = 7$; $15 - 1 = 14$.

Schnitt bei Höhe 3 (single): Cluster $\{1, 2\}$, $\{6, 8\}$, $\{15\}$ – der große Sprung $2 \rightarrow 4$ legt $K = 3$ nahe.

6 Vergleich mit K-Means

	K-Means	Hierarchisch
K	vorab	nachträglich (Schnitt)
Form	kugelförmig	flexibel (je Linkage)
Aufwand	$O(nKd)$	$O(n^2)$ und mehr
Ergebnis	flache Partition	Hierarchie

7 Vorteile

- ✓ kein K nötig, Dendrogramm
- ✓ beliebige Abstandsmaße
- ✓ gut für kleine Datensätze

8 Grenzen

- ✗ teuer bei großem n
- ✗ Verschmelzungen nicht umkehrbar
- ✗ empfindlich bei Rauschen/Ausreißern

9 Key Takeaways

- ✓ Baum statt fester Partition
- ✓ Linkage bestimmt die Clusterform
- ✓ Schnitt beim größten Höhengsprung

? Selbsttest: kannst du das erklären?

F1 Was bedeutet die Höhe im Dendrogramm?

A: Der Abstand (nach Linkage), bei dem zwei Cluster verschmolzen wurden.

F2 Single vs. Complete Linkage?

A: Single: kleinster Paarabstand (Ketten); Complete: größter (kompakte Cluster).

F3 Wie wählt man die Clusterzahl?

A: Dendrogramm dort schneiden, wo der vertikale Abstand zwischen Verschmelzungen am größten ist.

„Kein K , kein Problem -- der Baum zeigt alle möglichen Clusterungen zugleich.“

Latente Variablen: erst raten (E-Schritt), dann neu schätzen (M-Schritt). ♥

EM-Algorithmus

Gauß-Gemische & Expectation-Maximization

Merke:
Jede Iteration erhöht $\log p(x; \theta)$
-- nie weniger!

Quelle: main_notes.pdf (Mixture of Gaussians, EM), notes/cs229-notes7b.pdf, notes/cs229-notes8.pdf

1 Gauß-Gemisch (MoG)

$$p(x) = \sum_{k=1}^K \phi_k \mathcal{N}(x; \mu_k, \Sigma_k)$$

(K Komponenten, ϕ_k Mischgewicht, μ_k, Σ_k Mittel/Kovarianz von Komponente k .) Latente Variable $z \sim \text{Multinomial}(\phi)$ (welche Komponente?), $x|z=k \sim \mathcal{N}(\mu_k, \Sigma_k)$. z ist nicht beobachtet.

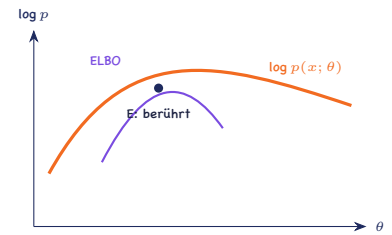
2 Jensen & ELBO

Jensen: für konkaves f gilt $\mathbb{E}[f(X)] \leq f(\mathbb{E}[X])$. Damit für jedes $Q(z)$:

$$\log p(x; \theta) \geq \sum_z Q(z) \log \frac{p(x, z; \theta)}{Q(z)}$$

(ELBO). Gleichheit, wenn $Q(z) = p(z|x; \theta)$. (x Beobachtung, z verborgene Variable, θ Parameter, Q frei wählbare Verteilung über z .)

3 Skizze: untere Schranke



4 Allgemeiner EM

EM (allgemein)

repeat bis Konvergenz:

E-Schritt

$$Q_i(z^{(i)}) \leftarrow p(z^{(i)} | x^{(i)}; \theta)$$

M-Schritt

$$\theta \leftarrow \arg \max_{\theta} \sum_i \sum_z Q_i(z) \log \frac{p(x^{(i)}, z; \theta)}{Q_i(z)}$$

E: ELBO scharf machen (Schranke berührt). M: ELBO in θ hochschieben $\Rightarrow \log p$ steigt monoton.

5 EM für Gauß-Gemische

EM für MoG

repeat:

E: Verantwortlichkeit $\gamma_{ik} = P(z_i=k|x_i)$

$$\gamma_{ik} \leftarrow \frac{\phi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_j \phi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}$$

M: gewichtete MLE

$$\phi_k \leftarrow \frac{1}{n} \sum_i \gamma_{ik}$$

$$\mu_k \leftarrow \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}$$

$$\Sigma_k \leftarrow \frac{\sum_i \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_i \gamma_{ik}}$$

Beispiel: eine EM-Iteration in 1D

Daten $x = \{0, 1, 4\}$, $\mu = (0, 4)$, $\sigma = 1$, $\phi = (0.5, 0.5)$.

Lösung E: $\gamma_{\cdot 1} = (0.9997, 0.982, 0.0003)$ (z. B. $x=1$:

$$\frac{\mathcal{N}(1; 0, 1)}{\mathcal{N}(1; 0, 1) + \mathcal{N}(1; 4, 1)} = \frac{0.242}{0.242 + 0.0044}$$

M: $\sum \gamma_{\cdot 1} = 1.982 \Rightarrow \phi_1 = 0.66$, $\mu_1 = \frac{0.982 + 0.0012}{1.982} \approx 0.50$, $\mu_2 \approx 3.95$.

Die Zentren rücken zu den Datenclustern.

6 Praxis & Konvergenz

- Nichtkonvex: nur lokales Maximum; mehrere Starts.
- Initialisierung oft mit K-Means.
- Degeneriert eine Komponente ($\Sigma_k \rightarrow 0$): regularisieren.
- K-Means = hartes EM (Verantwortlichkeit 0/1).

7 Vorteile

- ✓ weiche Zuordnung, Wahrscheinlichkeiten
- ✓ allgemein für latente Variablen
- ✓ monotoner Fortschritt

8 Grenzen

- ✗ lokale Optima, langsam
- ✗ K muss vorgegeben werden
- ✗ bei wenig Daten instabil

9 Key Takeaways

- ✓ E: $Q = p(z|x; \theta)$
- ✓ M: gewichtete MLE
- ✓ $\log p$ steigt monoton

? Selbsttest: kannst du das erklären?

F1 Was macht der E-Schritt?

A: $Q_i(z) = p(z|x_i; \theta)$: macht die ELBO-Schranke scharf.

F2 Warum steigt $\log p$ monoton?

A: Der M-Schritt hebt die ELBO, die stets unter $\log p$ liegt.

F3 K-Means und EM?

A: K-Means = hartes EM mit gleichen, kugelförmigen Gauß-Komponenten.

„Rate klug, schätze neu -- und wiederhole, bis das Raten nichts mehr ändert.“

Faktoren-analyse

Latentes Gauß-Modell für $n \gg m$

Viele Features, wenig Daten: erkläre die Korrelationen durch wenige latente Faktoren.

Merke:
 $x \sim \mathcal{N}(\mu, \Lambda \Lambda^\top + \Psi)$ --
Korrelation ohne volle
Kovarianzmatrix

Quelle: notes/cs229-notes9.pdf (Factor Analysis), section/gaussians.pdf, section/more_on_gaussians.pdf

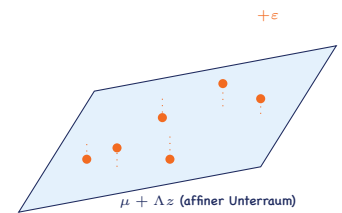
1 Problem: $n \gg m$

Mehr Features als Beispiele: die MLE-Kovarianz $\hat{\Sigma}$ hat Rang $\leq m - 1 < n$, ist **singulär**, $\hat{\Sigma}^{-1}$ existiert nicht $\Rightarrow$ Gauß-Dichte nicht berechenbar.

2 Einschränkungen von Σ

Diagonal: $\Sigma_{jj} = \frac{1}{m} \sum_i (x_j^{(i)} - \mu_j)^2$, nur Achsen-Ellipsen.
Kugel: $\Sigma = \sigma^2 I$.
Beide nicht singulär, aber keine Korrelationen. FA: Kompromiss.

3 Skizze: Ebene + Rauschen



4 Das Modell

$z \in \mathbb{R}^k$ latent ($k \ll n$; n Merkmale, Λ Ladungsmatrix, Ψ diagonale Rauschkovarianz):
 $z \sim \mathcal{N}(0, I)$, $x|z \sim \mathcal{N}(\mu + \Lambda z, \Psi)$
äquivalent $x = \mu + \Lambda z + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \Psi)$, Ψ diagonal, $z \perp \varepsilon$.
Randverteilung:
 $x \sim \mathcal{N}(\mu, \Lambda \Lambda^\top + \Psi)$
 $\Lambda \in \mathbb{R}^{n \times k}$ Ladungen: $O(nk)$ statt $O(n^2)$ Parameter.

5 Gauß-Formeln (Marginal/Bedingt)

Für $x = (x_1, x_2) \sim \mathcal{N}(\mu, \Sigma)$ (in zwei Blöcke zerlegt; Σ_{12} Kovarianz zwischen den Blöcken) gilt: $x_1 \sim \mathcal{N}(\mu_1, \Sigma_{11})$ und
 $\mu_{1|2} = \mu_1 + \Sigma_{12} \Sigma_{22}^{-1} (x_2 - \mu_2)$
 $\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12} \Sigma_{22}^{-1} \Sigma_{21}$
Diese Formeln liefern auch die GP-Vorhersage.

6 EM für Faktorenanalyse

```
</> EM für FA
# keine geschlossene MLE  $\Rightarrow$  EM
repeat:
  # E: Posterior über  $z$ 
   $\mu_{z|x} \leftarrow \Lambda^\top (\Lambda \Lambda^\top + \Psi)^{-1} (x_i - \mu)$ 
   $\Sigma_{z|x} \leftarrow I - \Lambda^\top (\Lambda \Lambda^\top + \Psi)^{-1} \Lambda$ 
  # M:  $\Lambda, \Psi, \mu$  aus  $\mathbb{E}[z|x], \mathbb{E}[zz^\top|x]$ 
```

Beispiel: 1 Faktor, 2 Features

$\Lambda = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}$, $\Psi = \text{diag}(0.1, 0.2)$.

Lösung $\Sigma = \Lambda \Lambda^\top + \Psi = \begin{pmatrix} 1.1 & 0.5 \\ 0.5 & 0.45 \end{pmatrix}$.

Korrelation $\rho = \frac{0.5}{\sqrt{1.1 \cdot 0.45}} \approx \mathbf{0.71}$ – allein durch den gemeinsamen Faktor.

Parameterzahl, $n = 100, k = 5$: volle Σ : 5050; FA: $nk + n = \mathbf{600}$.

7 Bezug zu PCA

FA ist probabilistisch (Rauschen Ψ , EM).
PCA entspricht dem Grenzfall $\Psi = \sigma^2 I$, $\sigma^2 \rightarrow 0$ und ist eine direkte Eigenvektor-Lösung.

8 Vorteile

- ✓ funktioniert bei $n \gg m$
- ✓ Korrelationen modelliert
- ✓ probabilistisch, EM

9 Grenzen

- ✗ Lösung nur bis auf Rotation eindeutig
- ✗ EM: lokale Optima
- ✗ k muss gewählt werden

10 Key Takeaways

- ✓ $x = \mu + \Lambda z + \varepsilon$, $\Sigma = \Lambda \Lambda^\top + \Psi$
- ✓ Formeln für Marginale/Bedingte von Gauß-Verteilungen
- ✓ Lernen per EM (E: Posterior $z|x$)

? Selbsttest: kannst du das erklären?

F1 Warum Faktorenanalyse?

A: Bei $n \gg m$ ist $\hat{\Sigma}$ singulär; FA braucht nur $nk + n$ Parameter.

F2 Randverteilung von x ?

A: $\mathcal{N}(\mu, \Lambda \Lambda^\top + \Psi)$.

F3 FA vs. PCA?

A: FA probabilistisch (Rauschen Ψ , EM);
PCA = Grenzfall $\Psi = \sigma^2 I$, $\sigma^2 \rightarrow 0$.

„Wenig Faktoren, viele Beobachtungen -- die Korrelation hat eine Ursache.“

Sequenzen: die Zustände sind versteckt, sichtbar sind nur die Ausgaben. ♥

Hidden Markov Models

Zeitreihen mit latenten Zuständen

Merke:
Drei Fragen: Likelihood -- Decoding -- Learning

Quelle: section/cs229-hmm.pdf (Daniel Ramage, CS229 Section Notes)

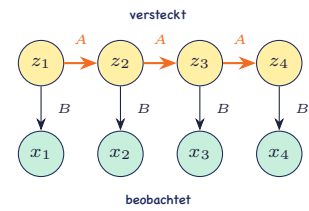
1 Markov-Modell

Zwei Annahmen: **Limited Horizon**
 $P(z_t | z_{t-1}, \dots, z_1) = P(z_t | z_{t-1})$ und
Stationarität. Übergangsmatrix
 $A_{ij} = P(z_t = s_j | z_{t-1} = s_i)$;
 Sequenzwahrscheinlichkeit
 $P(\vec{z}) = \prod_t A_{z_{t-1} z_t}$.

2 Hidden Markov Model

Zustände z_t unbeobachtet, sichtbar ist
 $x_t \in V$ mit **Emissionsmatrix**
 $B_{jk} = P(x_t = v_k | z_t = s_j)$ (Output
 Independence). Beispiel: Wetter
 (versteckt) $\rightarrow$ Eiscreme-Konsum
 (sichtbar).

3 Skizze: Kette



4 Forward-Prozedur (Frage 1)

(π_i Startwahrscheinlichkeit von Zustand i , A_{ij} Übergang, $B_{j,x}$ Emission, T Länge, $|S|$ Zustandszahl.) $\alpha_i(t) = P(x_1..x_t, z_t = s_i)$;
 statt $|S|^T$ Pfade nur $O(|S|^2 T)$:

</> Forward

```

 $\alpha_i(1) \leftarrow \pi_i B_{i,x_1}$ 
for  $t = 2..T$ , for  $j$ :
   $\alpha_j(t) \leftarrow (\sum_i \alpha_i(t-1) A_{ij}) B_{j,x_t}$ 
return  $P(\vec{x}) = \sum_i \alpha_i(T)$ 
  
```

Analog: Backward-Prozedur $\beta_i(t)$.

5 Viterbi (Frage 2)

Wie Forward, aber Maximum statt Summe; Vorgänger merken:

</> Viterbi

```

 $\delta_i(1) \leftarrow \pi_i B_{i,x_1}$ 
for  $t = 2..T$ , for  $j$ :
   $\delta_j(t) \leftarrow \max_i \delta_i(t-1) A_{ij} \cdot B_{j,x_t}$ 
   $\psi_j(t) \leftarrow \arg \max_i \delta_i(t-1) A_{ij}$ 
 $z_T \leftarrow \arg \max_j \delta_j(T)$ 
for  $t = T-1..1$ :  $z_t \leftarrow \psi_{z_{t+1}}(t+1)$  # Backtracking
  
```

6 Baum-Welch (Frage 3)

Lernen von A, B ist EM (nichtkonvex, mehrere Starts, Smoothing gegen Nullen).

</> Baum-Welch

```

repeat bis Konvergenz:
  # E: Forward + Backward
   $\gamma_t(i, j) \propto \alpha_i(t) A_{ij} B_{j,x_{t+1}} \beta_j(t+1)$ 
  # M: erwartete Häufigkeiten
   $A_{ij} \leftarrow \frac{\sum_t \gamma_t(i, j)}{\sum_j \sum_t \gamma_t(i, j)}$ 
   $B_{jk} \leftarrow \frac{\sum_i \sum_{t: x_t = v_k} \gamma_t(i, j)}{\sum_i \sum_t \gamma_t(i, j)}$ 
  
```

Beispiel: Heiß/Kalt, $x = (3, 1)$ Eisportionen

$\pi = (0.5, 0.5)$, $A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$; $B_H(1, 2, 3) = (0.2, 0.4, 0.4)$,
 $B_C = (0.5, 0.4, 0.1)$.

Lösung Forward: $\alpha_H(1) = 0.2$, $\alpha_C(1) = 0.05$.

$\alpha_H(2) = (0.2 \cdot 0.7 + 0.05 \cdot 0.4) \cdot 0.2 = 0.032$,

$\alpha_C(2) = (0.2 \cdot 0.3 + 0.05 \cdot 0.6) \cdot 0.5 = 0.045$

$P(x) = 0.032 + 0.045 = \mathbf{0.077}$.

Viterbi: $\delta_C(2) = \max(0.2 \cdot 0.3, 0.05 \cdot 0.6) \cdot 0.5 = 0.03$ (Vorgänger H) $\Rightarrow$ Pfad **(H,C)**.

7 Anwendungen

- Spracherkennung
- Part-of-Speech-Tagging
- Bioinformatik (Gene, Proteine)

8 Grenzen

- × Markov-Annahme (Gedächtnis 1)
- × lokale Optima bei EM
- × diskrete Zustände

9 Key Takeaways

- ✓ Forward: Likelihood, $O(|S|^2 T)$
- ✓ Viterbi: bester Pfad
- ✓ Baum-Welch: EM für A, B

? Selbsttest: kannst du das erklären?

F1 Was leistet die Forward-Prozedur?

A: $P(\vec{x})$ in $O(|S|^2 T)$ statt $|S|^T$ Pfaden.

F2 Viterbi vs. Forward?

A: Maximum statt Summe + Backtracking: wahrscheinlichste Zustandsfolge.

F3 Wie lernt man A, B ?

A: Baum-Welch (EM): erwartete Übergangs-/Emissionshäufigkeiten.

„Aus dem, was du siehst, schließt du auf das, was verborgen bleibt.“

Hauptkomponenten-analyse (PCA)

Dimensionsreduktion durch Eigenvektoren

Finde die Richtungen größter Varianz und projiziere die Daten darauf. ❤️

Merke:
PCA = Eigenvektoren der Kovarianzmatrix

Quelle: main_notes.pdf (PCA), notes/cs229-notes10.pdf

1 Idee

Daten $x \in \mathbb{R}^n$ liegen ungefähr in einem k -dimensionalen Unterraum ($k \ll n$). PCA findet ihn direkt – ohne Iteration, nur mit einer **Eigenwertzerlegung**. Nutzen: Kompression, Visualisierung, Rauschunterdrückung.

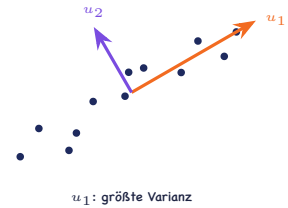
2 Optimierungsproblem

Wähle orthonormale Richtungen $U = [u_1 \dots u_k]$, die die projizierte Varianz maximieren:

$$\max_{U^T U = I_k} \text{tr}(U^T \hat{\Sigma} U)$$

Lösung: die k Eigenvektoren von $\hat{\Sigma}$ zu den größten Eigenwerten. ($\hat{\Sigma}$ Kovarianzmatrix, u_j Richtungen, tr Spur, I_k Einheitsmatrix.)

3 Skizze: Hauptachsen



4 Algorithmus

PCA
 $\bar{x} \leftarrow \frac{1}{n} \sum_i x_i$; $x_i \leftarrow x_i - \bar{x}$ # zentrieren
 $\hat{\Sigma} \leftarrow \frac{1}{n} X^T X$
 $(\lambda_j, u_j) \leftarrow$ Eigenpaare von $\hat{\Sigma}$ (oder SVD von X)
 $U_k \leftarrow [u_1 \dots u_k]$ zu den k größten λ_j
 $z_i \leftarrow U_k^T x_i \in \mathbb{R}^k$ # Projektion
 $\tilde{x}_i \leftarrow U_k z_i + \bar{x}$ # Rekonstruktion

5 Wie viele Komponenten?

Anteil erklärter Varianz (λ_j Eigenwert = Varianz entlang u_j , n Dimensionen, k behaltene Komponenten):

$$\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^n \lambda_j}$$

Wähle k so, dass z. B. 90–99% erklärt sind (Scree-Plot).

Vorher skalieren, wenn Features verschiedene Einheiten haben (sonst dominiert die größte Skala).

Beispiel: 4 Punkte in 2D

Zentrierte Punkte $(2, 0), (-2, 0), (0, 1), (0, -1)$.

Lösung $\hat{\Sigma} = \frac{1}{4} \sum x x^T = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}$, Eigenwerte $\lambda_1 = 2, \lambda_2 = 0.5$, $u_1 = (1, 0)$.

Erklärte Varianz von u_1 : $\frac{2}{2.5} = 80\%$.

Projektion auf u_1 : $(2, 0) \rightarrow 2, (0, 1) \rightarrow 0$ – aus 2D wird 1D, ein Fünftel der Varianz geht verloren.

6 Bezüge

- **Faktorenanalyse**: probabilistische Variante (mit Rauschen Ψ); PCA ist der Grenzfall $\Psi = \sigma^2 I \rightarrow 0$.
- **SVD**: $X = U \Sigma V^T$; die Spalten von V sind die Hauptachsen.
- **ICA**: sucht statt maximaler Varianz Unabhängigkeit.

7 Vorteile

- ✓ geschlossene Lösung
- ✓ unüberwacht, schnell
- ✓ nützlich vor anderen Verfahren

8 Grenzen

- ✗ nur lineare Struktur
- ✗ Varianz $\neq$ Relevanz für die Aufgabe
- ✗ skalenempfindlich

9 Key Takeaways

- ✓ zentrieren, Kovarianz, Eigenvektoren
- ✓ Projektion $z = U_k^T x$
- ✓ Varianzanteil wählt k

* PCA in Formeln & Praxis (aus den ML Notes)

Schritte: $Z = \frac{X - \mu}{\sigma}$ (standardisieren), $C = \frac{1}{n-1} Z^T Z$, Eigenzerlegung $C v_i = \lambda_i v_i$, sortieren, $V_k = [v_1 \dots v_k]$, $Z_k = Z V_k$. Erklärte Varianz: $\text{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j}$, kumulativ $\text{CEV}_k = \sum_{i \leq k} \text{EVR}_i$ (Scree-Plot: k am Ellenbogen). Vorteile: weniger Dimensionen, entfernt Multikollinearität, beschleunigt Modelle, Visualisierung (2D/3D). Grenzen: Komponenten schwer interpretierbar, nur linear, skalenempfindlich. Code: Vertiefungsseite.

? Selbsttest: kannst du das erklären?

F1 Was maximiert PCA?

A: Projizierte Varianz $\text{tr}(U^T \hat{\Sigma} U)$ mit $U^T U = I$.

F2 Wie viele Komponenten?

A: So viele, dass z. B. 90–99% der Varianz erklärt sind.

F3 Warum vorher skalieren?

A: Sonst dominiert das Feature mit der größten Skala.

„Behalte die Richtungen, in denen die Daten etwas zu erzählen haben.“

Cocktailparty: aus Mischungen die ursprünglichen Sprecher zurückgewinnen. ❤️

Independent Component Analysis

Blind Source Separation

Merke:
Quellen dürfen nicht gaußverteilt sein!

Quelle: main_notes.pdf (ICA), notes/cs229-notes11.pdf

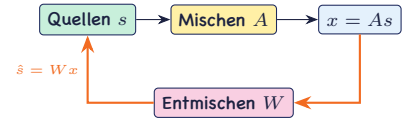
1 Problem

n Sprecher (Quellen $s \in \mathbb{R}^n$), n Mikrofone messen Mischungen $x = As$.
Gesucht: Entmischungsmatrix $W = A^{-1}$, sodass $s = Wx$ – ganz ohne Kenntnis von A oder s .

2 Ambiguitäten

- **Skalierung:** $A \text{diag}(c) \cdot \text{diag}(1/c) s$ erzeugt dasselbe x
- **Permutation:** Reihenfolge der Quellen unbestimmt
- **Gauß:** gaußsche Quellen sind nie identifizierbar (rotationssymmetrisch)

3 Skizze: Mischen & Entmischen



4 Likelihood & Regel

Mit unabhängigen Quellen $p(s) = \prod_j p_s(s_j)$ (w_j j -te Zeile von W , $|W|$ Determinante) gilt

$$p_x(x) = \prod_{j=1}^n p_s(w_j^T x) \cdot |W|$$

Wähle p_s mit CDF $g(s) = 1/(1 + e^{-s})$ (super-gaußsch).
Stochastischer Gradient-Aufstieg auf $\log p_x$ (α Lernrate):

$$W \leftarrow W + \alpha \left((1 - 2g(Wx)) x^T + (W^T)^{-1} \right)$$

5 Algorithmus

```

</> ICA (Gradient-Aufstieg)
W ← I (oder zufällig)
repeat bis Konvergenz:
  for Beispiel  $x_i$  in zufälliger Reihenfolge:
    W ← W +  $\alpha((1 - 2g(Wx_i))x_i^T + (W^T)^{-1})$ 
   $\hat{s}_i \leftarrow Wx_i$  # geschätzte Quellen
  
```

Beispiel: Mischen und Entmischen

$A = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, Quellen $s = (1, -1)$.

Lösung $x = As = (1 - 0.5, 0.5 - 1) = (0.5, -0.5)$.

$\det A = 0.75$, $W = A^{-1} = \frac{1}{0.75} \begin{pmatrix} 1 & -0.5 \\ -0.5 & 1 \end{pmatrix}$.

$Wx = \frac{1}{0.75} (0.5 + 0.25, -0.25 - 0.5) = (1, -1) = s$

Quellen zurückgewonnen.

6 Warum nicht gaußsch?

Bei gaußschen Quellen ist $p(x)$ rotationssymmetrisch: jede Drehung von W erklärt die Daten gleich gut $\Rightarrow$ keine eindeutige Lösung. Nur nicht-gaußsche Unabhängigkeit liefert Information.

PCA vs. ICA: PCA dekorreliert (2. Momente), ICA macht unabhängig (höhere Momente).

7 Anwendungen

- Audio: Cocktailparty
- EEG/MEG: Artefakte entfernen
- Bild-/Signalanalyse

8 Grenzen

- × Skalierung/Reihenfolge unbestimmt
- × Quellen nicht gaußsch nötig
- × Anzahl Mikrofone $\geq$ Quellen

9 Key Takeaways

- ✓ $x = As$, gesucht $W = A^{-1}$
- ✓ Unabhängigkeit + Nicht-Gauß
- ✓ SGD auf Log-Likelihood

? Selbsttest: kannst du das erklären?

F1 Was ist gesucht?

A: $W = A^{-1}$, sodass $s = Wx$ aus $x = As$.

F2 Warum keine Gauß-Quellen?

A: Rotationssymmetrie: jede Drehung erklärt die Daten gleich gut.

F3 Welche Ambiguitäten bleiben?

A: Skalierung und Permutation der Quellen.

„Aus dem Stimmengewirr die einzelne Stimme herausfiltern.“

Neue Daten erzeugen: latente Variablen mit neuronalen Netzen – VAE, GAN und Diffusion. ❤️

Generative Modelle: VAE, GAN & Co.

Von EM zu modernen generativen Netzen

Merke:
VAE = EM mit neuronalen Netzen und Reparametrisierung

Quelle: main_notes.pdf Kap. 11.5 (Variational Auto-Encoder); GAN und Diffusion als Web-Zusatz (markiert); Code: Vertiefungsseite

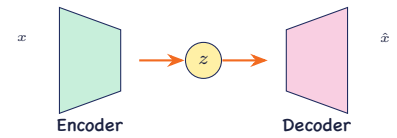
1 Idee

Diskriminative Modelle lernen $p(y|x)$; **generative** lernen $p(x)$ und können neue Beispiele sampeln. Latente Variable z (niedrigdimensional) erklärt die Daten: $z \rightarrow x$ (Decoder). Der VAE erweitert EM auf kontinuierliche z und neuronale Netze.

2 VAE-Modell (Notes)

$z \sim \mathcal{N}(0, I_k)$
 $x|z \sim \mathcal{N}(g(z; \theta), \sigma^2 I)$ (Decoder)
 $Q_i = \mathcal{N}(q(x^{(i)}; \phi), \text{diag}(v(x^{(i)}; \psi))^2)$
 Encoder q, v (Mittel und Streuung, Parameter ϕ, ψ) approximieren den intractablen Posterior $p(z|x)$; θ sind die Decoder-Parameter.

3 Skizze: Autoencoder



4 ELBO und Reparametrisierung

$$\text{ELBO} = \underbrace{\mathbb{E}_Q[\log p(x|z)]}_{\text{Rekonstruktion}} - \underbrace{\text{KL}(Q(z|x) \| p(z))}_{\text{Regularisierer}}$$

Gradient nach ϕ, ψ ist knifflig (Sampling hängt von ihnen ab) $\Rightarrow$

Reparametrisierungs-Trick: $z = q(x; \phi) + v(x; \psi) \odot \xi$ mit

$\xi \sim \mathcal{N}(0, I)$. Für Gauß: $\text{KL} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$.

5 VAE-Training

VAE-Schritt
 $(\mu, \log \sigma^2) \leftarrow \text{Encoder}(x)$
 $\xi \sim \mathcal{N}(0, I); z \leftarrow \mu + \sigma \odot \xi$
 $\hat{x} \leftarrow \text{Decoder}(z)$
 $\mathcal{L} \leftarrow \|x - \hat{x}\|^2 + \text{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, I))$
 Gradient über $\mathcal{L}$ (Backprop), Update von ϕ, ψ, θ

Sampling neuer Daten: $z \sim \mathcal{N}(0, I)$, dann Decoder.

Beispiel: KL-Term in 1D

Encoder liefert $\mu = 0.5, \sigma = 0.8$.

Lösung $\text{KL} = \frac{1}{2} (\mu^2 + \sigma^2 - 1 - \ln \sigma^2) = \frac{1}{2} (0.25 + 0.64 - 1 - \ln 0.64) = \frac{1}{2} (-0.11 + 0.446) = 0.168$ – klein, weil $\mathcal{N}(0.5, 0.8^2)$ nahe an $\mathcal{N}(0, 1)$ liegt. $\mu = 0, \sigma = 1$ ergäbe $\text{KL} = 0$.

Ausgabe des Code-Snippets (Vertiefungsseite): Rekonstruktion sinkt, KL steigt leicht – der klassische Kompromiss.

6 GAN: Spiel Generator vs. Diskriminator (Web)

$$\min_G \max_D \mathbb{E}_x \log D(x) + \mathbb{E}_z \log(1 - D(G(z)))$$

D unterscheidet echt/fake, G soll D täuschen. Abwechselnd:
 (1) D auf echten und erzeugten Daten trainieren, (2) G so ändern, dass $D(G(z))$ steigt (non-saturating: $-\log D(G(z))$).
 Probleme: instabil, Mode Collapse.

7 Diffusionsmodelle (Web)

Vorwärts (x_0 Originalbild, $\bar{\alpha}_t$ Rauschplan, ε Rauschen): nach und nach Rauschen hinzufügen, $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$.
 Rückwärts: ein Netz $\varepsilon_\theta(x_t, t)$ lernt, das Rauschen vorherzusagen (Verlust $\|\varepsilon - \varepsilon_\theta\|^2$). Sampling: aus reinem Rauschen schrittweise entrauschen. Heute Stand der Technik bei Bildern.

8 Vergleich

	VAE	GAN	Diffusion
Ziel	ELBO	Minimax	Rauschvorhersage
Training	stabil	instabil	stabil
Samples	eher unscharf	scharf	sehr gut
Sampling	schnell	schnell	langsam

9 Key Takeaways

- ✓ VAE: ELBO = Rekonstruktion – KL; Encoder = approximativer Posterior; Reparametrisierung ermöglicht Backprop durch das Sampling
- ✓ GAN: adversariales Spiel; Diffusion: iteratives Entauschen

? Selbsttest: kannst du das erklären?

F1 Warum der

Reparametrisierungs-Trick?

A: Damit der Gradient durch das Sampling fließt: Zufall steckt nur in ξ , nicht in ϕ, ψ .

F2 Was bewirkt der KL-Term?

A: Hält den Posterior nahe an $\mathcal{N}(0, I)$: glatter, samplebarer latenter Raum.

F3 Was ist Mode Collapse?

A: Der GAN-Generator erzeugt nur wenige Varianten der Daten.

„Wer die Daten erzeugen kann, hat sie verstanden.“

Leistungsfähig heißt nicht vertrauenswürdig: fünf Kritikpunkte an ML. ❤️

Kritische Perspektiven

Adversarial Examples, Interpretierbarkeit, Kosten & Ethik

Merke:
FGSM:
 $x_{\text{adv}} = x + \varepsilon \text{sign}(\nabla_x J)$

Quelle: materials/critiques-ml-aut19.pdf (C. Chute, T. Ding, A. Kurenkov, 2019) und materials/critiques-ml.pdf (2018)

1 Fünf Themen

1. Adversarial Examples
2. Interpretierbarkeit
3. Kosten: Daten & Rechenleistung
4. Schwächen der Community
5. Ethische Fragen (Fairness, Bias)

2 Adversarial Examples

Winzige, für Menschen kaum sichtbare Störungen erzwingen falsche Vorhersagen (+ε-Rauschen: Panda → Gibbon). Sie übertragen sich auf andere Architekturen und Datensätze und lassen sich physisch bauen (z. B. Verkehrsschilder).

3 Skizze: Bild + Rauschen

Bild $x + \varepsilon \text{sign}(\nabla_x J)$

„Panda“

x_{adv}

„Gibbon“

4 FGSM & Verteidigungen

Fast Gradient Sign Method: gehe in Richtung steigenden Verlusts.

`</> FGSM`

```
input: x, y, Modell θ, Schrittgröße ε # J: Verlust, xadv:  
gestörtes Bild  
g ← ∇xJ(θ, x, y)  
xadv ← x + ε · sign(g)
```

Abwehr: adversariales Training (saubere + gestörte Beispiele), Distillation mit Temperatur-Softmax. Weiterhin ein Wettrennen / offenes Problem.

5 Interpretierbarkeit (Lipton)

Zwei Bedeutungen: **algorithmische Transparenz** (Modell selbst verständlich: lineare Modelle, kleine Bäume) vs. **Post-hoc-Erklärung** (Saliency Maps, Grad-CAM, Attention, t-SNE). Fallacy: "lineare Modelle interpretierbar, neuronale Netze Blackbox" verwechselt beides. Fünf Fragen: Vertrauen, Kausalität, Übertragbarkeit, Informativität, Fairness – Metriken wie Accuracy/AUROC erfassen sie nicht (Vertiefung: Kapitel SHAP und Fairness).

Beispiel: warum lineare Modelle anfällig sind

$f(x) = w^T x$, $x \in \mathbb{R}^{1000}$, $|w_i| = 0.01$. Störung $\eta = \varepsilon \text{sign}(w)$ mit $\varepsilon = 0.1$ pro Feature.

Lösung $w^T \hat{x} = w^T x + w^T \eta$ mit

$w^T \eta = \varepsilon \|w\|_1 = 0.1 \cdot 1000 \cdot 0.01 = 1.0$.

Jedes Feature ändert sich nur um 0.1, die Ausgabe aber um 1.0 ⇒ in hohen Dimensionen summieren sich kleine Störungen (Goodfellow).

6 Kosten & Gesellschaft

- **Daten & Compute:** moderne Modelle brauchen riesige Datenmengen und Rechenleistung – Zugang und Umwelt.
- **Bias in Daten** ⇒ ungerechte Entscheidungen gegenüber unterrepräsentierten Gruppen.
- **Verantwortung:** Evaluation über reine Testfehler hinaus.

7 Was tun?

- Robustheit testen
- Erklärungen bereitstellen
- Fairness-Metriken prüfen

8 Offene Probleme

- × robuste Verteidigung
- × Kausalität statt Korrelation
- × Übertragung auf neue Verteilungen

9 Key Takeaways

- ✓ FGSM: $x + \varepsilon \text{sign}(\nabla_x J)$
- ✓ Zwei Arten von Interpretierbarkeit
- ✓ Genauigkeit ≠ Vertrauen

? Selbsttest: kannst du das erklären?

F1 Was ist FGSM?

A: $x_{\text{adv}} = x + \varepsilon \text{sign}(\nabla_x J)$: Schritt in Richtung steigenden Verlusts.

F2 Zwei Arten von Interpretierbarkeit?

A: Algorithmische Transparenz vs. Post-hoc-Erklärung.

F3 Warum sind lineare Modelle anfällig?

A: $w^T \eta = \varepsilon \|w\|_1$ wächst mit der Dimension.

„Ein Modell ist erst dann gut, wenn man ihm auch dann vertrauen kann, wenn niemand hinsieht.“

Bestärkendes Lernen (MDPs)

Markov-Entscheidungsprozesse, Value & Policy Iteration

Merke:
Bellman: $V^*(s) = \max_a [R + \gamma \sum P V^*]$

Quelle: main_notes.pdf (Reinforcement Learning), notes/cs229-notes12.pdf

1 MDP-Tupel

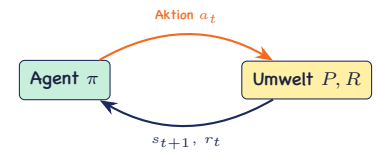
$(S, A, \{P_{sa}\}, R, \gamma)$:
 S Zustände, A Aktionen,
 $P_{sa}(s') = P(s'|s, a)$ Übergänge, $R(s, a)$
 Belohnung, $\gamma \in [0, 1)$ Diskontierung.
 $(s'$ Folgezustand, V Wert eines
 Zustands.) Ziel: **Policy** $\pi: S \rightarrow A$ mit
 maximalem erwarteten Ertrag.

2 Wertfunktion

$$V^\pi(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t R(s_t, a_t) \mid s_0 = s, \pi \right]$$

γ nahe 1: weitsichtig; nahe 0: kurzfristig. Optimal:
 $V^*(s) = \max_\pi V^\pi(s)$.

3 Skizze: Agent-Umwelt



4 Bellman-Gleichungen

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P_{s\pi(s)}(s') V^\pi(s')$$

$$V^*(s) = \max_a [R(s, a) + \gamma \sum_{s'} P_{sa}(s') V^*(s')]$$

$$\pi^*(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} P_{sa}(s') V^*(s')]$$

5 Value Iteration

```

(</> Value Iteration
V(s) ← 0 für alle s
repeat bis Konvergenz:
  for jedes s:
    V(s) ← max_a [R(s, a) + γ ∑_{s'} P_{sa}(s') V(s')]
return π(s) = arg max_a [⋯]
  
```

6 Policy Iteration

```

(</> Policy Iteration
π ← beliebig
repeat bis π stabil:
  # Evaluation: lineares System
  löse (I - γP^π)V^π = R^π
  # Verbesserung
  π(s) ← arg max_a [R + γ ∑ P_{sa} V^π]
  
```

Konvergiert in endlich vielen Schritten.

Beispiel: 2 Zustände, $\gamma = 0.9$

$R(s_1) = 0, R(s_2) = 1$. Aktionen deterministisch: bleiben oder wechseln.
Lösung Bester Plan: in s_2 bleiben $\Rightarrow V^*(s_2) = \frac{1}{1-0.9} = 10$; von s_1 nach s_2 wechseln: $V^*(s_1) = 0 + 0.9 \cdot 10 = 9$.
 Value Iteration: $V_0 = (0, 0), V_1 = (0, 1), V_2 = (0.9, 1.9), \dots \rightarrow (9, 10)$.

7 Kontinuierliche Zustände & unbekannte Dynamik

Approximiere $V(s) \approx \theta^\top \phi(s)$ und iteriere (Fitted Value Iteration):

$$y^{(i)} = R + \gamma \max_a \sum_{s'} P_{s(i)a}(s') V(s'), \quad \theta \leftarrow \arg \min_i \sum_i (\theta^\top \phi(s^{(i)}) - y^{(i)})^2$$

Unbekannte Dynamik: $\hat{P}_{sa}(s') = \frac{\#(s, a \rightarrow s')}{\#(s, a)}$ aus Daten schätzen.

8 Vorteile

- ✓ exakte Optimalität (diskret)
- ✓ Bellman: klare Rekursion
- ✓ Policy Iteration: endlich

9 Grenzen

- ✗ Fluch der Dimensionalität
- ✗ Kenntnis von P nötig (oder Schätzung)
- ✗ Diskretisierung bei kontinuierlichem S

10 Key Takeaways

- ✓ MDP (S, A, P, R, γ)
- ✓ Bellman-Optimalität
- ✓ VI: max-Update; PI: Evaluation + Verbesserung

? Selbsttest: kannst du das erklären?

F1 Bellman-Optimalität?

A: $V^*(s) = \max_a [R + \gamma \sum P_{sa} V^*]$.

F2 Value vs. Policy Iteration?

A: VI: max-Update bis Konvergenz; PI: Evaluation (lin. System) + Verbesserung.

F3 Rolle von γ ?

A: Nahe 1 weitsichtig, nahe 0 kurzfristig.

„Entscheide heute so, dass die Summe aller morgigen Belohnungen maximal wird.“

LQR, DDP & LQG

Lineare Dynamik plus quadratische Kosten ergibt eine exakte, analytische Lösung.



Regelung kontinuierlicher Systeme

Merke:
Die optimale Regel ist linear:
 $u_t = -K_t x_t$

Quelle: main_notes.pdf (LQR, DDP, LQG), notes/cs229-notes13.pdf (D. Boneh & A. Ng)

1 LQR-Setup

$$x_{t+1} = Ax_t + Bu_t + w_t$$

$$c_t = x_t^\top Q x_t + u_t^\top R u_t$$

Lineare Dynamik, quadratische Kosten ($Q \succeq 0, R \succ 0$), Rauschen

$w_t \sim \mathcal{N}(0, \Sigma_w)$. Ziel: Zustand nahe 0 mit wenig Stellenergie.

2 Riccati-Rekursion

Wertfunktion quadratisch:

$$V_t(x) = x^\top P_t x + q_t.$$

Legende: x_t Zustand, u_t Steuerung, A, B Dynamikmatrizen, Q, R Kostenmatrizen, P_t Kostenmatrix der Wertfunktion, K_t Reglerverstärkung, T Horizont.

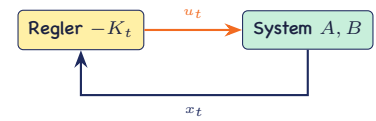
$$P_T = Q$$

$$P_t = Q + A^\top P_{t+1} A$$

$$- A^\top P_{t+1} B K_t$$

$$K_t = (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A$$

3 Skizze: Regelkreis



4 LQR-Algorithmus

Finite-Horizon LQR

$P_T \leftarrow Q$

for $t = T-1:0$: # rückwärts

$$K_t \leftarrow (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A$$

$$P_t \leftarrow Q + A^\top P_{t+1} A - A^\top P_{t+1} B K_t$$

for $t = 0:T-1$: # vorwärts

$$u_t \leftarrow -K_t x_t; \quad x_{t+1} \leftarrow A x_t + B u_t$$

5 DDP: nichtlineare Systeme

$x_{t+1} = f(x_t, u_t)$: linearisiere um eine Referenztrajektorie $(\bar{x}_t, \bar{u}_t)$:

$$f(x, u) \approx f(\bar{x}, \bar{u}) + F_x(x - \bar{x}) + F_u(u - \bar{u})$$

DDP / iterative LQR

repeat bis Konvergenz:

linearisiere f , quadrtisiere Kosten

Backward: LQR $\rightarrow K_t$

Forward: Politik anwenden $\rightarrow$ neue Trajektorie

Beispiel: skalarer LQR

$$x_{t+1} = x_t + u_t, \quad Q = R = 1, \quad P_T = 1.$$

Lösung $K = \frac{P}{1+P}$: $P_T = 1 \Rightarrow K = 0.5$; $P = 1 + 1 - \frac{1}{2} = 1.5$;

dann $K = 0.6, P = 1.6...$

Fixpunkt: $P^2 - P - 1 = 0 \Rightarrow P^* = 1.618, K^* = 0.618$.

Start $x_0 = 1$: $u_0 = -0.618, x_1 = 0.382$ Zustand schrumpft.

6 LQG: nicht beobachtbarer Zustand

Messung $y_t = Cx_t + v_t, v_t \sim \mathcal{N}(0, \Sigma_v)$. Separationsprinzip: Kalman-Filter und LQR-Regler unabhängig entwerfen.

LQG

for jeden Schritt:

$$\hat{x}_t \leftarrow \text{Kalman-Filter}(y_{1:t})$$

$$u_t \leftarrow -K_t \hat{x}_t \quad \# \text{ LQR mit Schätzung}$$

7 Vorteile

- ✓ analytisch, effizient
- ✓ lineare optimale Regel
- ✓ DDP: nichtlinear nutzbar

8 Grenzen

- ✗ nur lineare/lokal linearisierte Dynamik
- ✗ quadratische Kosten
- ✗ DDP: lokale Lösung

9 Key Takeaways

- ✓ Riccati rückwärts, Regelung vorwärts
- ✓ $u_t = -K_t x_t$
- ✓ LQG: Kalman + LQR

? Selbsttest: kannst du das erklären?

F1 Form der optimalen Policy?

A: $u_t = -K_t x_t$ (linear im Zustand).

F2 Was liefert die Riccati-Rekursion?

A: P_t rückwärts (quadratische Wertfunktion) und damit K_t .

F3 Separationsprinzip?

A: Kalman-Filter und LQR unabhängig entwerfen; $\hat{x}$ statt x nutzen.

„Wenn die Welt linear ist und die Kosten quadratisch, gibt es eine Formel statt einer Suche.“

Policy Gradient

Statt einer Wertfunktion: die Policy direkt optimieren. ❤️

REINFORCE, Baselines & Actor-Critic

Merke:
 $\nabla J = \mathbb{E} \left[\sum_t \nabla \log \pi_\theta(a_t | s_t) G_t \right]$

Quelle: main_notes.pdf (Policy Gradient / REINFORCE)

1 Direkte Parametrisierung

Policy $\pi_\theta(a|s)$ (z. B. Softmax oder Gauß).
 Ziel:

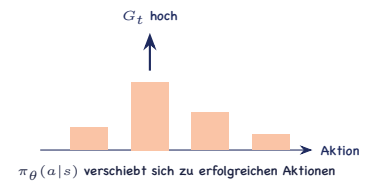
$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

(τ Episode, r_t Belohnung zur Zeit t , γ Diskontfaktor, θ Policy-Parameter.)
 Gradienten-Aufstieg auf J ; funktioniert auch bei kontinuierlichen Aktionen.

2 Log-Trick

$\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$. Damit wird der Gradient zu einem Erwartungswert, der sich per **Sampling** schätzen lässt – ohne Modell der Umwelt.

3 Skizze: Gute Aktionen verstärken



4 Policy-Gradient-Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right]$$

mit dem zukünftigen Ertrag $G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$. Intuition: Aktionen mit hohem Ertrag werden wahrscheinlicher, solche mit niedrigem unwahrscheinlicher.

5 REINFORCE

`</> REINFORCE`

repeat:

Episode $\tau = (s_0, a_0, r_0, \dots)$ unter π_θ ziehen

for t : $G_t \leftarrow \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$

$g \leftarrow \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) (G_t - b(s_t))$

$\theta \leftarrow \theta + \alpha g$

Mit $b \equiv 0$: reines REINFORCE.

6 Varianzreduktion

REINFORCE hat hohe Varianz. Eine **Baseline** $b(s_t)$ senkt sie, ohne den Erwartungswert zu verzerren:

$g = \sum_t \nabla \log \pi(G_t - b(s_t))$. Typisch $b = V^\pi(s_t)$;

$A(s, a) = Q(s, a) - V(s)$ heißt Advantage und führt zu

Actor-Critic: Actor = Policy, Critic = Wertfunktion.

Beispiel: 2-armiger Bandit

$\pi(a_1) = \sigma(\theta)$, $\theta = 0 \Rightarrow \pi(a_1) = 0.5$. Belohnung: $a_1 \rightarrow 1$, $a_2 \rightarrow 0$.

Gezogen: a_1 , $G = 1$, $\alpha = 0.5$.

Lösung $\nabla_\theta \log \sigma(\theta) = 1 - \sigma(\theta) = 0.5$.

Ohne Baseline:

$\theta \leftarrow 0 + 0.5 \cdot 0.5 \cdot 1 = 0.25 \Rightarrow \pi(a_1) = \sigma(0.25) = \mathbf{0.56}$.

Mit $b = 0.5$: $\Delta\theta = 0.5 \cdot 0.5 \cdot (1 - 0.5) = 0.125$ (kleinerer, stabilerer Schritt).

7 Policy Gradient vs. Wertmethoden

	Wert-basiert	Policy Gradient
Lernt	V, Q	π_θ direkt
Aktionen	diskret	auch kontinuierlich
Varianz	niedrig	hoch
Modell	oft P nötig	modellfrei

8 Vorteile

- ✓ kontinuierliche Aktionen
- ✓ modellfrei, flexibel
- ✓ stochastische Policies

9 Grenzen

- ✗ hohe Varianz
- ✗ stichproben-ineffizient
- ✗ lokale Optima

10 Key Takeaways

- ✓ $\nabla J = \mathbb{E}[\sum \nabla \log \pi G_t]$
- ✓ Baseline gegen Varianz
- ✓ Actor-Critic kombiniert beides

? Selbsttest: kannst du das erklären?

F1 Policy-Gradient-Theorem?

A: $\nabla J = \mathbb{E}[\sum_t \nabla \log \pi(a_t | s_t) G_t]$.

F2 Wozu eine Baseline?

A: Senkt die Varianz, ohne den Erwartungswert zu verzerren.

F3 Vorteil ggü. Wertmethoden?

A: Kontinuierliche Aktionen, modellfrei; Nachteil: hohe Varianz.

„Verstärke, was funktioniert hat -- und probiere trotzdem Neues.“

Welcher Algorithmus passt wann? Alle Methoden der Vorlesung im Vergleich. 

Algorithmen-Spickzettel

Aufgabe, Idee, Lösungsverfahren, Stärken & Grenzen

Merke:
Frage dich immer: Modell?
Verlust? Optimierung?

A Vergleichstabelle

Methode	Aufgabe	Schlüsselidee	Lösung	Stärke	Grenze
Lineare Regression	Regression	Least Squares, $\theta^\top x$	Normalgl., (S)GD	einfach, interpretierbar	nur linear, Ausreißer
Logistische Regr.	Klassifikation	Sigmoid + MLE	GD, Newton	Wahrscheinlichkeiten, konvex	nur lineare Grenze
GLM	beides	ExpFamilie, $\eta = \theta^\top x$	GD, Newton	einheitliche Theorie	Verteilungsannahme
GDA	Klassifikation	Gauß $p(x y)$ + Bayes	geschl. MLE	wenig Daten	starke Annahme
Naive Bayes	Klassifikation	Unabhängigkeit + Laplace	Zählen	schnell, skalierbar	„naive“ Annahme
SVM	Klassifikation	max. Margin + Kernel	QP, SMO	Kernel, sparse	teuer bei großem n
Perceptron	Klassifikation	Update bei Fehler	online	Schranke $(D/\gamma)^2$	nur trennbar
k-NN	beides	Mehrheit/Mittel der k Nachbarn	kein Training	einfach, nichtlinear	langsam, Fluch der Dimension
Decision Tree	beides	rekursive Splits	greedy, Pruning	interpretierbar	hohe Varianz
Random Forest	beides	Bagging + Feature-Zufall	Bäume mitteln	robust, wenig Tuning	schwer interpretierbar
Boosting	beides	sequentielle schwache Lerner	forward stagewise	senkt Bias	Overfitting bei großem M
Gaussian Process	Regression	Prior über Funktionen	Konditionierung	Unsicherheit	$O(n^3)$
CNN / RNN-LSTM	Bild / Sequenz	Faltung+Sharing / Gates	Backprop + Adam	nutzt Datenstruktur	datenhungrig
Neuronales Netz	beides	tiefe Nichtlinearität	Backprop + SGD	universell	datenhungrig, nichtkonvex
K-Means	Clustering	Zentroide	alternieren	einfach, schnell	lokale Optima, K nötig
Hierarchisches Clustering	Clustering	Baum aus Verschmelzungen	Linkage, Dendrogramm	kein K nötig	$O(n^2)$, nicht umkehrbar
EM / Gauß-Gemisch	Dichte, Cluster	latente Variable, ELBO	E/M-Schritt	weiche Zuordnung	lokale Optima
Faktorenanalyse	Dim.-Reduktion	latente Faktoren	EM	funktioniert bei $n \gg m$	Rotation nicht eindeutig
PCA	Dim.-Reduktion	max. Varianz	Eigenvektoren	geschlossen, schnell	nur linear
ICA	Quellentrennung	Unabhängigkeit, Nicht-Gauß	SGD auf Likelihood	Cocktailparty	Skalierung/Permutation
HMM	Sequenzen	latente Zustandskette	Forward, Viterbi, Baum-Welch	Zeitreihen	Markov-Annahme
VAE / GAN / Diffusion	Generieren	latente Variable, Minimax, Entrauschen	ELBO-SGD, adversarial	neue Samples	instabil / teuer
Transformer / SSL	Repräsentation	Self-Attention, Next-Token	SGD	skaliert mit Daten	sehr teuer
Value / Policy Iteration	RL (diskret)	Bellman-Gleichung	dyn. Programmierung	exakt	Fluch der Dimension
LQR / DDP / LQG	Regelung	lin. Dynamik, quadr. Kosten	Riccati	analytisch	Linearität nötig
REINFORCE	RL (allgemein)	Policy Gradient	Sampling + GD	flexibel	hohe Varianz

B Faustregeln zur Wahl

- Wenig Daten, Gauß-artig → GDA / regularisiert linear
- Tabellarische Daten, viel Rauschen → Random Forest / Boosting
- Bilder, Text, Audio → neuronale Netze
- Unsicherheit wichtig, kleines n → Gaussian Process
- Struktur ohne Labels → Clustering / PCA

C Farblegende

überwacht Deep Learning unüberwacht bestärkend

Lesetipp: für jede Zeile die drei Fragen beantworten – Modell (Hypothese), Verlust (Ziel), Optimierung (Verfahren).

„Kein Algorithmus gewinnt immer -- gute Praxis heißt, die Annahmen zu kennen.“

Formel-Spickzettel

Kompakte Sammlung für die Klausurvorbereitung

Merke:
Formeln verstehen, nicht nur auswendig lernen!

1 Lineare Regression

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

$$\theta \leftarrow \theta + \alpha(y - h_{\theta}(x))x$$

$$\text{LWLR: } w^{(i)} = e^{-\|x^{(i)} - x\|^2 / 2\tau^2}$$

2 Logistisch / Softmax

$$h = \frac{1}{1 + e^{-\theta^T x}}, \quad \theta \leftarrow \theta + \alpha(y - h)x$$

$$P(k|x) = \frac{e^{\theta_k^T x}}{\sum_j e^{\theta_j^T x}}$$

$$\text{Newton: } \theta \leftarrow \theta - H^{-1} \nabla \ell$$

3 GLM

$$p(y; \eta) = b(\eta) e^{\eta^T T(y) - a(\eta)}$$

$$h(x) = \mathbb{E}[T(y)|x], \quad \eta = \theta^T x$$

$$\text{Bernoulli: } \phi = \frac{1}{1 + e^{-\eta}}; \text{ Poisson: } \lambda = e^{\eta}$$

4 Generativ

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

$$\text{GDA: } x|y \sim \mathcal{N}(\mu_y, \Sigma)$$

$$\text{NB: } p(x|y) = \prod_j p(x_j|y), \quad \hat{\phi} = \frac{1 + \#}{2 + \#}$$

5 SVM & Kernel

$$\min \frac{1}{2} \|w\|^2 \text{ s.t. } y_i(w^T x_i + b) \geq 1$$

$$f(x) = \sum_i \alpha_i y_i K(x_i, x) + b, \quad 0 \leq \alpha_i \leq C$$

$$K = \phi^T \phi \Leftrightarrow \text{Gram PSD}; \quad \theta = \sum \alpha_i x_i$$

6 Verluste (Margin z)

$$\phi_{\log} = \log(1 + e^{-z})$$

$$\phi_{\text{hinge}} = \max(1 - z, 0)$$

$$\phi_{\text{exp}} = e^{-z}, \quad \phi_{01} = \mathbf{1}\{z \leq 0\}$$

7 Bias-Varianz & Ensemble

$$\text{MSE} = \sigma^2 + \text{Bias}^2 + \text{Var}$$

$$\text{Var}(\bar{X}) = \rho \sigma^2 + \frac{1 - \rho}{n} \sigma^2$$

$$\text{AdaBoost: } \alpha_m = \log \frac{1 - \text{err}_m}{\text{err}_m}$$

8 Lerntheorie

$$P(|\bar{Z} - \mathbb{E}\bar{Z}| \geq t) \leq 2e^{-2nt^2/(b-a)^2}$$

$$n \geq \frac{1}{2\gamma^2} \log \frac{2k}{\delta}, \quad \text{VC}_{\text{lin}} = d + 1$$

$$\text{Perceptron: Fehler} \leq (D/\gamma)^2$$

9 Regularisierung

$$J_{\lambda} = J + \frac{\lambda}{2} \|\theta\|_2^2 \quad / \quad J + \lambda \|\theta\|_1$$

$$\text{MAP: } \arg \max [\log p(y|\theta) + \log p(\theta)]$$

$$\text{Ridge: } (X^T X + \lambda I)^{-1} X^T y$$

10 Netze & Backprop

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]}, \quad a = \sigma(z)$$

$$\delta_{\ell-1} = (W_{\ell}^T \delta_{\ell}) \odot \sigma'(z_{\ell-1})$$

$$\theta \leftarrow \theta - \alpha \frac{1}{B} \sum_{i \in B} \nabla J^{(i)}$$

11 EM & Gemische

$$\text{E: } Q_i(z) = p(z|x_i; \theta)$$

$$\gamma_{ik} = \frac{\phi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_j \phi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}$$

$$\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}$$

12 PCA / FA / ICA

$$\text{PCA: } \max_{U^T U = I} \text{tr}(U^T \hat{\Sigma} U)$$

$$\text{FA: } x \sim \mathcal{N}(\mu, \Lambda \Lambda^T + \Psi)$$

$$\text{ICA:}$$

$$W \leftarrow W + \alpha((1 - 2g(Wx))x^T + (W^T)^{-1})$$

13 HMM & GP

$$\text{HMM: } \alpha_j(t) = (\sum_i \alpha_i(t-1) A_{ij}) B_{j, x_t}$$

$$\text{GP: } \mu_* = K_*(K + \sigma^2 I)^{-1} y$$

$$\Sigma_* = K_{**} - K_*(K + \sigma^2 I)^{-1} K_*^T$$

14 Metriken

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P+R}, \quad \text{Acc} = \frac{TP + TN}{n}$$

$$\text{Brier: } \frac{1}{n} \sum (\hat{p}_i - y_i)^2$$

15 Bestärkendes Lernen

$$V^*(s) = \max_a [R + \gamma \sum_{s'} P_{sa}(s') V^*(s')]$$

$$\text{LQR: } u_t = -K_t x_t, \quad \text{Riccati rückwärts}$$

$$\nabla J = \mathbb{E}[\sum_t \nabla \log \pi(G_t - b)]$$

16 Optimierer & Normalisierung

$$v \leftarrow \beta v + g, \quad \theta \leftarrow \theta - \alpha v$$

$$\text{Adam: } \theta \leftarrow \theta - \alpha \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}}$$

$$\text{BN}(z) = \gamma \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

17 VAE / Faltung

$$\text{ELBO} = \mathbb{E}_Q \log p(x|z) - \text{KL}(Q||p(z))$$

$$z = \mu + \sigma \odot \xi,$$

$$\text{KL} = \frac{1}{2} \sum (\mu^2 + \sigma^2 - 1 - \log \sigma^2)$$

$$\text{Conv-Größe} \lfloor \frac{n+2p-k}{s} \rfloor + 1$$

18 Regressionsmetriken

$$\text{MAE} = \frac{1}{n} \sum |e_i|, \quad \text{MSE} = \frac{1}{n} \sum e_i^2$$

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}$$

$$R_{\text{adj}}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

„Wer die Formeln herleiten kann, muss sie nicht auswendig lernen.“